

Regime-Conditional Mixture-of-Experts

Theory and Engineering of a Quant Research System

A personal textbook, written against the `nec_baseline` codebase

Tom Kores Lesjak
with Claude (Anthropic)

Draft — Parts I–II
July 20, 2026

Preface

This book exists to answer one question in full: *how do you build a machine-learning research system for cross-sectional return prediction that you can actually trust?* Its subject is a single concrete artifact — the `nec_baseline` package, a regime-conditional mixture-of-experts written for a Master’s thesis — but its method is general. Every chapter starts from a problem you would genuinely face at that stage of the build, develops the theory needed to solve it, derives the mathematics in full, and only then shows how the engineering falls out of the theory. Motivation always precedes mechanism.

Three running principles shape the text.

First, failure is the curriculum. The project began as a prototype that was wrong in eight distinct, instructive ways. Chapter 3 performs the autopsy; every later chapter repairs a named defect. When you meet the mixture negative log-likelihood in Part II, you will already know exactly which silent failure it exists to prevent.

Second, everything is derived. The reader is assumed to know probability, linear algebra and basic machine learning at a Master’s level, and nothing else is taken on faith: the EM algorithm’s monotonicity, the gate gradient $\pi - r$, the forward filter, the deflated Sharpe ratio — all are derived from first principles, because in this field a formula you cannot derive is a formula you cannot debug.

Third, honesty over impressiveness. Quantitative research has a structural bias toward self-deception: a leak or a lucky seed always *improves* a backtest, so errors are asymmetrically rewarded. The engineering discipline in this book — data contracts, planted-truth tests, purged evaluation, trial registries, pre-registration — is not software aesthetics. It is epistemology made executable. Where the project has a known limitation, the book says so plainly, because the defensible posture is the entire point.

How to read. The parts are designed to be read in order on a first pass. Each chapter ends with exercises — a mix of pen-and-paper derivations and hands-on tasks against the real codebase — with solutions in Appendix A. Boxes punctuate the text: *Engineering consequence* (where theory forces a code decision), *What breaks without this* (the concrete failure that occurs if the lesson is skipped), *Origins* (where the idea comes from in the literature), and *Code walk-through* (annotated excerpts from the real package). The companion reference for day-to-day use of the code is `nec_baseline/HANDBOOK.md`; this book teaches you to be the person who could have written it.

Contents

Preface	ii
I The Problem	1
1 Predicting the Cross-Section	2
1.1 The object of study	2
1.1.1 Panels	2
1.1.2 Two kinds of features	3
1.1.3 Notation	4
1.2 The scale of the signal	4
1.2.1 The information coefficient	4
1.2.2 The null distribution: skill-free ICs are large	5
1.2.3 Aggregation and the arithmetic of proof	5
1.3 Why the problem resists standard machine learning	6
1.3.1 Noise dominates: variance is the binding constraint	6
1.3.2 Nothing is stationary	7
1.3.3 Samples are deeply dependent	7
1.3.4 Errors are rewarded asymmetrically	8
1.4 What counts as a result	8
1.4.1 Distributional prediction, scored properly	8
1.4.2 Economic significance, costed	9
1.4.3 Significance that survives its own search	9
Exercises	10
2 The Case for Regimes	12
2.1 Three stylized facts	12
2.2 Mixtures generate the stylized facts	13
2.2.1 Fat tails from mixing	13
2.2.2 Clustering requires persistence	13
2.3 From marginal regimes to conditional regimes	15
2.3.1 The economics: momentum as the canonical example	16
2.3.2 The statistics: what pooling destroys	16
2.3.3 Why not just add state features?	17
2.4 The model class	18
Exercises	19

3	Autopsy of a Prototype	21
3.1	The specimen	21
3.2	The ledger	22
3.3	A taxonomy, and the shape of the cure	25
3.4	The road ahead	26
	Exercises	27
II	The Mathematics	29
4	Latent Variables and EM	30
4.1	The trouble with sums inside logs	30
4.2	Responsibilities	31
4.3	The evidence lower bound	31
4.4	EM: coordinate ascent on the bound	32
4.5	The gradient identity	33
4.6	Generalized EM, and EM as the trainer's semantics	34
4.7	What EM does not promise	35
4.7.1	Local optima	35
4.7.2	The likelihood is unbounded	35
	Exercises	36
5	Mixture of Experts	38
5.1	From mixture to conditional mixture	38
5.2	The objective, fused	39
5.3	The gate gradient	39
5.4	The expert and scale gradients	40
5.5	Prediction and uncertainty	41
5.6	The interface: log-prior versus training weights	41
	Exercises	42
6	The Routing Family	44
6.1	One interface, five mechanisms	44
6.2	Soft and uniform: the two ends	45
6.3	Hard routing done right: the hard-EM surrogate	45
6.4	Top- k : the trap	46
6.5	Gumbel-softmax: stochastic routing with a temperature	47
6.6	Load balancing, and where to measure it	48
	Exercises	49
7	Hidden Markov Dynamics	51
7.1	The model: one chain, many names	51
7.2	The forward filter, derived	51
7.3	The likelihood is the filter's by-product	52
7.4	Filtering versus smoothing: the causal line	53
7.5	Training through the filter	54
7.6	Parameterizing the transition matrix	55

7.7	Label permutation across refits	55
	Exercises	56
8	Local Optima and Symmetry	58
8.1	The landscape has $K!$ of everything	58
8.2	The symmetric stationary point	59
8.3	Why seed diversity fails	60
8.4	Breaking symmetry on purpose	60
8.5	What remains multimodal	62
	Exercises	63
A	Solutions to Exercises	64

Part I

The Problem

Chapter 1

Predicting the Cross-Section

Before any model, understand the substrate. Cross-sectional return prediction is not a hard instance of a standard machine-learning problem; it is a different problem wearing the same clothes. This chapter fixes the task, quantifies exactly how faint the signal is, and identifies the four features of the environment — noise, drift, dependence, and asymmetric error rewards — that dictate every design decision made in the rest of the book.

1.1 The object of study

1.1.1 Panels

Everything in this book lives on a *panel*: a set of entities (stocks) observed jointly over time. Let $t = 1, \dots, T$ index trading dates and let $\mathcal{U}_t$ denote the *universe* at date t — the set of names we are willing to trade and able to observe, with $N_t = |\mathcal{U}_t|$. The universe changes over time as firms list, delist, merge, and enter or leave an index; Chapter 13 is devoted to why getting $\mathcal{U}_t$ right (and not, say, using today’s universe throughout history) is a first-order correctness issue rather than a detail.

For each name $i \in \mathcal{U}_t$ we observe features and, later, a realized outcome:

Definition 1.1 (The prediction task). At each date t , given features $\mathbf{x}_{i,t}$ for every $i \in \mathcal{U}_t$, produce scores $s_{i,t} \in \mathbb{R}$. The target is the *forward return* over a horizon of h days,

$$y_{i,t} = \frac{P_{i,t+h} - P_{i,t}}{P_{i,t}},$$

where $P_{i,t}$ is the (dividend- and split-adjusted) price. The task is *cross-sectional*: what is graded at date t is the correspondence between the ranking of $\{s_{i,t}\}_{i \in \mathcal{U}_t}$ and the ranking of the realized $\{y_{i,t}\}_{i \in \mathcal{U}_t}$.

Two structural remarks, both of which will matter enormously.

First, note what the task is *not*. It is not forecasting the market: whether stocks collectively rise or fall tomorrow contributes nothing to the grade, because a constant added to every $y_{i,t}$ leaves all rankings unchanged. The time-series problem (“will the index go up?”) and the cross-sectional problem (“which names will outperform which?”) are different problems with different difficulty profiles, and this book is about the second. This is also an economic choice:

a portfolio built to be long the top-ranked names and short the bottom-ranked ones, in equal dollar amounts, is approximately immune to the market’s direction — it monetizes exactly the ranking skill being graded and nothing else. Chapter 16 makes this construction precise (quantiles, turnover, costs); Chapter 13 goes further and makes the *target* itself market-neutral by residualizing returns against a rolling market beta.

Second, the label spans time. The target $y_{i,t}$ is not knowable until date $t + h$: a “sample” dated t quietly reaches h days into the future. This banal observation has a sharp consequence — any evaluation split that puts date t in the training set and any of $t + 1, \dots, t + h$ in the test set has *leaked test-period prices into training labels*. The purging rule of Chapter 15 (drop the last h training dates before every test block) is nothing more than this paragraph turned into an inequality, but the prototype dissected in Chapter 3 is far from the only system to have missed it.

1.1.2 Two kinds of features

The package splits features into two tensors, and the split is conceptual, not cosmetic:

	shape	meaning
$x_{i,t}^{\text{seq}}$	(T_w, d_{seq})	a trailing <i>window</i> of T_w days of per-day channels (returns, ranged volatility, volume shocks, ...)
$x_{i,t}^{\text{snap}}$	$(d_{\text{snap}},)$	a <i>snapshot</i> of characteristics known at t (momentum, size, short-horizon reversal, ...)

The sequence tensor exists to be summarized by a recurrent encoder into a *state*: “what kind of market has name i been living in lately?” The snapshot tensor carries the cross-sectional characteristics that actually differentiate names on a date. Keeping them separate keeps a later question crisp — *who gets to see what?* In the architecture of Part III, the encoder’s summary drives the regime gate, while the experts read the snapshot; whether the experts should *also* see the encoder state is a genuine design decision (Decision A) with arguments on both sides, and it is decided, documented, and made switchable rather than picked silently.

Every feature carries one non-negotiable invariant:

Definition 1.2 (Point-in-time discipline). A feature value $x_{i,t}$ may depend only on information available at or before date t . The target $y_{i,t}$ is the *only* forward-looking quantity in the panel, and it never appears inside a feature tensor.

Definition 1.2 sounds too obvious to state. It is stated because violating it is the single most common way quantitative research fails, the violations are almost never obvious (a volatility estimate computed over a centered window; a rank normalization fitted on the full history; a “reserved column” in a feature matrix that turns out to be the target), and because the violation *always improves the backtest* — an asymmetry we return to in Section 1.3. In the package this definition is not prose: it is a test (`test_no_lookahead`) that shifts the raw price history and asserts which features are and are not allowed to change.

1.1.3 Notation

i, t	entity (stock) and date indices
$\mathcal{U}_t, N_t$	universe at t and its size
h	label horizon in days
$y_{i,t}$	forward return over $(t, t + h]$
$\mathbf{x}_{i,t}^{\text{seq}}, \mathbf{x}_{i,t}^{\text{snap}}$	sequence window and snapshot features
$s_{i,t}$	model score for name i at date t
K	number of regimes / experts
$S_t \in \{1, \dots, K\}$	latent regime state
π_k, r_k	prior and posterior (responsibility) regime weights
$\mathcal{N}(y; \mu, \sigma^2)$	Gaussian density in y
$\text{IC}_t, \bar{\text{IC}}, \text{ICIR}$	per-date rank correlation and its aggregates

1.2 The scale of the signal

The defining fact of this problem — the one from which capacity control, evaluation protocol, and multiplicity discipline all follow — is that the predictable component of the cross-section is *tiny*. This section quantifies “tiny.”

1.2.1 The information coefficient

Definition 1.3 (Rank information coefficient). Fix a date t with N scoreable names. Let $\rho_i^{(s)}$ and $\rho_i^{(y)}$ be the within-date ranks of the scores and of the realized targets. The (rank) information coefficient at t is the Pearson correlation of the rank vectors,

$$\text{IC}_t = \frac{\sum_i (\rho_i^{(s)} - \bar{\rho})(\rho_i^{(y)} - \bar{\rho})}{\sqrt{\sum_i (\rho_i^{(s)} - \bar{\rho})^2} \sqrt{\sum_i (\rho_i^{(y)} - \bar{\rho})^2}}, \quad \bar{\rho} = \frac{N+1}{2},$$

i.e. the Spearman correlation between scores and outcomes across the date’s cross-section.

Ranks rather than raw values for two reasons: returns are heavy-tailed (Chapter 2), so a Pearson correlation on raw returns can be dominated by a single outlier; and the trading construction we ultimately grade against (sort names, go long the top, short the bottom) only consumes the ranking anyway.

What is a *good* IC? For daily-horizon equity signals, a persistent mean IC of 0.02–0.05 is respectable professional work; 0.10 would be exceptional. It is worth internalizing what such numbers mean in terms of explained variance. For a correlation ρ between score and outcome, the fraction of cross-sectional variance explained by the best linear predictor is $R^2 = \rho^2$. Thus:

mean IC	R^2	share of outcome variance that is noise
0.02	0.0004	99.96%
0.05	0.0025	99.75%
0.10	0.01	99.00%

A model at the top of the realistic range explains *one quarter of one percent* of what it is asked to predict. Machine-learning intuitions imported from vision or language — where a

model failing to explain 20% of the variance is considered weak — are not merely unhelpful here; they are actively misleading about model capacity, about evaluation, and about what an impressive-looking result probably is (Section 1.3).

1.2.2 The null distribution: skill-free ICs are large

The second half of the calibration is how big ICs are when there is *no* skill at all.

Proposition 1.4 (Null variance of the rank IC). *Let the score ranking be independent of the outcome ranking (zero skill), with no ties. Then*

$$\mathbb{E}[\text{IC}_t] = 0, \quad \text{Var}(\text{IC}_t) = \frac{1}{N-1}.$$

Proof. Condition on the outcome ranks; by independence the score ranks are then a uniformly random permutation Π of $\{1, \dots, N\}$. Write the IC as $\text{IC} = \sum_i a_i b_{\Pi(i)}$ where $a_i = (i - \bar{\rho}) / \sqrt{\sum_j (j - \bar{\rho})^2}$ and likewise b ; both vectors are centered ($\sum a_i = 0$) and normalized ($\sum a_i^2 = 1$). Unbiasedness is immediate: $\mathbb{E}[b_{\Pi(i)}] = \frac{1}{N} \sum_j b_j = 0$. For the variance, using $\mathbb{E}[b_{\Pi(i)}^2] = \frac{1}{N} \sum_j b_j^2 = \frac{1}{N}$ and, for $i \neq i'$, $\mathbb{E}[b_{\Pi(i)} b_{\Pi(i')}] = \frac{1}{N(N-1)} \sum_{j \neq j'} b_j b_{j'} = \frac{0 - \sum_j b_j^2}{N(N-1)} = -\frac{1}{N(N-1)}$:

$$\text{Var}(\text{IC}) = \sum_i a_i^2 \mathbb{E}[b_{\Pi(i)}^2] + \sum_{i \neq i'} a_i a_{i'} \mathbb{E}[b_{\Pi(i)} b_{\Pi(i')}] = \frac{1}{N} - \underbrace{\left(\sum_{i \neq i'} a_i a_{i'} \right)}_{=0-1} \frac{1}{N(N-1)} = \frac{1}{N} + \frac{1}{N(N-1)} = \frac{1}{N-1}.$$

□

So with $N = 100$ names per date, a *zero-skill* model produces daily ICs with standard deviation $1/\sqrt{99} \approx 0.10$ — twice the size of a genuinely good signal's mean. Figure 1.1 shows the simulated null: on roughly a third of all dates, pure noise scores $|\text{IC}_t| > 0.10$. Single-date (and single-month) performance is close to meaningless, and any evaluation must aggregate.

1.2.3 Aggregation and the arithmetic of proof

Average the daily series: $\overline{\text{IC}} = \frac{1}{T} \sum_t \text{IC}_t$. If the daily ICs were independent with standard deviation σ_{IC} , the standard error of the mean would be $\sigma_{\text{IC}}/\sqrt{T}$, giving the t -statistic

$$t = \frac{\overline{\text{IC}}}{\sigma_{\text{IC}}/\sqrt{T}} = \underbrace{\frac{\overline{\text{IC}}}{\sigma_{\text{IC}}}}_{\text{ICIR}} \cdot \sqrt{T},$$

where the ratio ICIR — the per-date information ratio of the IC series — measures the signal's *consistency*. This immediately yields the book's first piece of experimental arithmetic: the number of test dates needed before a true mean IC is statistically distinguishable from zero at level t^* is

$$T \geq \left(\frac{t^* \sigma_{\text{IC}}}{\overline{\text{IC}}} \right)^2.$$

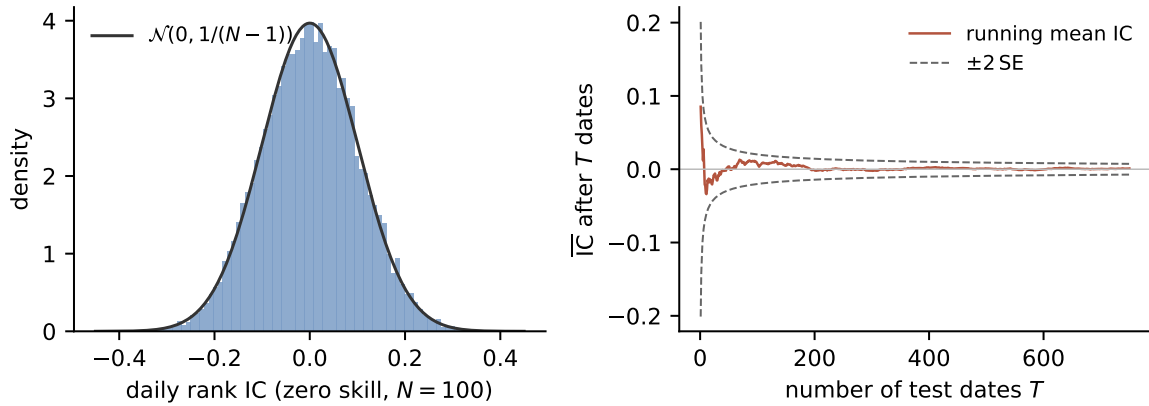


Figure 1.1: Zero skill produces large ICs. Left: the simulated distribution of the daily rank IC for a signal with no predictive content ($N = 100$ names), with the $\mathcal{N}(0, 1/(N-1))$ overlay of Proposition 1.4. Right: the running mean IC of one zero-skill strategy over 750 test dates, against ± 2 standard-error bands. Early stretches of apparent skill (or anti-skill) of $|\overline{\text{IC}}| \approx 0.03$ — the size of a genuinely good signal — are routine and mean nothing.

Example 1.5. A true $\overline{\text{IC}} = 0.03$ with realistic daily dispersion $\sigma_{\text{IC}} = 0.15$ (the null’s 0.10 plus regime-driven variation) needs $T \geq (2 \times 0.15/0.03)^2 = 100$ test dates for $t = 2$, and 225 dates for $t = 3$ — roughly five and eleven months of daily evaluation respectively, *if the daily ICs were independent*. They are not: with an h -day label, adjacent dates share $h - 1$ days of the outcome window, which positively correlates the IC series and *reduces* the effective T . Every “significant” claim in this project is built to survive this arithmetic; most published claims in the field would not.

Engineering consequence — the evaluation harness

This arithmetic is why the package’s walk-forward harness (`nec_moe/evaluation.py`) pools ICs *across all test dates of all folds* before summarizing, reports $\overline{\text{IC}}$, σ_{IC} , ICIR and $t = \text{ICIR}\sqrt{T}$ together rather than any of them alone, and why the label overlap of Example 1.5 resurfaces twice: as the purging rule (train/test separation must respect the h -day label span) and as a caution against taking the nominal t -statistic at face value when $h > 1$.

1.3 Why the problem resists standard machine learning

Four properties of this environment, in increasing order of subtlety. They are worth naming precisely, because each one dictates a specific piece of the system built in this book.

1.3.1 Noise dominates: variance is the binding constraint

With R^2 of order 10^{-3} , the mean-squared-error decomposition

$$\mathbb{E}[(\hat{f}(x) - f(x))^2] = \underbrace{\text{bias}^2}_{\text{model too simple}} + \underbrace{\text{variance}}_{\text{model too responsive to noise}}$$

is wildly lopsided: the function f being estimated is minuscule, while the noise available for a flexible model to memorize is everything else. A model class with the capacity to represent rich structure will, on a finite panel, mostly represent noise. Concretely: a panel of $N = 500$ names over ten years ($T \approx 2,500$) has 1.25 million rows, which sounds like plenty — until Section 1.3.3 shreds the “independent samples” reading, and until one compares it with the four *million* parameters of the prototype’s LSTM stack (Chapter 3, defect 5). The project’s response is threefold: small encoders (a one-layer GRU with hidden size 32, not an eight-layer LSTM), a mixture architecture whose components are individually simple, and capacity-*matched* baselines (Chapter 16) so that any claimed advantage of the mixture cannot be an artifact of parameter count.

1.3.2 Nothing is stationary

The mapping $x \mapsto y$ is not fixed. It drifts on two timescales. Slowly: signals decay as they become known and traded away — the out-of-sample deterioration of published anomalies is one of the most robust findings in empirical asset pricing (McLean and Pontiff, 2016, document returns falling by roughly a third to a half after publication). And quickly: the *same* characteristic can predict with opposite signs in different market states — momentum’s behavior in calm markets versus its crashes in volatile rebounds being the canonical example, developed in detail in Chapter 2. Slow drift motivates walk-forward evaluation (train strictly on the past, test strictly on the future, refit as time advances — Chapter 15). Fast, recurring, state-dependent change is the thesis itself: it motivates *regime-conditional* models, in which the cross-sectional mapping is allowed to depend on a latent market state. That motivation is the subject of the next chapter, so here we only flag the distinction: walk-forward handles drift you cannot model; regimes are a bet that part of the drift is *recurring structure* you can.

1.3.3 Samples are deeply dependent

The 1.25 million rows above are not 1.25 million observations in any statistically meaningful sense.

Cross-sectionally, names co-move through common factors: on a given date, a large share of every stock’s return is the market and industry component. The effective number of independent observations per date is far smaller than N . A back-of-envelope: if pairwise return correlations average $\bar{\rho}_c$, the variance of the equal-weighted mean of N names is $\frac{\sigma^2}{N}(1 + (N-1)\bar{\rho}_c)$, so the effective sample size is $N_{\text{eff}} = N/(1 + (N-1)\bar{\rho}_c)$; with $\bar{\rho}_c = 0.25$ and $N = 500$, $N_{\text{eff}} \approx 4$ — for estimating the common component. Cross-sectional *ranking* signals fare better (the common component cancels between long and short legs, which is precisely why the task is defined cross-sectionally), but the lesson stands: row counts overstate information content by orders of magnitude.

Temporally, overlapping h -day labels make adjacent dates share outcome information (Example 1.5), and features themselves are slow-moving (today’s momentum is yesterday’s momentum plus a daily increment), so consecutive rows of the design matrix are near-duplicates.

What breaks without this — i.i.d. assumptions applied to panels

Every standard tool that assumes independent samples — cross-validation with random splits, bootstrap standard errors, learning-curve heuristics, early stopping on a random

validation set — silently breaks on panel data. Random splits are the worst offender: they place rows from date t in training and rows from $t \pm 1$ (near-duplicates with overlapping labels) in validation, so validation error estimates in-sample error, model selection overfits, and reported performance can be pure leakage. This is why the evaluation protocol of Part V never once splits randomly: every split in the project is chronological, and every train/test boundary is purged by the label horizon.

1.3.4 Errors are rewarded asymmetrically

The most insidious property is not statistical but sociological, and it deserves a name.

Remark 1.6 (The leakage ratchet). In most software, bugs make results worse, so testing and debugging drive quality up. In backtesting, an entire class of bugs — look-ahead in features, survivorship in the universe, test data touched during tuning, selection among many trials — make results *better*. A researcher who debugs whatever hurts performance and leaves alone whatever helps it is running a ratchet that converges to maximally self-deceiving code, all while every individual action feels like diligence.

The ratchet cannot be defeated by intending to be careful, because each of its clicks is invisible at the moment it happens. It is defeated structurally, by making the honest path the default path: features constructed under a schema in which the target *cannot* occupy a feature column (Chapter 12); leakage stated as executable invariants rather than intentions; evaluation code shared between the model and every baseline so the grading cannot silently differ; every trained configuration logged to an append-only registry so the multiplicity of Section 1.4 cannot be understated; and selection events (“best of N ”) recorded as data. Part V and Part VI are, in essence, a systematic dismantling of Remark 1.6, and it is the single most important idea in this book that is not mathematics.

1.4 What counts as a result

Given all of the above, we can state what the project is actually obliged to produce. Three commitments define “a result.”

1.4.1 Distributional prediction, scored properly

The models in this book output not a point estimate but a conditional distribution $p(y \mid x)$ — for the mixture models of Part II, a K -component Gaussian mixture. Distributions are graded by the negative log-likelihood on held-out data, $\text{NLL} = -\frac{1}{n} \sum_j \log p(y_j \mid x_j)$, which is a *proper scoring rule*: its expectation is uniquely minimized by reporting the true conditional distribution, so a model cannot improve its grade by hedging or exaggerating its uncertainty.

It is worth deriving what NLL means in the simplest case, because the result recurs throughout the book. For a single Gaussian model $p(y \mid x) = \mathcal{N}(y; \mu(x), \sigma^2)$,

$$\text{NLL} = \frac{1}{n} \sum_j \left[\frac{1}{2} \log(2\pi\sigma^2) + \frac{(y_j - \mu(x_j))^2}{2\sigma^2} \right],$$

so at *fixed* σ , minimizing NLL is exactly minimizing mean-squared error — nothing new is being asked of the mean. The new content appears when σ is *learned*: setting $\partial \text{NLL} / \partial \sigma^2 = 0$ gives $\hat{\sigma}^2 = \frac{1}{n} \sum_j (y_j - \mu(x_j))^2$, the mean squared residual, and substituting back,

$$\min_{\sigma} \text{NLL} = \frac{1}{2} \log \hat{\sigma}^2 + \frac{1}{2} \log(2\pi) + \frac{1}{2}.$$

Two consequences. First, NLL comparisons between models are (monotone transformations of) residual-variance comparisons in this simple case, so nothing exotic is smuggled in by switching metrics. Second — and this is the reason the thesis models are distributional at all — once σ may depend on the regime, the model can say “*in this state, everything is noisier*,” and the likelihood rewards it for being *right about its own uncertainty*. Chapter 2 argues that time-varying σ is the single most robust empirical fact about returns; a scoring rule blind to it (like raw MSE) throws away precisely the structure the thesis is about. NLL is also what makes the model comparison fair: the package’s MLP baseline is deliberately a single NEC expert with its own learned σ , trained on the same Gaussian NLL, so “mixture versus single model” compares like with like (Chapter 16).

1.4.2 Economic significance, costed

Statistical significance does not pay transaction costs. A result must also survive translation into the portfolio construction it implies: the quantile long-short of Chapter 16, with turnover measured and a cost per unit of traded notional subtracted. A signal with a fine IC that turns over its entire book daily can be worthless net of costs; the report that omits turnover is not a result but an advertisement. (Per-period information ratios are also never annualized in this project unless the dates are real trading days and the annualization is stated — a small honesty with a long history of being fudged.)

1.4.3 Significance that survives its own search

Finally, the multiplicity commitment. A research project does not train one model; it trains dozens of configurations, seeds, and variants, and reports the ones that look best. Under the null, the maximum of M independent standard normal t -statistics is approximately $\sqrt{2 \ln M}$ — with $M = 20$ trials, expect a best t of about 2.4 *from pure noise*. “We ran many things and this one worked” is therefore not evidence unless the many things are counted and the threshold adjusted. The machinery — trial registries that make the count un-losable, false-discovery-rate control across the claim family, the deflated Sharpe ratio, and pre-registration of the final experiment — is Chapter 17’s subject, and it exists because of this paragraph.

Engineering consequence — the thesis question, made falsifiable

The commitments above compile the vague ambition “build a regime model that predicts returns” into a falsifiable protocol: *does the regime-conditional mixture beat capacity-matched single-model baselines on held-out NLL and pooled rank-IC, under purged walk-forward evaluation, with seeds treated as replicates, the claim family corrected for multiplicity, and costs charged?* Every clause traces to a section of this chapter. The rest of the book is the construction of a system that can answer the question — and that deserves to be believed when it does.

Origins — reading for this chapter

The information coefficient and its aggregation arithmetic are treated in Grinold and Kahn, *Active Portfolio Management* (2000), whose “fundamental law” $IR \approx IC \cdot \sqrt{\text{breadth}}$ is the industry ancestor of Section 1.2.3. The out-of-sample decay of published signals is documented by McLean and Pontiff (2016, *Journal of Finance*). The leakage ratchet and the multiplicity crisis are the animating concerns of López de Prado, *Advances in Financial Machine Learning* (2018), whose purging/embargo constructions reappear in Chapter 15 and whose deflated Sharpe ratio (with Bailey) is derived in Chapter 17. Proper scoring rules go back to Brier (1950) and are surveyed by Gneiting and Raftery (2007).

Code walkthrough — `nec_moe/data.py` --- the panel contract

The chapter’s definitions exist in code as a frozen dataclass; it is worth seeing how directly. A Panel is five aligned tensors — $\mathbf{x}_{\text{seq}}(n, T_w, d_{\text{seq}})$, $\mathbf{x}_{\text{snap}}(n, d_{\text{snap}})$, $\mathbf{y}(n,)$, $\text{date}(n,)$, $\text{entity}(n,)$ — one row per (name, date) pair:

```
@dataclass(frozen=True)
class Panel:
    x_seq: Tensor      # (n, T_w, d_seq) trailing windows
    x_snap: Tensor     # (n, d_snap) point-in-time snapshots
    y: Tensor          # (n,) forward return -- the ONLY
                    # forward-looking column
    date: Tensor       # (n,) int64 date index
    entity: Tensor     # (n,) int64 name index
```

Three details echo the chapter. The target is a separate tensor — there is no “reserved column” inside a feature matrix for it to hide in (Definition 1.2; this kills defect 7 of Chapter 3 structurally). All splitting utilities are date-based (`subset_dates`, `split_by_date`) — the API simply does not offer a random row split, making the honest path the only paved one (Remark 1.6). And date/entity ride along every batch, because per-date grading (Definition 1.3) and turnover accounting need them at scoring time.

Exercises

Exercise 1.1. (Null IC, finite-sample.) Complete the proof of Proposition 1.4 by verifying the two permutation moments used: for a uniformly random permutation Π and a centered, normalized vector b , show $\mathbb{E}[b_{\Pi(i)}^2] = 1/N$ and $\mathbb{E}[b_{\Pi(i)}b_{\Pi(i')}] = -1/(N(N-1))$ for $i \neq i'$. Then show that as $N \rightarrow \infty$, $\sqrt{N-1}IC_t$ is asymptotically standard normal. (Hint: Hoeffding’s combinatorial central limit theorem, or argue via the projection onto simple linear rank statistics.)

Exercise 1.2. (From IC to portfolio spread.) Suppose scores and outcomes are jointly Gaussian across a date’s cross-section with correlation ρ , outcomes with standard deviation σ_y . Show the expected top-minus-bottom quintile spread is $\mathbb{E}[L - S] = \rho \sigma_y (\mathbb{E}[Z \mid Z > z_{0.8}] - \mathbb{E}[Z \mid Z < z_{0.2}]) \approx 2.8 \rho \sigma_y$, where Z is standard normal and z_q its quantiles. (Truncated-normal means: $\mathbb{E}[Z \mid Z > c] = \phi(c)/(1 - \Phi(c))$.) With $\rho = 0.03$ and daily cross-sectional vol $\sigma_y = 2\%$, what is the expected daily spread in basis points, before costs?

Exercise 1.3. (*Hands-on: the ratchet in miniature.*) Using numpy, simulate $M = 20$ zero-skill strategies, each scored on 250 dates with $N = 100$ names. Report the best strategy's $\overline{\text{IC}}$, its naïve t -statistic, and how often (over 1,000 repetitions) the best t exceeds 2. Compare the frequency with the $\sqrt{2 \ln M}$ heuristic of Section 1.4.

Exercise 1.4. (*Experimental arithmetic.*) You expect $\overline{\text{IC}} = 0.02$ and $\sigma_{\text{IC}} = 0.12$. (a) How many test dates do you need for $t \geq 3$? (b) The label horizon is $h = 5$ days; assuming the overlap makes the IC series behave like an $\text{MA}(h-1)$ process with first-order autocorrelation $\approx (h-1)/h$ decaying linearly to zero at lag h , derive the variance inflation factor for $\overline{\text{IC}}$ and the corrected date count. (c) The package's default evaluation reserves `n_folds` $\times$ `test_dates_per_fold` dates; what settings meet your corrected count on a ten-year daily panel?

Exercise 1.5. (*NLL and MSE.*) (a) Verify the profiled-NLL identity $\min_{\sigma} \text{NLL} = \frac{1}{2} \log \hat{\sigma}^2 + \text{const}$ of Section 1.4. (b) Two models have held-out NLLs 1.372 and 1.351 (natural logs). By what percentage do their implied residual variances differ? (c) Explain why NLL can prefer model A over model B even when B has lower MSE — construct a two-point example with heteroskedastic data.

Exercise 1.6. (*Hands-on: probing the contract.*) In the repository, construct a synthetic panel via `SyntheticRegimePanel` (see `nec_baseline/README.md`), and (a) verify from tensor shapes that no feature column equals the target; (b) use `split_by_date` and confirm no training date exceeds any test date; (c) try to build a random row split with the public `Panel` API and report what stops you. Read `tests/test_no_lookahead` (Stage-B tests) and summarize, in one paragraph, the exact invariant it pins.

Chapter 2

The Case for Regimes

Chapter 1 established that the cross-sectional mapping drifts. This chapter argues that part of the drift is not formless: markets move between persistent, recurring states, and the statistical footprint of those states is visible in the most robust stylized facts about returns. We derive that footprint (fat tails from mixing, volatility clustering from persistence), meet the classical model built on it (Hamilton’s Markov switching), and then take the step that defines the thesis: letting the cross-sectional prediction function itself depend on the regime. The chapter ends with the precise model class the rest of the book constructs, trains, and evaluates.

2.1 Three stylized facts

Daily equity returns, at the index or single-name level, exhibit three regularities so stable across markets and decades that any credible model must account for them:

- (F1) *Fat tails.* Return distributions have far more mass in their tails than a Gaussian of the same variance. Sample excess kurtosis of daily index returns is typically in the range 5–30 (a Gaussian has 0); multi-sigma days occur orders of magnitude more often than normality predicts.
- (F2) *Volatility clustering.* Large-magnitude days follow large-magnitude days: the autocorrelation of $|r_t|$ and r_t^2 is positive and decays slowly over weeks to months, even while the autocorrelation of r_t itself is negligible. Mandelbrot’s 1963 formulation stands: “large changes tend to be followed by large changes, of either sign, and small changes by small changes.”
- (F3) *State-dependent relationships.* Correlations between assets rise sharply in stressed markets, and — most important for this thesis — the *predictive content of cross-sectional characteristics changes with the market state*. Section 2.3 develops the canonical example.

The regime hypothesis is a single explanation for all three: returns are conditionally simple (say, Gaussian) given a latent market state S_t , the state modulates the parameters (above all the volatility), and the state is *persistent*. The next section shows this is not hand-waving — the first two facts follow *mathematically* from exactly that structure.

2.2 Mixtures generate the stylized facts

2.2.1 Fat tails from mixing

Consider the simplest regime model: each day belongs to state $k \in \{1, \dots, K\}$ with probability p_k , and conditional on the state the return is centered Gaussian with state-specific volatility, $r \mid S = k \sim \mathcal{N}(0, \sigma_k^2)$. The unconditional law is the *scale mixture* $r \sim \sum_k p_k \mathcal{N}(0, \sigma_k^2)$.

Proposition 2.1 (Scale mixtures are leptokurtic). *Let r follow the scale mixture above and write $m_2 = \sum_k p_k \sigma_k^2$ for its variance. Then its kurtosis is*

$$\kappa = \frac{\mathbb{E}[r^4]}{\mathbb{E}[r^2]^2} = 3 \frac{\sum_k p_k \sigma_k^4}{(\sum_k p_k \sigma_k^2)^2} \geq 3,$$

with equality if and only if all the σ_k with $p_k > 0$ are equal.

Proof. Condition on the state and use Gaussian moments ($\mathbb{E}[r^2 \mid S=k] = \sigma_k^2$, $\mathbb{E}[r^4 \mid S=k] = 3\sigma_k^4$):

$$\mathbb{E}[r^2] = \sum_k p_k \sigma_k^2, \quad \mathbb{E}[r^4] = 3 \sum_k p_k \sigma_k^4.$$

Let V be the random variable taking value σ_k^2 with probability p_k (the “randomized variance”). Then $\kappa = 3 \mathbb{E}[V^2]/(\mathbb{E}[V])^2$, and by Jensen’s inequality (or Cauchy–Schwarz) $\mathbb{E}[V^2] \geq (\mathbb{E}[V])^2$, with equality iff V is almost surely constant. $\square$

The inequality carries a reading worth memorizing: *excess kurtosis is the variance of the variance*. Precisely, $\kappa - 3 = 3 \text{Var}(V)/(\mathbb{E}V)^2$ — unconditional fat tails are what time-varying (here: state-varying) volatility looks like after the state is integrated out. A two-state example calibrated to the package’s synthetic defaults, $\sigma = (0.5, 2.5)$ with equal weights: $\mathbb{E}V = 3.25$, $\mathbb{E}V^2 = 19.56$, so $\kappa = 3 \times 19.56/3.25^2 \approx 5.6$ — nearly double the Gaussian’s 3, from conditional normality alone. And the *proportions* matter as much as the levels: the same $5\times$ volatility ratio concentrated in a *rare* turbulent state (weight $\approx 4\%$) drives κ above 20, squarely into the empirical range — Exercise 2.1 derives the optimal rarity. Fat tails are the signature of occasional, distinct turbulence, not of two similar-sized populations. Figure 2.1 (bottom left) shows the effect on a log-density scale: the mixture’s tails sit far above the moment-matched Gaussian’s.

The moral for modeling: one does not need heavy-tailed *conditional* distributions to reproduce heavy-tailed data. A mixture of Gaussians with state-dependent σ_k — exactly the emission structure of the thesis model — generates realistic tails while keeping every component analytically tractable. (This is also the honest reading of “Gaussian assumption” criticisms: the model class is only conditionally Gaussian; its unconditional predictions are not.)

2.2.2 Clustering requires persistence

Fact (F1) needed only a *mixture*. Fact (F2) — clustering — needs more: it is a statement about *dependence over time*, and an i.i.d. state sequence cannot produce it. If S_t is drawn independently each day, then r_t^2 and r_{t+s}^2 are independent for $s \geq 1$, so $\text{Corr}(r_t^2, r_{t+s}^2) = 0$: an i.i.d. mixture has fat tails but no memory. Clustering is evidence that the state *persists*.

The minimal persistent-state model is a Markov chain. Let S_t be a two-state chain with stay probabilities $a = \mathbb{P}(S_{t+1}=1 \mid S_t=1)$ and $b = \mathbb{P}(S_{t+1}=2 \mid S_t=2)$, and $r_t = \sigma_{S_t} \varepsilon_t$ with ε_t i.i.d. standard normal.

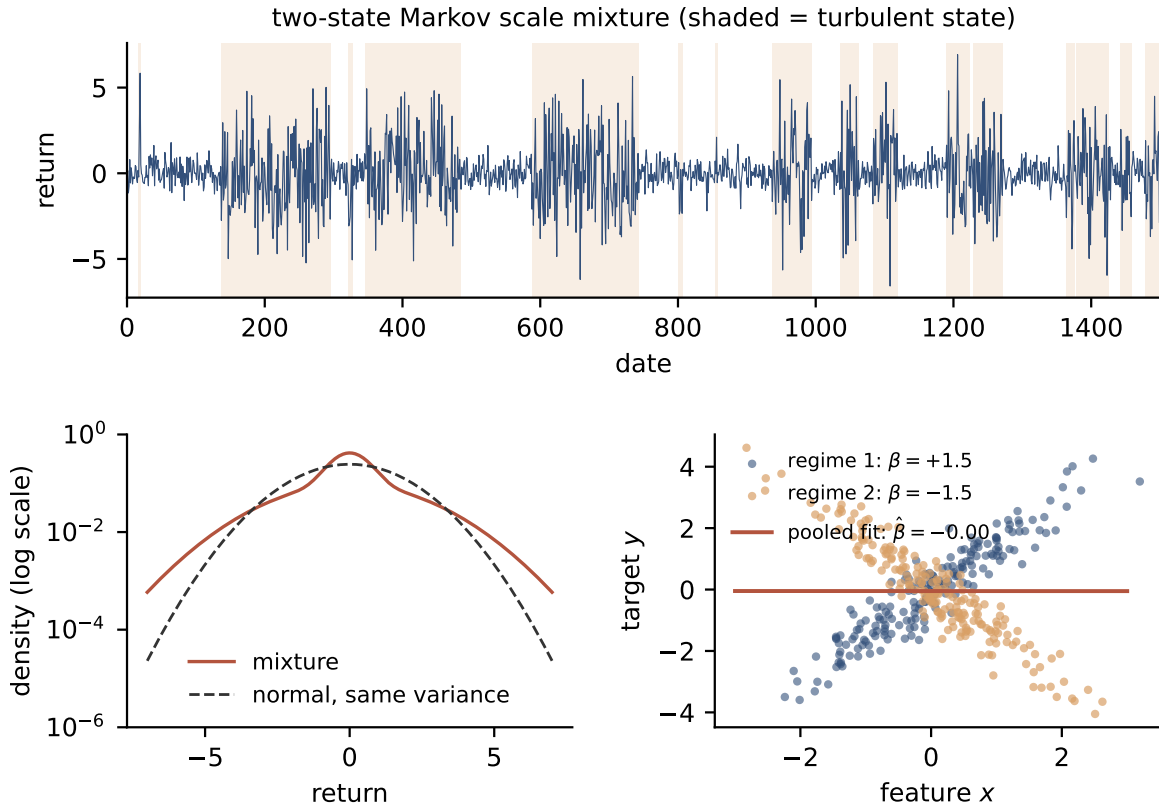


Figure 2.1: Regimes generate the stylized facts. Top: a return series from a two-state Markov scale mixture ($\sigma = (0.6, 2.2)$, stay probability 0.98); turbulent-state periods shaded. Volatility clustering is visible to the naked eye — and provably absent if the same states are shuffled i.i.d. Bottom left: the unconditional density against a Gaussian of identical variance (log scale) — fat tails from Proposition 2.1. Bottom right: the sign-flip construction of Section 2.3.2 — two regimes with slopes $\pm\beta$; the pooled fit (red) estimates $\hat{\beta} \approx 0$ and is structurally blind to a relationship that is perfectly strong within each regime.

Proposition 2.2 (Squared-return autocorrelation). *Under the two-state Markov scale mixture, for lags $s \geq 1$,*

$$\text{Corr}(r_t^2, r_{t+s}^2) = c \cdot \lambda^s, \quad \lambda = a + b - 1,$$

where $c \in (0, 1)$ is a constant depending on $(a, b, \sigma_1, \sigma_2)$ but not on s . The chain's second eigenvalue λ — its persistence — is the decay rate of volatility clustering.

Proof. Write $v_t = \sigma_{S_t}^2$, so $r_t^2 = v_t \varepsilon_t^2$ with ε_t^2 i.i.d., mean 1, independent of the state path. Then for $s \geq 1$, $\text{Cov}(r_t^2, r_{t+s}^2) = \text{Cov}(v_t, v_{t+s})$ (the independent ε^2 factors contribute $\mathbb{E}[\varepsilon^2]^2 = 1$ across distinct times), while $\text{Var}(r_t^2) = \mathbb{E}[v_t^2] \mathbb{E}[\varepsilon^4] - \mathbb{E}[v_t]^2 = 3\mathbb{E}[v_t^2] - \mathbb{E}[v_t]^2 > \text{Var}(v_t)$; hence the correlation of r^2 is a fixed positive multiple $c < 1$ of the correlation of v . It remains to compute $\text{Corr}(v_t, v_{t+s})$. The transition matrix $A = \begin{pmatrix} a & 1-a \\ 1-b & b \end{pmatrix}$ has eigenvalues 1 and $\lambda = a + b - 1$; its stationary distribution is $\bar{\pi} = (\frac{1-b}{2-a-b}, \frac{1-a}{2-a-b})$. For a two-state chain, any function $f(S_t)$ — here v_t , which takes two values — has autocorrelation exactly λ^s : decompose f into its stationary mean plus a component along the second left-eigenvector; the s -step transition operator scales that

component by λ^s , and correlations are invariant to the affine part. Therefore $\text{Corr}(v_t, v_{t+s}) = \lambda^s$ and $\text{Corr}(r_t^2, r_{t+s}^2) = c \lambda^s$. $\square$

With stay probabilities $a = b = 0.98$ (mean regime duration $1/(1 - a) = 50$ days), $\lambda = 0.96$: squared-return autocorrelations decay by only $\sim 4\%$ per day and remain visible for months — matching (F2). The top panel of Figure 2.1 is this process, and the clustering is unmistakable.

Two lessons transfer directly to the thesis model. First, *persistence is where the memory lives*: the conditional distribution of r_t given the state needs no dependence structure at all; the Markov chain carries all of it. This division of labor — simple emissions, dynamic state — is exactly the architecture of Part II’s models. Second, mean regime duration $1/(1 - a)$ gives persistence an interpretable scale, which will return in Part III as a *persistence-biased initialization*: the learned transition matrix is initialized sticky (a near 0.9–0.98), encoding the prior belief that regimes last weeks, not hours.

Origins — Hamilton 1989 and what came before

Mixture-of-normals descriptions of speculative prices go back to Mandelbrot (1963) and Fama (1965) on fat tails, and to Clark (1973), whose subordinated process — returns as Gaussians with a randomized variance — is Proposition 2.1 in continuous form. Engle’s ARCH (1982) and Bollerslev’s GARCH (1986) capture clustering with an observable, recursively defined variance. The regime-switching alternative is due to James D. Hamilton, “A New Approach to the Economic Analysis of Nonstationary Time Series and the Business Cycle” (*Econometrica*, 1989): an autoregression whose parameters switch according to an unobserved two-state Markov chain, estimated on U.S. GNP growth, with the states identified ex post as expansions and recessions — NBER-dated recessions were recovered by the filter without ever being shown to it. Hamilton’s technical contribution is the estimation machinery: because the state is latent, the likelihood must integrate over all state paths, which his *forward filter* accomplishes recursively in $O(TK^2)$. That filter — and the distinction between filtering ($\mathbb{P}(S_t \mid \text{data up to } t)$), which is causal, and smoothing ($\mathbb{P}(S_t \mid \text{all data})$), which is not — is derived in full in Chapter 7 and sits verbatim inside the thesis model’s HMM gate. The classical Hamilton model itself survives in the package as `ExpertConfig(kind="classical") × PriorConfig(kind="hmm")`: constant per-regime Gaussians under a learned Markov prior, used as a parameter-recovery test bed and an honest classical baseline.

2.3 From marginal regimes to conditional regimes

Everything so far concerned the *marginal* distribution of returns — enough for risk modeling, but not yet a prediction model. The thesis takes one further step, and it is the step that turns regime modeling into cross-sectional alpha research:

*Let the map from characteristics to expected returns
depend on the regime.*

2.3.1 The economics: momentum as the canonical example

Fact (F3) is best seen through the best-documented cross-sectional signal: momentum (buy the past year's winners, short the losers; Jegadeesh and Titman, 1993). Its average profitability is among the most replicated findings in finance — and so are its *crashes*. Daniel and Moskowitz (“Momentum crashes,” *JFE*, 2016) document that momentum's worst episodes are concentrated in a specific, identifiable state: panic markets — high volatility, following a market decline — especially at the turn into a sharp rebound. In 1932 and 2009, the losers-rebound effect was so violent that the strategy lost most of a century's compounding in weeks. In the language of this book: the coefficient linking the momentum characteristic to expected return is positive in calm states and *negative* in high-volatility rebound states. Similar state-dependence is documented for reversal strength, size, quality, and the low-vol anomaly; regime-dependent factor performance is the working assumption of practitioner asset allocation.

For prediction, this is qualitatively different from (F1)–(F2). A signal whose *sign flips* with the state is invisible to any model that assumes one global mapping — not weakened, *invisible*, as the next subsection makes exact.

2.3.2 The statistics: what pooling destroys

Idealize the situation. Two regimes, equally likely; within regime k the cross-section obeys a linear model with regime-dependent slope:

$$y_i = \beta_S x_i + \varepsilon_i, \quad \beta_1 = +\beta, \quad \beta_2 = -\beta, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_\varepsilon^2),$$

with x_i standardized and independent of the regime. What does the best *global* (regime-blind) model do?

Proposition 2.3 (Pooling annihilates a sign-flipping signal). *Under the model above, the population regression of y on x , pooled over regimes, has slope*

$$\beta_{\text{pool}} = \frac{\text{Cov}(y, x)}{\text{Var}(x)} = \mathbb{E}[\beta_S] = \frac{1}{2}\beta + \frac{1}{2}(-\beta) = 0,$$

and the pooled model's cross-sectional R^2 — hence its IC — is zero. An oracle that knows S attains $R^2 = \beta^2/(\beta^2 + \sigma_\varepsilon^2)$ in every regime. Moreover, no measurable function $f(x)$ of the characteristic alone can do better than the pooled slope: $\mathbb{E}[y | x] = \mathbb{E}[\beta_S] x = 0$ for all x .

Proof. $\text{Cov}(y, x) = \mathbb{E}[\beta_S x^2] + \mathbb{E}[\varepsilon x] = \mathbb{E}[\beta_S] \mathbb{E}[x^2]$ by independence of S and x and exogeneity of ε ; the last claim is the tower property: $\mathbb{E}[y | x] = \mathbb{E}[\mathbb{E}[y | x, S] | x] = x \mathbb{E}[\beta_S] = 0$. $\square$

The final clause is the sharp one: the failure is not linearity. *Any* model — a gradient-boosted forest, a deep network — that consumes only the characteristic has zero predictive content in this world, because the conditional mean given the characteristic alone is identically zero. The information is real (the oracle's R^2 can be near 1) but it lives entirely in the *interaction* between the characteristic and the state. Figure 2.1 (bottom right) draws the picture: two clean regressions of slope ± 1.5 ; one flat, useless pooled line.

This construction is deliberately extreme — real regime-dependence is a matter of degree, attenuation rather than annihilation (if regime probabilities are $p \neq \frac{1}{2}$ or slopes merely shrink, the pooled slope is $\mathbb{E}[\beta_S] \neq 0$ and pooling loses part of the signal). But its extremity is precisely

what makes it *useful*: it is the ideal **planted truth** for testing a regime model. In a world generated by Proposition 2.3, a working regime-conditional model must achieve high IC while every regime-blind baseline must, by mathematics rather than by luck, achieve none. The package’s synthetic panel generator implements exactly this world, and the claim “the mixture beats the pooled baseline *here*” is enforced as a permanent regression test — both directions of it (the baselines must also *win* on a no-regime world, proving they are not straw men). Chapter 14 builds the instrument; the walk-through below shows its heart.

What breaks without this — if regimes are real and the model is global

Proposition 2.3 is the precise content of the claim “non-stationarity is not just drift” from Section 1.3. Where relationships flip with the state, a global model does not degrade gracefully — it averages the states and reports noise. Worse, it does so *silently*: the fit succeeds, the loss decreases, the diagnostics look ordinary; only the out-of-sample IC hovers at zero. If a rich global model is given extra capacity it will typically *overfit within-sample noise* before it discovers a state interaction it has no features to express. The failure mode is invisible without a regime-aware alternative to compare against — one of the quieter reasons to build the mixture even if one ultimately bets against it.

2.3.3 Why not just add state features?

Proposition 2.3 shows the model must see *some* state information. The obvious cheap fix: append a state proxy (VIX, trailing market volatility) as a feature to a global model, and let it learn the interaction, e.g. $\hat{y} = g(x, v)$. With enough capacity, g can represent $\beta(v)x$; a universal approximator handles it in principle. Why, then, an explicit mixture? Four reasons, in increasing order of importance to the thesis:

1. *Statistical structure.* The mixture encodes the hypothesis as *a small number of discrete states, each with a full, simple cross-sectional model, plus persistence in the state process*. A generic interaction model must discover discreteness, persistence, and the within-state models from scratch, spending capacity (hence variance — Section 1.3) on what the mixture gets as inductive bias. In a domain where variance is the binding constraint, inductive bias is not a convenience; it is the budget.
2. *Distributional honesty.* The mixture is a model of $p(y \mid x)$ with state-dependent σ_k — it natively expresses “high-vol state $\Rightarrow$ wider predictive distribution,” which is (F1)–(F2) speaking, and it is graded on it via NLL (Section 1.4). A point-prediction interaction model expresses none of this.
3. *The state is latent.* VIX is a proxy, not the state. A mixture with a learned gate can *infer* the state from inputs (and, in the HMM variant, from its own recent posterior — carrying the persistence of Section 2.2.2 into inference). Hamilton’s central lesson applies: treat the state as unobserved and integrate over it, rather than hard-coding a proxy.
4. *Falsifiability.* This is the decisive one for a thesis. In the mixture, the state is an explicit object: one can ask “do the learned regimes correspond to anything real?” and answer by correlating the gate’s inferred posterior with *held-out* state proxies it never saw in training (the alignment analysis of Chapter 18’s practicum cousin, Stage C). In a monolithic

$g(x, v)$, the regime hypothesis is smeared across weights, and no such question can be asked. Crucially, this requires the gate to be *unsupervised* — the moment one trains the gate to predict VIX, “the model discovered volatility regimes” becomes circular. That is Decision B of Part III, made here, in the theory, before any code.

The honest counterpoint is also worth recording: the mixture buys these properties at the price of a harder optimization landscape (multimodality, symmetric local optima — Chapter 8), an identifiability problem (labels can permute — Chapters 7–8), and more machinery to get wrong. The prototype of Chapter 3 is what “more machinery to get wrong” looks like in practice. The thesis is precisely the wager that the structure is worth its optimization cost — and Part V exists so that the wager is settled by evidence rather than by enthusiasm.

2.4 The model class

We can now write down the object this book builds. Everything later — the loss (Chapter 5), the routing variants (Chapter 6), the HMM gate (Chapter 7), the architecture (Part III), the evaluation (Part V) — is this definition, taken seriously.

Definition 2.4 (Regime-conditional mixture of experts). For a name–date pair with features $x = (x^{\text{seq}}, x^{\text{snap}})$, the model is the conditional mixture

$$p(y | x) = \sum_{k=1}^K \pi_k \mathcal{N}(y; \mu_k(x), \sigma_k^2),$$

with three learned ingredients:

- **Experts** $\mu_k(\cdot)$: K regime-specialized prediction functions (small MLPs on the snapshot features; or constants, recovering the classical Hamilton emission), each with its own volatility σ_k ;
- **A gate**, producing the mixture weights from the encoded sequence state $h = \text{enc}(x^{\text{seq}})$;
- **A prior mechanism** $\pi = \text{prior}(z, \cdot)$ turning the gate’s logits $z(h)$ into weights — *pluggable*: a memoryless softmax $\pi_k \propto e^{z_k}$ (and its uniform / hard / top- k / annealed variants) for thesis Variation 2, or a causal Markov filter $\pi_{t,k} \propto \sum_j A_{jk} r_{t-1,j}$ carrying yesterday’s posterior through a learned transition matrix for Variation 3.

Training maximizes the likelihood of held-out-able data — equivalently, minimizes $\text{NLL} = -\frac{1}{n} \sum \log p(y | x)$ — with the latent state never observed and the gate never supervised.

Each clause of Definition 2.4 answers to a section of these first two chapters. Conditional Gaussians with per-regime σ_k : fat tails and honest uncertainty (Proposition 2.1, §1.4). Regime-indexed $\mu_k(x)$: the sign-flip argument (Proposition 2.3). A latent, unsupervised state inferred by a gate: §2.3.3. A pluggable prior with a Markov option: persistence (Proposition 2.2) and Hamilton’s filter. Small experts on snapshot features, a small encoder for the state: the variance budget of §1.3. Nothing in the model is unmotivated; that is what a design derived from theory looks like, and Part III will hold the implementation to it.

Code walkthrough — nec_moe/data.py --- the planted world

The synthetic generator is Proposition 2.3 plus Proposition 2.2 in about a page of code. Its spec exposes exactly the theory's knobs:

```
@dataclass(frozen=True)
class SyntheticSpec:
    n_regimes: int = 2
    regime_process: str = "iid"      # "iid" | "markov"
    transition_stay: float = 0.95    # Markov persistence (Prop 2.2)
    vol_levels: tuple = (0.5, 2.5)   # sigma_k (Prop 2.1)
    beta_scale: float = 1.0          # +/- beta sign flip (Prop 2.3)
    noise_std: float = 0.5           # sigma_eps
    seed: int = 0
```

Per date, one regime S_t is drawn (i.i.d. or sticky-Markov); every name's target is $y_{i,t} = \beta_{S_t}^\top x_{i,t}^{\text{snap}} + \sigma_{S_t} \varepsilon_{i,t}$ with $\beta_2 = -\beta_1$ — the sign flip. The sequence features carry a volatility footprint of the regime, so the state is *inferable but not given*. Because the generator also returns the true S_t , tests can score regime recovery directly. The two process modes matter later: "iid" suffices for the memoryless models of Variation 2, while "markov" plants the persistence that the HMM gate of Variation 3 must (and, in the tests, provably does) exploit.

Exercises

Exercise 2.1. (*Kurtosis, quantitatively.*) (a) Show $\kappa - 3 = 3 \text{Var}(V)/(\mathbb{E}V)^2$ where V is the randomized variance of Proposition 2.1. (b) For the two-state case with weights $(p, 1-p)$ and volatilities (σ_1, σ_2) , find the weight p maximizing κ at fixed $\sigma_2/\sigma_1 = 5$, and comment: why do *rare* volatile regimes generate the fattest tails? (c) Real daily index returns show $\kappa \approx 10$ –30, the equal-weight two-state model of this chapter gives $\kappa \approx 5.6$, and your answer to (b) exceeds 20 with the same $5\times$ volatility ratio. What do these three numbers together suggest about how much of empirical kurtosis regime switching can explain, and about what the regime *proportions* must look like for it to do so?

Exercise 2.2. (*Persistence and memory.*) (a) Complete the proof of Proposition 2.2 by computing the constant c explicitly for $a = b$: show $c = \frac{\text{Var}(v)}{3\mathbb{E}[v^2] - (\mathbb{E}v)^2}$ with $v \in \{\sigma_1^2, \sigma_2^2\}$ equally likely, and evaluate c and the lag-10 autocorrelation for $\sigma = (0.6, 2.2)$, $a = 0.98$. (b) Show that mean regime duration is geometric with mean $1/(1-a)$. (c) An econometrician fits a GARCH(1,1) to data generated by this model. Which GARCH parameter will absorb λ ? (No proof — one paragraph of reasoning.)

Exercise 2.3. (*Pooling, beyond the symmetric case.*) Generalize Proposition 2.3: regimes with probabilities $(p, 1-p)$ and slopes (β_1, β_2) . (a) Find β_{pool} and the pooled R^2 as functions of $(p, \beta_1, \beta_2, \sigma_\varepsilon)$. (b) Show the pooled model's *residual variance* exceeds the oracle's by $\text{Var}(\beta_S) \mathbb{E}[x^2]$ — regime dispersion in slopes reappears as irreducible-looking noise. (c) Connect (b) to Proposition 2.1: in what precise sense is “variance of the slope” the cross-sectional analogue of “variance of the variance”?

Exercise 2.4. (*When is a mixture visibly a mixture?*) For the equal-weight, equal-variance two-component mixture $\frac{1}{2}\mathcal{N}(\mu_1, \sigma^2) + \frac{1}{2}\mathcal{N}(\mu_2, \sigma^2)$, show the density is bimodal if and only if $|\mu_1 - \mu_2| > 2\sigma$. (Reduce to $f(x) \propto \cosh(\delta x)e^{-x^2/2}$ form and study its critical points.) Conclude: regime structure can be strong yet invisible in the marginal density — which is why Chapter 14's planted-truth tests grade models on *recovery of the latent state*, never on eyeballing histograms.

Exercise 2.5. (*Hands-on: verify the theory against the generator.*) Using `SyntheticRegimePanel` with the defaults of the code walkthrough: (a) simulate 50,000 dates of the "markov" process (`transition_stay=0.98`) and verify Proposition 2.1's kurtosis and Proposition 2.2's geometric ACF decay numerically (log-ACF should be linear in lag with slope $\log \lambda$); (b) regress y on the snapshot features pooled versus within-regime (the generator exposes true regimes) and reproduce Proposition 2.3's zero-versus-oracle contrast; (c) switch to `regime_process="iid"` and confirm the ACF flattens while the kurtosis survives — the cleanest possible demonstration that tails come from *mixing* but memory comes from *persistence*.

Chapter 3

Autopsy of a Prototype

The project did not begin with a blank page. It began with a working — running, loss-decreasing, plausible-looking — prototype that was wrong in eight distinct ways, none of which produced an error message. This chapter dissects it. The point is not self-flagellation: the prototype’s architectural idea was sound, and the corrected system keeps it. The point is that each defect is a specimen of an entire class of failure, and the corrected system is best understood as eight class-level defenses. Every later chapter of this book repairs something that dies on this table.

3.1 The specimen

The prototype (preserved untouched in the repository at `OP_model/nec/nec_hybrid_explained.py`) implements a *normal-versus-extreme conditional* model — NEC — in three pieces:

- C_L: a stacked LSTM sequence encoder (defaults: hidden size 256, *eight* layers) reading the feature window;
- C_F: a classifier head on one selected time step of the LSTM output — Linear → BatchNorm1d → activation → Linear → `nn.Softmax` — emitting probabilities over *three* market classes;
- two NEC_MLP regressors, “extreme” and “normal,” each reading a raw slice of the input window.

The forward pass routes hard:

```
prob = self.classifier(h)          # (B, 3) softmax output
e_out = self.extreme(x[:, -1, :-1]) # slice: last step, drop last col
n_out = self.normal(x[:, -1, :-1])
pred = torch.where(torch.argmax(prob, dim=1) == 1, e_out, n_out)
```

Figure 3.1 draws the whole machine with the defects marked where they live.

Before the knife comes out, record what is *right*. The prototype’s thesis — encode the recent sequence into a state, classify the market regime from that state, and route to a regime-specialized regressor — is exactly Definition 2.4. The corrected system keeps this skeleton wholesale: encoder → gate → experts. Diagnosing which part of a broken system is sound is as much a skill as finding the breaks; the ledger’s verdict on the architecture was *keep*, and the whole project is downstream of that judgment call being correct.

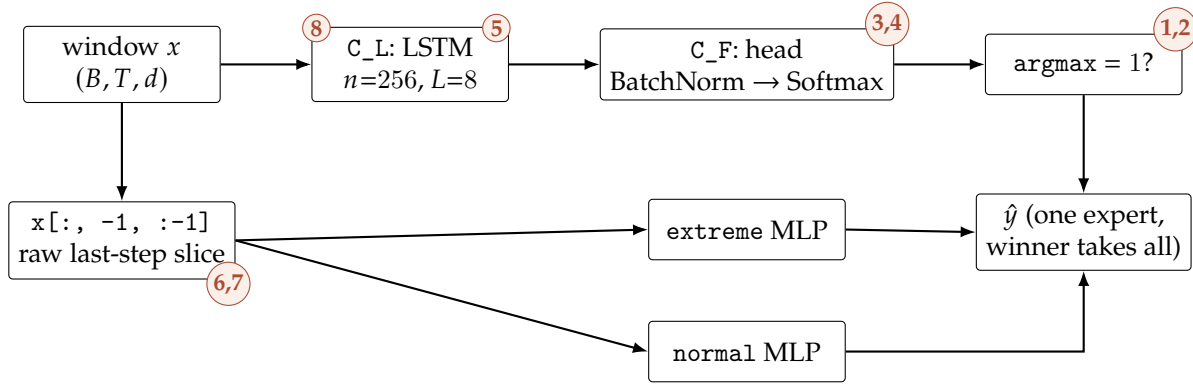


Figure 3.1: The prototype, with defects in place. Numbers key to the ledger of Section 3.2. Note the two load-bearing absurdities visible even at block-diagram level: the gradient cannot cross the `argmax` (1), so the entire left half of the diagram — the four-million-parameter encoder and its head — receives no training signal from the prediction task; and the experts read a raw, undocumented slice of the input (6, 7), so the encoder’s summary influences *nothing* except the untrainable routing decision.

3.2 The ledger

Eight defects, in logical rather than severity order. For each: what the code does, why it is wrong (with the mathematics where mathematics is the right scalpel), and the repair — with a pointer to the chapter that builds it properly.

Defect 1 — `argmax` routing: the gate cannot learn

The routing decision `torch.where(argmax(prob) == 1, e_out, n_out)` selects one expert per sample. As a function of the classifier’s parameters θ , the selection $1\{\arg \max_k \text{prob}_k(\theta) = 1\}$ is piecewise constant: small perturbations of θ leave the `argmax` unchanged almost everywhere, so

$$\frac{\partial \hat{y}}{\partial \theta} = 0 \quad \text{almost everywhere.}$$

Autograd dutifully computes this zero. Whatever loss is attached to $\hat{y}$, *no gradient ever reaches the classifier* through the routing path — the gate’s weights receive exactly zero training signal from the prediction task. The four-million-parameter encoder feeding it is trained, if at all, only by whatever auxiliary classification loss the gate carries, and the routing quality can never improve from prediction feedback. Nothing crashes; the loss on the experts still decreases; the model “trains.”

The repair (Chapter 5) is to stop making a decision and start mixing: weight each expert’s *density* by a probability and maximize the mixture likelihood. The gradient of that objective with respect to the gate logits will turn out to be the beautiful $\pi - r$ — prior minus posterior — restoring a rich training signal to precisely the module that had none. The lesson generalizes far beyond this file:

*hard decisions inside a differentiable model are gradient graves;
make the decision soft, or move it outside the training path.*

And the story has a coda that will recur in Chapter 6: the “obvious” soft fix of renormalizing the top-1 weight is *equally* gradient-dead — a constant 1 is as flat as an indicator — a fact the corrected package enforces with a validation error and proves with a test.

Defect 2 — three classes collapsed to one bit

The classifier emits three market classes; the routing consumes only 1{class = 1}. Classes 0 and 2 — presumably “calm” and some third state — are silently merged, the distinction the classifier spends capacity learning is discarded, and the number of regimes is hard-wired into an equality test buried in the forward pass. K is not a parameter; it is folklore.

The repair is structural, not cosmetic: *all* K components enter the likelihood with their own weights and densities (Chapter 5), K lives in one configuration field, and no class index has hardcoded meaning anywhere — which later forces an honest confrontation with the fact that mixture components are exchangeable (the label-permutation problem, Chapters 7–8), a problem the prototype never met only because it had welded the labels to the furniture.

Defect 3 — `nn.Softmax` inside the model, feeding a second loss

The head ends in `nn.Softmax`, so downstream code sees probabilities, and any classification loss must be computed as $\log(\text{softmax}(z))$ in two separate steps. Two distinct failures hide here. *Numerical*: $\log(\text{softmax}(z))$ computed naïvely overflows for logits as small as $z \approx 90$ (since $e^{90} \approx 10^{39}$ exceeds float32’s range) and catastrophically cancels for confident predictions, where the winning probability rounds to 1.0 and its log to exactly 0; the fused `log_softmax` identity $\log \pi_k = z_k - \text{LSE}(z)$, with the max-subtracted log-sum-exp, is exact and stable at any scale — Chapter 11 makes log-domain computation a system-wide law. *Statistical*: the gate was to be trained by a *separate* classification objective against externally supplied regime labels, while the experts trained on regression — two losses, two sources of truth, and a gate whose notion of “regime” is whatever the labeler said rather than whatever best explains the returns. The corrected model has *one* scalar objective, the mixture NLL, in which routing and prediction are the same optimization (Chapter 5) and the gate is unsupervised by design (Decision B, Chapters 2 and 9).

Defect 4 — `BatchNorm1d` in the head: samples that whisper to each other

`BatchNorm` normalizes each activation by statistics of the *current batch*. Consequences, in ascending severity for this domain: (i) train/eval asymmetry — at eval time the layer switches to running statistics, so the very function being computed differs between training and deployment; (ii) batch-composition sensitivity — a sample’s output depends on which other samples share its batch; (iii) *leakage* — in a panel, a batch can mix dates, and normalizing by batch statistics lets information flow between dates within the batch: sample A at date t is transformed using moments computed partly from samples at $t' > t$. That is look-ahead, laundered through a normalization layer, invisible in any code review that does not know to look for it. And by Remark 1.6, if it helps the backtest, nothing downstream will ever flag it.

The repair is the drop-in `LayerNorm` — identical normalization *within* each sample across features, no cross-sample statistics, no train/eval asymmetry, no leakage channel (Chapter 11). The general lesson: in time-ordered data, *any* operation that couples samples — batch normalization, batch-level feature scaling, cross-sectional ranks computed over a batch rather than

a date — is a leakage suspect until proven innocent, and the proof must be structural (the operation cannot see the future) rather than empirical (“the backtest looks fine”).

Defect 5 — four million parameters against a whisper of signal

The encoder defaults: LSTM, hidden size $n = 256$, $L = 8$ stacked layers. Parameter count for an LSTM layer is $4(n(n + d) + n)$ — four gates, each with input and recurrent weights and a bias — so with d small, layer one costs $4(256 \cdot 256 + 256 \cdot d + 256) \approx 270\text{K}$ and layers two through eight, whose input dimension is n , cost $4(256^2 + 256^2 + 256) \approx 525\text{K}$ each: roughly *four million trainable parameters* before the head. Chapter 1 priced the other side of the scale: a generous panel offers order 10^6 rows whose effective information content is orders of magnitude lower (§1.3.3), pursuing a signal of $R^2 \sim 10^{-3}$. This is not a subtle bias–variance trade-off to be tuned; it is a category error, resolvable by arithmetic before any experiment is run.

The repair (Chapters 9 and 11): a one-layer GRU with $n = 32$ — about $3(n(n+d)+n) \approx 3,500$ parameters — with the GRU-over-LSTM choice itself an application of the same arithmetic (three gates versus four: a 25% parameter cut at equal hidden size is pure variance reduction at this data scale). The deeper practice being taught: *effective-sample arithmetic is a design-time activity*. Parameter budgets in this field are set by the noise floor, not by what fits in GPU memory, and “bigger model” claims must clear a capacity-matched baseline (Chapter 16) precisely because capacity itself is a confounder.

Defect 6 — the encoder’s summary influences nothing trainable

Follow the information flow in Figure 3.1. The experts read $x[:, -1, :-1]$ — a raw slice of the input window — not the encoder’s state. The encoder state reaches only the classifier, whose routing decision is gradient-dead by defect 1. Net effect: the LSTM’s summary of history influences no trainable pathway. Four million parameters compute a representation whose only consumer cannot learn. The model is, functionally, two MLPs on a raw feature slice plus an elaborate, untrained switch.

The repair is a genuine *decision*, not a reflex — because once defect 1 is fixed and the gate trains, “what should the experts see?” has two defensible answers. Feed experts the snapshot features only, and the encoder specializes in *state* while experts specialize in *cross-section* — clean division of labor, fewer parameters, and the regime interpretation of Chapter 2 stays legible. Feed them snapshot $\oplus$ encoder state, and experts can condition on history — more capacity, blurrier semantics, and the gate/expert division of labor can collapse (if the experts see the state, they can privately un-mix the regimes and the gate’s job evaporates). The corrected design makes this Decision A: snapshot-only by *default*, switchable by configuration, with the trade-off documented and the switch exposed as an ablation (Chapter 9). Defects that are really decisions deserve to be decided in daylight.

Defect 7 — $x[:, -1, :-1]$: the undocumented reserved column

The slice quietly asserts a convention: *the last feature column is not a feature*. Somewhere upstream, the data pipeline packed something — by all evidence the *target* — into the final column of the feature tensor, and every consumer must remember to slice it off. The contract lives in a -1 in the middle of a forward pass: undocumented, untyped, unchecked. Now enumerate the failure modes. A new feature appended after the reserved column shifts the target *into*

the model’s view: instant, silent, catastrophic look-ahead (the model “predicts” returns it was shown). A consumer that forgets to slice trains on the target directly. A pipeline change that drops the reserved column makes every downstream slice off by one — silently, since shapes still match. By Remark 1.6, the first failure mode is the one you will never find, because it makes results *better*.

The repair (Chapter 12): a named feature schema — every column of both tensors has a name checked at panel construction; the target is a separate tensor of the `Panel`, so there *is no* reserved column to slice off; and the model never slices feature tensors, anywhere. The class-level lesson: *data-layout conventions are contracts, and contracts that live in comments (or worse, in a -1) eventually get broken by someone who never saw them*. Encode contracts in types, schemas, and constructors that refuse to build the invalid object.

Defect 8 — default dimensions that disagree

`C_L` defaults its hidden size to 256; `C_F` defaults its expected input dimension to 64. Instantiated with defaults, the pieces do not fit — and whether that produces a loud shape error or a silent mis-projection depends on incidental details of which linear layer touches the tensor first. The defect is not the mismatch itself but the *design that made it possible*: each module carries its own copy of a shared quantity, and nothing owns the truth. Configuration drift is the generic name — two defaults, one meaning, no reconciliation.

The repair (Chapters 9 and 11): a single configuration tree in which every dimension is written exactly once, cross-validated at construction time (`NECConfig.validate()` checks every derived dimension and raises with names and numbers), with derived quantities *computed*, never restated — the gate’s input dimension, for instance, is a property derived from the encoder’s hidden size and is deliberately not a settable field, making the prototype’s mismatch *unrepresentable*. Shape assertions at module boundaries (`assert_shape`, with tensor names in the error) catch what validation cannot. The slogan for the whole family: *make illegal states unrepresentable, and make the remaining errors loud and early*.

3.3 A taxonomy, and the shape of the cure

Eight defects, four families — and the families, not the instances, are what the rest of the book is organized around:

family	defects	defense layer	built in
Objective pathologies — the loss cannot train what it must	1, 2, 3	derive the objective and its gradients <i>before</i> coding; fused log-domain losses; gradient-flow regression tests	Part II
Leakage channels — information crosses the time boundary	4, 7	data contracts (schema, separate target tensor); leakage stated as executable invariants; no cross-sample ops in models	Parts III, IV
Capacity delusions — parameters vastly outnumber effective samples	5, 6	effective-sample arithmetic at design time; small defaults; capacity-matched baselines; decisions-as-ablations	Parts II, III, V
Contract failures — modules disagree about shared truths	8	single config tree, validated at construction; derived (not settable) dimensions; shape assertions at boundaries	Part III

Notice what is *absent* from the defense column: “be more careful.” None of the eight defects would have been prevented by vigilance, because each was invisible at the moment of writing — the code ran, the loss fell, and (Remark 1.6) the failures that helped the numbers had negative incentive to be found. Every defense above is *structural*: a derivation that must exist before the loss is coded, a constructor that refuses the invalid object, a test that fails on the defect, an API that does not offer the dangerous operation. The prototype’s deepest lesson is that in this field, correctness is a property of *systems*, not of authors.

Engineering consequence — every defect becomes a test

The corrected package operationalizes the ledger as a regression suite: for each defect, a test that *would have failed on the prototype*. Defect 1: after one backward pass of the mixture NLL, the gate’s weight gradient is asserted nonzero — and a companion test builds an argmax-routing double and asserts its gate gradient is *exactly* zero, pinning the defect itself, not just the fix. Defect 4: LayerNorm’s output for a sample is asserted invariant to its batch companions. Defect 7: a target column smuggled into a feature tensor changes its width and fails the schema check. Defect 8: constructing the config with mismatched dimensions raises at build time. Tests-as-theorems is the working style of the whole project — Chapter 19 is devoted to it — and it began here, as the only honest response to a prototype that had passed every informal inspection.

3.4 The road ahead

The book now proceeds in the only defensible order: theory first, because five of the eight defects (1, 2, 3, 5, 6) are visible *only* to someone who has derived the objective and priced the statistics — no amount of code review finds a gradient grave if the reviewer does not know what the gradient must be.

defect	repaired in	by
1	Ch. 5, 6	mixture NLL; the π - r gate gradient; trainable hard routing done right (hard-EM), top-1 renormalization proven dead
2	Ch. 5, 7, 8	all- K likelihood; K as configuration; label permutation confronted honestly
3	Ch. 5, 11	one fused objective; log-domain law; logits-only modules
4	Ch. 11, 12	LayerNorm; no cross-sample operations; leakage invariants
5	Ch. 9, 11	effective-sample arithmetic; GRU $n=32$; capacity-matched baselines (Ch. 16)
6	Ch. 9	Decision A — expert inputs decided, documented, switchable
7	Ch. 12	feature schema; target outside the feature tensors
8	Ch. 9, 11	single validated config tree; derived dimensions; shape assertions

One closing observation. The prototype was not written by a fool; it was written by the same intelligence that later diagnosed all eight defects. What changed in between was not talent but *method*: derivations before code, arithmetic before architecture, contracts before conventions, and tests that encode the failures already survived. That method is the actual subject of this book. The mixture-of-experts is merely the occasion.

Exercises

Exercise 3.1. (*The gradient grave, exhibited.*) In five lines of PyTorch, build $z \in \mathbb{R}^3$ with `requires_grad=True`, compute $\hat{y} = \text{torch.where}(\arg \max_k \text{softmax}(z)_k = 1, a, b)$ for constants $a \neq b$, backpropagate a loss through $\hat{y}$, and print `z.grad`. Explain the result via the piecewise-constance argument of defect 1. Then replace the hard selection with the probability-weighted combination $\hat{y} = \sum_k \pi_k \mu_k$ and show the gradient comes alive. (This is, in miniature, the package’s `test_gate_receives_gradient` and its `argmax` companion.)

Exercise 3.2. (*Capacity arithmetic.*) (a) Derive the LSTM per-layer parameter count $4(n(n + d_{\text{in}}) + n)$ from the four-gate equations, and the GRU’s $3(n(n + d_{\text{in}}) + n)$. (b) Verify the $\approx 4\text{M}$ figure for the prototype ($n = 256, L = 8$, small d) and compute the corrected encoder’s count (GRU, $n = 32, L = 1, d = 2$). (c) Using Chapter 1’s panel ($N = 500$ names, $T = 2,500$ dates) and the crude rule “at most one parameter per ten effective samples,” bound the affordable parameter count if effective samples are (i) all rows, (ii) rows discounted by cross-sectional effective size $N_{\text{eff}} = 20$. Where does each encoder sit?

Exercise 3.3. (*Hands-on: BatchNorm leaks, LayerNorm does not.*) Construct a toy two-date batch where date 2’s samples have a large common shift (simulating a future volatility spike). Pass it through `nn.BatchNorm1d` in training mode and show date 1’s *normalized* outputs change when date 2’s samples are included versus excluded — information flowed backward in time. Repeat with `nn.LayerNorm` and show date 1’s outputs are bit-identical in both cases. State, in one sentence each, which of the three BatchNorm consequences of defect 4 your experiment exhibited.

Exercise 3.4. (*The reserved column, weaponized.*) Simulate defect 7: build a feature matrix whose final column is the target, train (i) a model that correctly slices it off and (ii) a model that “forgets,” and compare their in-sample and out-of-sample R^2 on a task where the true signal is $R^2 = 0.01$. Then append one extra innocent feature *after* the reserved column (the shift failure) and measure again. Write the two-line schema check that would have made all three configurations fail loudly at construction.

Exercise 3.5. (*Classify new specimens.*) Assign each of the following real-world bugs to a family of Section 3.3 and name the structural defense that catches it: (a) a validation split created with `sklearn.train_test_split(shuffle=True)` on a daily panel; (b) a z-score normalization whose mean and standard deviation were fitted on the full history before the train/test split; (c) an attention model whose mask allows position t to attend to $t+1$ during training only; (d) a config file where `seq_len` appears in three places and a refactor changes two of them; (e) a reinforcement-learning reward computed from next-day prices already visible in the state. (One sentence each.)

Exercise 3.6. (*The judgment call.*) The ledger’s verdict on the prototype’s architecture was *keep*. Write one paragraph for the opposite brief: the strongest honest argument that the encoder–gate–experts skeleton should have been abandoned (consider: Chapter 1’s variance budget, the optimization pathologies previewed in §2.3.3, and what a plain pooled ridge on snapshot features costs to build and maintain by comparison). Then write the rebuttal you find most convincing. Keep both; Part V will referee.

Part II

The Mathematics

Chapter 4

Latent Variables and EM

Chapter 2 committed us to a model with a latent regime. This chapter is about what that commitment costs at estimation time and how the cost is paid. The tools built here — responsibilities, the evidence lower bound, the EM algorithm, and the gradient identity connecting them — are not background: the training loop of the thesis model is these tools, executed by an optimizer. Everything is done for the plain, unconditional Gaussian mixture first, where every formula is closed-form and every pathology is visible. The conditional version (Chapter 5) will then be a change of parameterization, not a change of theory.

4.1 The trouble with sums inside logs

Fix the model class from Chapter 2, stripped of covariates: a K -component Gaussian mixture with weights $\pi_k > 0$, $\sum_k \pi_k = 1$, means μ_k and scales σ_k ,

$$p(y \mid \theta) = \sum_{k=1}^K \pi_k \mathcal{N}(y; \mu_k, \sigma_k^2), \quad \theta = (\pi_{1:K}, \mu_{1:K}, \sigma_{1:K}). \quad (4.1)$$

Given a sample $y_1, \dots, y_n$, maximum likelihood asks us to maximize

$$\ell(\theta) = \sum_{j=1}^n \log \sum_{k=1}^K \pi_k \mathcal{N}(y_j; \mu_k, \sigma_k^2). \quad (4.2)$$

Contrast this with the single-Gaussian case, where the log reaches the density directly and the problem separates into closed-form moments. Here a *sum sits inside the log*, and the log of a sum factorizes into nothing. The obstruction has a precise interpretation: the sum is a marginalization. Introduce the latent assignment $z_j \in \{1, \dots, K\}$ — “which component generated observation j ” — with $\mathbb{P}(z_j = k) = \pi_k$ and $y_j \mid z_j = k \sim \mathcal{N}(\mu_k, \sigma_k^2)$. Then (4.1) is exactly $p(y_j) = \sum_k p(z_j = k) p(y_j \mid z_j = k)$: the mixture is a joint model over (y, z) observed only through y . The likelihood (4.2) is called the *incomplete-data* likelihood for this reason.

If a benevolent oracle revealed the z_j , estimation would collapse to triviality: the *complete-data* log-likelihood

$$\ell_c(\theta; z) = \sum_j \log(\pi_{z_j} \mathcal{N}(y_j; \mu_{z_j}, \sigma_{z_j}^2)) = \sum_j \sum_k \mathbf{1}\{z_j = k\} \log(\pi_k \mathcal{N}(y_j; \mu_k, \sigma_k^2)) \quad (4.3)$$

separates over k , and each component is fit by the weighted moments of its own points. Everything that follows — EM, responsibilities, the ELBO — is machinery for acting *as if* we had the oracle, replacing the indicator $1\{z_j = k\}$ by our best current guess of it.

4.2 Responsibilities

The best current guess has a name and a formula.

Definition 4.1 (Responsibility). Under parameters θ , the *responsibility* of component k for observation y_j is the posterior assignment probability

$$r_{jk} = \mathbb{P}(z_j = k \mid y_j, \theta) = \frac{\pi_k \mathcal{N}(y_j; \mu_k, \sigma_k^2)}{\sum_l \pi_l \mathcal{N}(y_j; \mu_l, \sigma_l^2)}. \quad (4.4)$$

This is Bayes' rule with the mixture weights as prior and the component densities as likelihoods. Three readings, all load-bearing later:

1. *Soft assignment.* $r_{jk} \in (0, 1)$ and $\sum_k r_{jk} = 1$: each point is fractionally allocated among components, in proportion to how well each explains it.
2. *Inference output.* In the regime model, r_{jk} is the regime belief given the data — the quantity a desk would actually consume (“what regime are we in?”). The training-time object and the interpretive object are the same tensor.
3. *Gradient carrier.* Every gradient of ℓ will turn out to route through r ; a model whose r is degenerate (all mass on one component, or frozen at uniform) has degenerate learning signal. Several of Chapter 3's defects and Chapter 6's theorems are statements about broken responsibility flow.

4.3 The evidence lower bound

The central identity of this chapter decomposes the intractable objective into two tractable pieces. Let $q = (q_1, \dots, q_K)$ be *any* probability vector over the components of one observation y (the n -observation version is a product over j).

Proposition 4.2 (ELBO decomposition). *For any distribution q over z with $q_k > 0$,*

$$\log p(y \mid \theta) = \underbrace{\sum_k q_k \log \frac{\pi_k \mathcal{N}(y; \mu_k, \sigma_k^2)}{q_k}}_{\mathcal{L}(q, \theta) \text{ (the ELBO)}} + \underbrace{\text{KL}(q \parallel r(\theta))}_{\geq 0}, \quad (4.5)$$

where $r(\theta)$ is the responsibility vector (4.4). Hence $\mathcal{L}(q, \theta) \leq \log p(y \mid \theta)$ for every q , with equality if and only if $q = r(\theta)$.

Proof. Write $p_k = \pi_k \mathcal{N}(y; \mu_k, \sigma_k^2)$, so $p(y) = \sum_k p_k$ and $r_k = p_k / p(y)$. Then

$$\mathcal{L}(q, \theta) = \sum_k q_k \log \frac{p_k}{q_k} = \sum_k q_k \log \frac{r_k p(y)}{q_k} = \log p(y) - \sum_k q_k \log \frac{q_k}{r_k} = \log p(y) - \text{KL}(q \parallel r).$$

Non-negativity of KL (Jensen on the convex function $u \mapsto u \log u$) gives the bound; KL vanishes iff $q = r$. $\square$

The decomposition converts “maximize the awkward $\ell(\theta)$ ” into “push up a bound that touches it.” The bound has a second expansion that explains why pushing it up is easy:

$$\mathcal{L}(q, \theta) = \underbrace{\sum_k q_k \log(\pi_k \mathcal{N}(y; \mu_k, \sigma_k^2))}_{\mathbb{E}_q[\ell_c] \text{ — expected complete-data log-lik}} + \underbrace{\left(-\sum_k q_k \log q_k\right)}_{H(q), \text{ no } \theta}. \quad (4.6)$$

For fixed q , maximizing $\mathcal{L}$ over θ is maximizing the expected complete-data log-likelihood — the oracle problem of Section 4.1 with indicators replaced by q , which separates over components and is closed-form for Gaussians.

4.4 EM: coordinate ascent on the bound

The two observations — equality at $q = r(\theta)$, and easy θ -maximization at fixed q — interleave into the *expectation–maximization* algorithm:

step	action	effect on (4.5)
E	$q^{(t)} \leftarrow r(\theta^{(t)})$	bound touches: $\mathcal{L}(q^{(t)}, \theta^{(t)}) = \ell(\theta^{(t)})$
M	$\theta^{(t+1)} \leftarrow \arg \max_{\theta} \mathcal{L}(q^{(t)}, \theta)$	bound rises under a fixed q

For the Gaussian mixture the M-step is the weighted-moments update (Exercise 4.1 derives it, including the Lagrange step for the weights):

$$\pi_k \leftarrow \frac{1}{n} \sum_j r_{jk}, \quad \mu_k \leftarrow \frac{\sum_j r_{jk} y_j}{\sum_j r_{jk}}, \quad \sigma_k^2 \leftarrow \frac{\sum_j r_{jk} (y_j - \mu_k)^2}{\sum_j r_{jk}}. \quad (4.7)$$

Theorem 4.3 (Monotonicity of EM). $\ell(\theta^{(t+1)}) \geq \ell(\theta^{(t)})$ for every iteration, with equality only at stationary points of ℓ .

Proof. Chain the two properties through the bound:

$$\ell(\theta^{(t+1)}) \stackrel{(a)}{\geq} \mathcal{L}(q^{(t)}, \theta^{(t+1)}) \stackrel{(b)}{\geq} \mathcal{L}(q^{(t)}, \theta^{(t)}) \stackrel{(c)}{=} \ell(\theta^{(t)}),$$

where (a) is the ELBO inequality of Proposition 4.2 at the new θ , (b) is the definition of the M-step, and (c) is the equality achieved by the E-step. No step size, no line search, no learning rate: the likelihood cannot decrease. $\square$

Figure 4.1 shows the geometry on a one-parameter slice: each E-step erects a minorizing surrogate tangent to ℓ at the current iterate; each M-step jumps to the surrogate’s peak; the true objective at the landing point is at least as high as at takeoff.

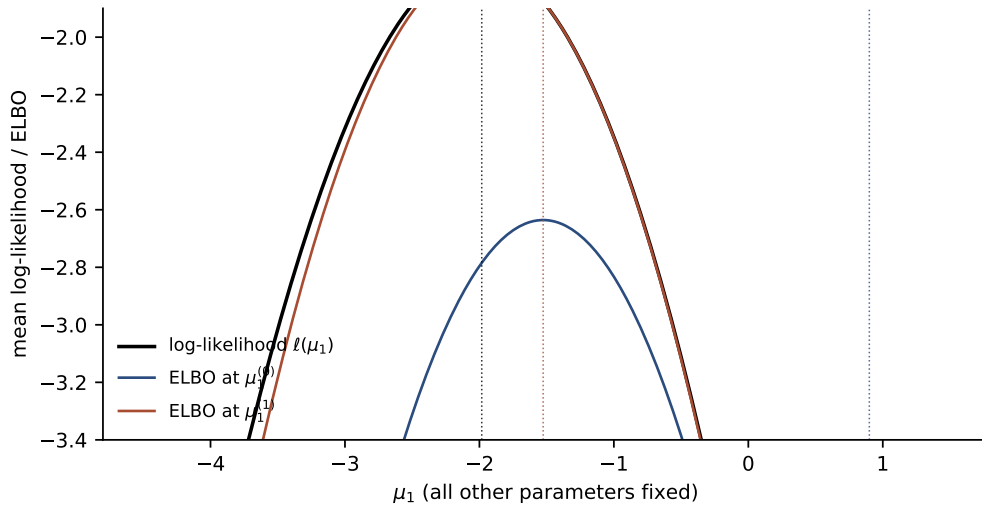


Figure 4.1: EM as bound tangency. The mixture log-likelihood (black) over one coordinate (μ_1 ; all else fixed), with the ELBO surrogates erected by two successive E-steps. Each surrogate touches ℓ at its anchor point and minorizes it everywhere else; maximizing the surrogate (the M-step) moves the anchor uphill. Note also what the figure quietly demonstrates: ℓ itself is multimodal — the bound climbs to whichever peak its start point drains toward (Chapter 8).

Origins — EM before and after 1977

Weighted-reassignment iterations for mixtures date at least to Newcomb (1886); the modern formulation, name, and general monotonicity proof are Dempster, Laird and Rubin, “Maximum Likelihood from Incomplete Data via the EM Algorithm” (*JRSS-B*, 1977) — one of the most cited statistics papers ever written, covering missing data, censoring, and latent structure in one framework. The view used in this chapter — EM as coordinate ascent on a single objective $\mathcal{L}(q, \theta)$ over *both* arguments — is due to Neal and Hinton, “A View of the EM Algorithm that Justifies Incremental, Sparse, and Other Variants” (1998), and it is the version that matters for us: once EM is coordinate ascent, *partial* steps in either coordinate remain valid, which is the door through which stochastic gradients enter (Section 4.6). The same objective under the name “variational free energy” is the foundation of variational inference at large; Proposition 4.2 is the small-print contract behind the modern ELBO industry.

4.5 The gradient identity

EM’s closed-form M-step (4.7) is a Gaussian-mixture privilege. The thesis model’s components are neural networks; no closed form will exist. The bridge to gradient training is an identity worth proving twice.

Proposition 4.4 (Fisher’s identity). *For the mixture (4.1),*

$$\frac{\partial \ell}{\partial \theta} = \sum_j \sum_k r_{jk} \frac{\partial}{\partial \theta} \log(\pi_k \mathcal{N}(y_j; \mu_k, \sigma_k^2)) : \quad (4.8)$$

the gradient of the incomplete-data log-likelihood equals the responsibility-weighted gradient of the complete-data log-likelihood.

Proof. Differentiate one term of (4.2) directly. With $p_{jk} = \pi_k \mathcal{N}(y_j; \mu_k, \sigma_k^2)$,

$$\frac{\partial}{\partial \theta} \log \sum_k p_{jk} = \frac{\sum_k \partial_\theta p_{jk}}{\sum_l p_{jl}} = \sum_k \frac{p_{jk}}{\sum_l p_{jl}} \frac{\partial_\theta p_{jk}}{p_{jk}} = \sum_k r_{jk} \partial_\theta \log p_{jk}. \quad \square$$

The one-line mechanism — $\partial \log \sum_k a_k = \sum_k (a_k / \sum a) \partial \log a_k$ — says that *differentiating through a marginalization automatically performs posterior inference*: the weights that appear are exactly the Bayes posterior. Nobody asks the optimizer to compute responsibilities; they emerge from the chain rule. This is the sense in which “just take gradients of the NLL” does not bypass EM but contains it.

4.6 Generalized EM, and EM as the trainer’s semantics

Theorem 4.3’s proof used the M-step only through inequality (b): any $\theta^{(t+1)}$ that *improves* $\mathcal{L}(q^{(t)}, \cdot)$ — not necessarily maximizes it — preserves monotonicity. Algorithms of this form are *generalized EM* (GEM). One gradient step on the expected complete-data log-likelihood improves it (for small enough step); by Fisher’s identity that gradient *equals* the gradient of ℓ itself. Stacking the pieces:

SGD on the mixture NLL is generalized EM in which the E-step is implicit in the forward pass and the M-step is one gradient step.

This sentence is the semantic contract of the package’s training loop, and it earns its box:

Engineering consequence — the trainer is an EM loop wearing an optimizer

The package never implements an E-step or an M-step. It implements a forward pass that computes the joint scores $a_{jk} = \log \pi_k + \log \mathcal{N}(y_j; \mu_k, \sigma_k^2)$, a fused loss — $\text{LSE}_k(a_{jk})$, and a call to `loss.backward()`. By Fisher’s identity, autograd’s gradient is the responsibility-weighted complete-data gradient — responsibilities included, at the current parameters, for the current minibatch. AdamW’s update is a (preconditioned, momentum-carrying) partial M-step. What is *lost* relative to classical EM is the monotonicity guarantee — minibatch noise and adaptive step sizes void Theorem 4.3’s inequality chain — which is why the package’s training diagnostics (Chapter 10) monitor the NLL rather than assume it falls. What is *gained* is everything: neural components, minibatch scaling, one optimizer for every module, and a single code path shared with the HMM variant (Chapter 7). The trade was accepted with its eyes open, and the vocabulary of this chapter — “the E-step is in the forward pass” — is used verbatim in the design document.

Code walkthrough — `nec_moe/losses.py` --- the fused objective

The entire chapter compresses into eleven lines of the package:

```
a = log_prior + log_lik          # (B, K): a_k = log pi_k + log N_k
lse = torch.logsumexp(a, dim=-1)  # (B,): log p(y | x), the evidence
per_sample = -lse                # the NLL
```

```

log_filtered = a - lse.unsqueeze(-1)  # (B, K): log r, ATTACHED
return MixtureNLLOutput(
    nll=per_sample.mean(),
    per_sample_nll=per_sample,
    log_filtered=log_filtered,
    responsibilities=log_filtered.detach().exp(), # diagnostics copy
)

```

Three deliberate choices. (i) *Log domain end to end*: densities are never exponentiated and re-logged; logsumexp subtracts the row maximum internally, so the computation survives residuals that would underflow float64 densities (a fat-tailed return under a calm component reaches e^{-1500} without blinking). (ii) *One fused pass*: evidence and posterior come from the same tensor a — the E-step really is the forward pass. (iii) *Two exits for r* : `log_filtered` stays attached to the graph because the HMM recursion threads it forward in time (Chapter 7); `responsibilities` is a detached probability-domain copy for dashboards, so no diagnostic can ever become a second, accidental training signal.

4.7 What EM does not promise

Two failure modes survive all of the above, and both have engineering consequences that the package carries visibly.

4.7.1 Local optima

Theorem 4.3 promises ascent, not summit. The mixture likelihood is multimodal — Figure 4.1 shows two peaks in a *one*-dimensional slice — and both EM and SGD converge to the basin of their initialization. For the thesis model the dominant mode of failure is not a bad local peak but a *symmetric saddle* that looks like convergence; Chapter 8 is devoted to it. Flag planted here: nothing in this chapter’s theory prevents it.

4.7.2 The likelihood is unbounded

Proposition 4.5 (Degenerate spikes). *For $K \geq 2$, the Gaussian-mixture likelihood (4.2) is unbounded above: $\sup_{\theta} \ell(\theta) = +\infty$.*

Proof. Park component 1 on a data point: $\mu_1 = y_1$, and let $\sigma_1 \rightarrow 0$ with all other parameters fixed and non-degenerate. The $j = 1$ term contains $\pi_1 \mathcal{N}(y_1; y_1, \sigma_1^2) = \pi_1 / (\sqrt{2\pi} \sigma_1) \rightarrow \infty$, so $\log p(y_1) \rightarrow \infty$ at rate $\log(1/\sigma_1)$. Every other term is bounded below by its (fixed, finite) other-component mass: $\log p(y_j) \geq \log(\pi_2 \mathcal{N}(y_j; \mu_2, \sigma_2^2)) > -\infty$. The sum diverges. $\square$

The global “optimum” of the exact problem is therefore a pathology: one component collapsing onto one observation with vanishing variance. MLE for mixtures only makes sense as a search for a good *interior* local maximum, and every practical mixture fitter constrains scales away from zero.

What breaks without this — unconstrained σ

A mixture fit on returns with free scales and an unlucky initialization will manufacture a “regime” containing one crash day, $\sigma_k \rightarrow 0$, $\text{NLL} \rightarrow -\infty$, and a model that is numerically dead (the next forward pass overflows) while *reporting its best loss ever* — the leakage ratchet’s evil twin: an objective that rewards self-destruction. The package’s defenses are layered and mundane: σ is parameterized as $\log \sigma$ (positivity is structural, and the spike must travel infinitely far in parameter space rather than to the boundary); `sigma_init` calibrates the starting scale to the target’s standard deviation; and `sigma_freeze_steps` holds $\log \sigma$ fixed during early training — the spike is most reachable when means are still garbage, which is precisely when the freeze is on. The freeze keys off `step_count`, which is why checkpoint resume restoring `step_count` exactly (Chapter 10) is a correctness requirement, not a convenience.

Exercises

Exercise 4.1. Derive the M-step (4.7) from (4.6): (a) for μ_k and σ_k^2 by direct differentiation; (b) for the weights π_k by maximizing $\sum_j \sum_k r_{jk} \log \pi_k$ subject to $\sum_k \pi_k = 1$ with a Lagrange multiplier. (c) Where does the constraint qualification fail if some component receives zero total responsibility, and what does the package’s load-balance *monitoring* (not the aux loss — the dashboard) have to do with it?

Exercise 4.2. Prove Fisher’s identity (4.8) a second way: differentiate the decomposition (4.5) with respect to θ at the point $q = r(\theta_0)$, $\theta = \theta_0$, using the fact that the KL term is minimized (hence has zero gradient) there. Conclude that the ELBO and ℓ have identical gradients wherever the bound touches — the reason one gradient step after an E-step is a GEM step.

Exercise 4.3. By hand, run two full EM iterations for a $K = 2$ mixture with fixed $\sigma_1 = \sigma_2 = 1$, $\pi = (\frac{1}{2}, \frac{1}{2})$, on the four observations $y = (-2, -1, 1, 2)$ from the start $\mu = (-0.5, 0.5)$. Report the responsibilities and means at each step. Then argue (no computation) where the iteration converges, and what would happen instead from the start $\mu = (0, 0)$ — you have met this phenomenon before, and you will meet it again in Chapter 8.

Exercise 4.4. Quantify Proposition 4.5’s spike: with $\mu_1 = y_1$ and $\sigma_1 = \epsilon$, show ℓ grows as $\log(1/\epsilon) + O(1)$, and compute how small ϵ must be for the spike to beat an interior optimum by Δ nats. Then explain why the same spike is *harder* (but not impossible) to reach for the conditional model of Chapter 5, where $\mu_k(\cdot)$ is a shared function across observations rather than a free scalar.

Exercise 4.5. (Hands-on.) Implement classical EM (4.4)–(4.7) in ~25 lines of numpy. Generate $n = 2,000$ points from a two-regime scale mixture (Chapter 2’s world, means zero, $\sigma = (0.5, 2.5)$), and (a) verify monotonicity numerically at every iteration; (b) compare the fixed point against the package’s SGD route: build a `ClassicalGaussianEmission` $\times$ uniform-prior model on the same data and train with `Trainer.fit`; both should recover $\sigma \approx (0.5, 2.5)$ up to permutation. (c) Time both. Reflect: what did SGD buy, and what did it cost, on a problem where EM has closed forms?

Exercise 4.6. The E-step was derived as “set q to the posterior.” Show it is also the solution of $\min_q \text{KL}(q \parallel r)$ trivially, and — less trivially — that among all product distributions $q(z_{1:n}) = \prod_j q_j(z_j)$, coordinate ascent on (4.5) recovers exactly the per-observation posteriors, i.e. mean-field structure costs nothing here. Why does this stop being true for the HMM of Chapter 7, where the latent states are chained?

Chapter 5

Mixture of Experts

One substitution turns Chapter 4’s mixture into the thesis model: let the parameters be functions of the observables. The weights become a gate evaluated on the encoded market state; the means become expert networks evaluated on the characteristics. This chapter derives what that substitution does to training — above all the gate-logit gradient $\pi - r$, the two-symbol formula that repairs the prototype’s central defect and that you should be able to reproduce on a napkin for the rest of your career.

5.1 From mixture to conditional mixture

Chapter 4 fit one global mixture: every observation faced the same weights π_k and the same component means μ_k . The thesis model (Definition 2.4) makes both *conditional*:

$$p(y_{i,t} \mid \mathbf{x}_{i,t}, \mathbf{h}_t) = \sum_{k=1}^K \pi_k(\mathbf{h}_t) \mathcal{N}(y_{i,t}; \mu_k(\mathbf{x}_{i,t}), \sigma_k^2), \quad (5.1)$$

with the three modules of Chapter 2’s blueprint:

- the *gate* produces logits $\mathbf{z} = g(\mathbf{h}_t) \in \mathbb{R}^K$ from the encoded market state and weights $\pi = \text{softmax}(\mathbf{z})$ — the regime *prior* before seeing the outcome;
- the *experts* produce means $\mu_k(\mathbf{x}_{i,t})$ from the name-level characteristics — K regime-conditional prediction functions;
- the *scales* σ_k are per-expert free parameters (as $\log \sigma_k$), not functions — each regime carries its own noise level, which Chapter 2 argued is most of what a regime *is*.

Everything in Chapter 4 survives the substitution wholesale, because none of its derivations used the fact that π_k and μ_k were free scalars: responsibilities are still (4.4) with $\pi_k(\mathbf{h}_t)$ and $\mu_k(\mathbf{x}_{i,t})$ inserted; the ELBO decomposition, Fisher’s identity, and the GEM interpretation hold verbatim. What changes is *where gradients go next*: after the chain rule reaches $\partial/\partial\pi_k$ and $\partial/\partial\mu_k$, it continues into network weights instead of stopping at scalars. The new mathematics of this chapter is entirely about that continuation.

5.2 The objective, fused

Define per-sample joint scores and the loss exactly as the package computes them:

$$a_k = \log \pi_k + \log \mathcal{N}(y; \mu_k, \sigma_k^2), \quad \text{NLL} = -\text{LSE}_k(a_k) = -\log \sum_k e^{a_k}, \quad (5.2)$$

with $\log \pi = \log_{\text{softmax}}(z)$ and the Gaussian log-density expanded in the log domain,

$$\log \mathcal{N}(y; \mu_k, \sigma_k^2) = -\frac{1}{2} \log 2\pi - \log \sigma_k - \frac{(y - \mu_k)^2}{2\sigma_k^2}.$$

Numerically, LSE subtracts the row maximum before exponentiating, so the pipeline never materializes a density — only densities' logs — and never underflows. The responsibilities emerge from the same tensor: $\log r_k = a_k - \text{LSE}(a)$, i.e. $r = \text{softmax}(a)$. One fused computation yields objective and posterior; Chapter 4's E-step is literally a by-product of evaluating the loss.

What breaks without this — the prototype's separated pipeline

The prototype (defect 3) placed `nn.Softmax` *inside* the model and attached a separate cross-entropy loss to the gate. Two distinct failures follow. Numerical: probabilities near 0 or 1 followed by a downstream log is catastrophic cancellation by design — $\log(\text{softmax})$ loses precision exactly where confident routing lives; the fused $\log_{\text{softmax}}$ /LSE path is algebraically identical and numerically stable everywhere. Statistical: a gate trained by its own auxiliary loss against external labels is no longer maximizing (5.1)'s likelihood — the mixture's components stop negotiating through a shared objective, and the “prior” stops being the quantity the likelihood needs it to be. One model, one objective, one log domain — the entire fix.

5.3 The gate gradient

Here is the book's most important derivation. It is four lines.

Theorem 5.1 (The gate learns $\pi - r$). *For the fused objective (5.2) with $\pi = \text{softmax}(z)$,*

$$\frac{\partial \text{NLL}}{\partial z_k} = \pi_k - r_k. \quad (5.3)$$

Proof. Two softmax Jacobian passes, chained. First, $\text{NLL} = -\text{LSE}(a)$ gives $\partial \text{NLL} / \partial a_j = -\text{softmax}(a)_j = -r_j$. Second, $a_j = z_j - \text{LSE}(z) + \log \mathcal{N}_j$ (writing $\log \pi_j = z_j - \text{LSE}(z)$), so $\partial a_j / \partial z_k = \delta_{jk} - \pi_k$, which does not depend on the likelihood term. Chain and use $\sum_j r_j = 1$:

$$\frac{\partial \text{NLL}}{\partial z_k} = \sum_j (-r_j) (\delta_{jk} - \pi_k) = -r_k + \pi_k \sum_j r_j = \pi_k - r_k. \quad \square$$

Every property of this formula deserves a sentence.

It is a Bayes update, imposed as a gradient. The gate's output π is the prior over regimes; r is the posterior after the outcome y is seen. Gradient descent moves the logits to shrink

$\pi - r$: the gate is perpetually trained toward *its own posterior*. The mixture teaches its prior to predict what its likelihood will conclude — self-distillation of Bayes’ rule, with no labels anywhere. Decision B (the gate stays unsupervised) is not an abstinence; it is the recognition that (5.3) already supplies the supervision, and that any external label would overwrite the only training signal that makes “do the learned regimes mean anything?” an open, testable question.

It is softmax cross-entropy with soft targets. Classification training gives logit gradients $p - e_y$ (predicted probabilities minus the one-hot label). Formula (5.3) is the same expression with the hard label replaced by the responsibility vector: the experts jointly manufacture the label the gate is trained against, each date, each name. Everything known about cross-entropy training — calibration at the optimum, gradient boundedness, saturation behavior — transfers.

It vanishes exactly at calibration. $\partial \text{NLL} / \partial \mathbf{z} = 0$ iff $\pi = r$: the stationary gate is the one whose prior already matches the posterior on average (Exercise 5.4 makes the “on average” precise). A well-trained gate is thus a *calibrated* regime predictor by construction — the property Chapter 18’s reliability diagnostics test empirically.

It is what the prototype destroyed. Route through `argmax` instead of the mixture and the entire derivation collapses at line one: the loss ceases to depend on $\mathbf{z}$ except through a piecewise-constant selection, and the gradient is zero almost everywhere (Chapter 3, defect 1; Chapter 6 proves the subtler renormalized variant). The package’s regression test `test_gate_receives_gradient` asserts (5.3) is alive after one backward pass — and its companion asserts the `argmax` double is exactly dead. The two-line formula is guarded by a two-assertion test.

5.4 The expert and scale gradients

Differentiating (5.2) with respect to the remaining outputs (Exercise 5.1 checks all of these against autograd):

$$\frac{\partial \text{NLL}}{\partial \mu_k} = r_k \frac{\mu_k - y}{\sigma_k^2}, \quad (5.4)$$

$$\frac{\partial \text{NLL}}{\partial \log \sigma_k} = r_k \left(1 - \frac{(y - \mu_k)^2}{\sigma_k^2} \right). \quad (5.5)$$

Equation (5.4) is *responsibility-weighted regression*: expert k receives an ordinary squared-error pull toward y , scaled by how responsible the posterior holds it. Experts with $r_k \approx 0$ on a sample are untouched by it — specialization is not just permitted but enforced by the gradient’s structure. Equation (5.5) is stationary when σ_k^2 equals the responsibility-weighted mean squared residual of expert k ’s own samples: each regime’s noise level is estimated from the data that regime claims. Summing over a batch and setting to zero recovers exactly Chapter 4’s M-step (4.7) — as it must, by Fisher’s identity.

The full division of labor, in one table:

quantity	gradient	EM reading	failure if broken
gate logits $\mathbf{z}$	$\pi - r$	prior $\rightarrow$ posterior	defect 1 (dead gate)
means μ_k	$r_k(\mu_k - y) / \sigma_k^2$	weighted M-step	pooled experts (Ch. 8)
scales $\log \sigma_k$	$r_k(1 - (y - \mu_k)^2 / \sigma_k^2)$	weighted variance	spikes / flat r (Ch. 4)
responsibilities r	(not a parameter)	E-step, in the forward pass	—

Origins — Adaptive mixtures of local experts

The architecture and the competitive-training insight are from Jacobs, Jordan, Nowlan and Hinton, “Adaptive Mixtures of Local Experts” (*Neural Computation*, 1991). Their central observation is worth retelling because it is a loss-design lesson. An earlier “committee” objective, $(y - \sum_k \pi_k \mu_k)^2$, trains the *blended* prediction: each expert’s target is the residual left by the blend, every expert ends up correcting every other, and the ensemble *cooperates* into homogeneity. Switching to the mixture (log-)likelihood — $\sum_k \pi_k \exp(-\frac{1}{2}(y - \mu_k)^2)$ in their Gaussian case — makes experts *compete* for responsibility on each case, and specialization follows. The moral, which recurs in Chapter 6: with mixtures, the difference between an architecture that specializes and one that homogenizes is not the wiring but the objective. Jordan and Jacobs (1994) supplied the EM view and hierarchical gates; Shazeer et al. (2017) scaled sparse variants to language models, whose load-balancing pathologies reappear in our Section 6.1 context. The thesis model is the 1991 design, taken seriously with 2020s hygiene.

5.5 Prediction and uncertainty

Training maximizes a density; the desk needs numbers. The model’s point prediction is the mixture mean,

$$\hat{y} = \mathbb{E}[y \mid \mathbf{x}, \mathbf{h}] = \sum_k \pi_k \mu_k(\mathbf{x}), \quad (5.6)$$

— the posterior-weighted blend, *not* the best expert’s output. Selecting $\mu_{\arg \max_k \pi_k}$ discards the model’s own uncertainty about the regime and reintroduces a discontinuity (as the gate wavers across 0.5, predictions jump), for zero statistical benefit: under squared error the conditional mean is optimal, and under rank-IC grading any monotone transform of it scores identically.

The predictive *variance* decomposes by the law of total variance (Exercise 5.3):

$$\text{Var}(y \mid \mathbf{x}, \mathbf{h}) = \underbrace{\sum_k \pi_k \sigma_k^2}_{\text{within-regime noise}} + \underbrace{\sum_k \pi_k \mu_k^2 - \hat{y}^2}_{\text{between-regime disagreement}}. \quad (5.7)$$

The second term is zero exactly when the experts agree; it is the model’s self-reported “the regimes disagree about this name” signal — information a single-Gaussian model structurally cannot emit, and one of the concrete payoffs of carrying a full conditional density rather than a point estimate (Section 1.4’s argument, now with a formula).

5.6 The interface: log-prior versus training weights

One design decision in the package’s loss layer belongs to the theory and must be stated now. The fused objective (5.2) consumes *some* log-weight vector alongside the expert log-likelihoods. For the soft gate the choice is unambiguous: the log-prior $\log_{\text{softmax}}(\mathbf{z})$, both for training and for held-out evaluation. Chapter 6’s hard-routing variant will need the two roles

to *differ*: the quantity that makes its objective trainable (a hard-EM surrogate) is not the quantity that honestly states its predictive distribution (the selected expert’s density). The package’s `PriorOutput` therefore carries `log_prior` always, and optionally `log_train_weights`; the trainer uses the training weights when present *for the loss only*, and every held-out NLL in every table of Part V is computed under `log_prior` — the model’s honest predictive stance. This is a small interface with a large moral: *when a training surrogate and a predictive distribution diverge, name both, and never grade on the surrogate.*

Code walkthrough — `nec_moe/model.py` + `likelihood.py` --- assembly

The forward pass assembles (5.1) in five steps, each one module, each module owning exactly one formula of this chapter:

```
h_t = self.encoder(x_seq)           # (B, n)   market state
z   = self.gate(h_t)                # (B, K)   logits -- no softmax!
out = self.prior(z, ctx)             # log pi   (pluggable, Ch. 6-7)
mu, log_sigma = self.experts(x_exp)  # (B, K), (K,)
log_lik = expert_log_likelihood(mu, log_sigma, y)  # (B, K)
nll_out = mixture_nll(out.log_prior, log_lik)      # fused (Ch. 4)
```

The gate module ends at logits — the softmax lives inside the fused loss (defect-3 fix, enforced by the architecture rather than by discipline). `expert_log_likelihood` is a pure, parameter-free function computing the Gaussian log-density directly in log domain; its purity is what makes “same likelihood, different prior” literally true when Chapter 7 swaps the prior box. And the un-fused alternative — `prob.log()` after a softmax, densities exponentiated then logged — is not merely discouraged: no code path in the package can express it.

Exercises

Exercise 5.1. (Hands-on.) Build a tiny mixture forward pass in raw torch (no package): random $z \in \mathbb{R}^{1 \times 3}$ with `requires_grad`, fixed $\mu \in \mathbb{R}^{1 \times 3}$, $\sigma = 1$, scalar y . Compute the fused NLL and `backward()`. Verify numerically that `z.grad` equals $\pi - r$, that `mu.grad` equals (5.4), and (promoting $\log \sigma$ to a parameter) that its gradient matches (5.5). Then break it: recompute with `torch.where(torch.argmax(z) ...)` routing and confirm `z.grad` is zero — you have reproduced both halves of the package’s defect-1 regression test.

Exercise 5.2. Show from (5.3) that $\sum_k \partial \text{NLL} / \partial z_k = 0$ for every sample. Connect this to the softmax’s invariance under adding a constant to all logits, and explain why the gate’s logits therefore never receive a signal about their overall level — only about their differences. What (harmless) consequence does this have for the learned logits over a long training run, and how does the gate head’s LayerNorm + weight decay (Chapter 9) keep it tidy?

Exercise 5.3. Derive (5.7) from the law of total variance conditioning on the component index. Then compute both terms for a two-expert model with $\pi = (0.5, 0.5)$, $\mu = (+1, -1)$, $\sigma = (1, 1)$ and for $\pi = (0.95, 0.05)$, same μ, σ . Which situation would a risk manager rather know about, and which summary — $\hat{y}$ alone or (5.7)’s split — reveals it?

Exercise 5.4. Define the gate’s *calibration residual* as $\mathbb{E}[r \mid \mathbf{h}] - \pi(\mathbf{h})$. Show that the expected gate-logit gradient at fixed $\mathbf{h}$ is exactly its negative, so a stationary gate satisfies $\pi(\mathbf{h}) = \mathbb{E}[r \mid \mathbf{h}]$: the prior equals the average posterior, conditionally. Explain (two sentences) why this is precisely the property that Chapter 18’s reliability diagram tests on held-out data, and why finding it *violated* out-of-sample is evidence of regime drift rather than a training bug.

Exercise 5.5. (Decision A’s gradient anatomy.) Suppose the experts read the concatenation $[x; \mathbf{h}_t]$ (`input_mode="snapshot_plus_hidden"`) instead of x alone. Write the gradient of the per-sample NLL with respect to the encoder state $\mathbf{h}_t$ as a sum of a gate path and an expert path, and show the expert path is $\sum_k r_k \frac{\mu_k - y}{\sigma_k^2} \partial \mu_k / \partial \mathbf{h}_t$. In the prototype, defect 1 killed the gate path and defect 6 severed the expert path — conclude what total gradient its four-million-parameter encoder received from the prediction task, and why the corrected default (snapshot-only experts) makes the gate path the encoder’s *sole*, and therefore interpretable, training signal.

Exercise 5.6. Reproduce the 1991 cooperation/competition contrast in numpy: two linear experts, a fixed uniform gate, data from the sign-flip world (Proposition 2.3). Train once on the committee loss $(y - \frac{1}{2}(\mu_1 + \mu_2))^2$ and once on the mixture NLL, from the same asymmetric initialization $(b_1, b_2) = (0.8, -0.6)$. Report the terminal slopes. One objective drives both experts to the pooled solution; the other preserves specialization. Explain each outcome from the two losses’ gradients.

Chapter 6

The Routing Family

Variation 2 of the thesis is not “a model” but a controlled comparison: hold the encoder, experts, likelihood, data and evaluation fixed, and vary only the mechanism that turns gate logits into mixture weights. This chapter derives each mechanism in the family — soft, uniform, hard, top-k, Gumbel — as a small theorem about gradients, including the one genuine trap (renormalized top-1 looks soft and is provably dead), and ends with the auxiliary losses and the scope argument that decides where load balancing is measured. The design payoff is stated up front: because every mechanism is a pluggable “prior” behind one interface, the entire comparison grid is a config sweep, not a rewrite.

6.1 One interface, five mechanisms

Chapter 5 fixed the fused objective $NLL = -LSE_k(\log w_k + \log \mathcal{N}_k)$ and left one degree of freedom: which log-weights $\log w$ enter it. Abstract that choice into a component — the package calls it a `RegimePrior` — with the contract:

input: gate logits z (and, for Chapter 7, a carried state) $\mapsto$ *output:* $\log \pi$ (predictive weights), optionally $\log \tilde{w}$ (training weights).

Every mechanism below is one implementation of this contract; the shared likelihood core and the trainer never change. The five memoryless members, in one table, each row a theorem proved in this chapter:

kind	$\log \pi$ (predictive)	train weights	gate gradient	role
soft	$\log_softmax(z)$	same	$\pi - r$	baseline
uniform	$-\log K$ (frozen)	same	0 by design	routing-value ablation
hard	one-hot at $\arg \max z$	$\log \pi$ masked to $\arg \max$	$\pi - e_{sel}$	trainable top-1
topk	softmax over top- k logits	same	alive iff $k \geq 2$	sparse routing
gumbel	$\log_softmax((z + g)/\tau)$	same	stochastic $\pi - r$	annealed exploration

Engineering consequence — the ablation grid is a config sweep

`PriorConfig(kind="soft"|"uniform"|"hard"|"topk"|"gumbel"|"hmm")` selects the mechanism through a registry; nothing else in the system knows which one is active. This is not convenience — it is experimental *validity*. The thesis question “does routing mechanism X beat Y?” is only well-posed if X and Y differ in the mechanism alone; the moment each variant is its own script, the comparison silently absorbs differences in data handling, initialization, or evaluation. One interface, many mechanisms, byte-identical everything else — the same logic that will make Chapter 16’s baselines fair.

6.2 Soft and uniform: the two ends

`soft` is Chapter 5 verbatim: $\log \pi = \log_{\text{softmax}}(z)$, gradient $\pi - r$, the baseline against which everything else is judged.

`uniform` freezes $\log \pi \equiv -\log K$. By Theorem 5.1’s derivation the gate gradient is identically zero — here *by design*: the mechanism is the control arm that answers “how much of the mixture’s value is the *routing*, as opposed to merely having K experts and a mixture likelihood?” The experts still specialize or not on their own: with uniform weights the responsibilities reduce to $r_k \propto \mathcal{N}_k$, likelihood-ratio assignment, and the model is an EM mixture with fixed equal weights. If `soft` cannot beat `uniform`, the encoder-gate pathway earned nothing on that panel — an ablation the thesis must be able to lose honestly (and one recent MoE-for-finance paper found surprisingly close to a tie, which is exactly why the arm is in the grid).

6.3 Hard routing done right: the hard-EM surrogate

The prototype’s sin was not *wanting* winner-take-all routing — at deployment, running one expert instead of K is a legitimate wish — but implementing the winner selection as an untrainable index operation. The correct construction keeps hard selection *in the objective*, where it remains differentiable in the quantities that matter.

Define the hard-EM (classification-EM, Viterbi-training) surrogate: replace the mixture’s LSE with a max,

$$\text{NLL}_{\text{hard}} = -\max_k a_k = -(\log \pi_{k^*} + \log \mathcal{N}_{k^*}), \quad k^* = \arg \max_k a_k, \quad (6.1)$$

i.e. score the sample under its single best component, prior included. Since $\max_k a_k \leq \text{LSE}_k(a_k) \leq \max_k a_k + \log K$ (Exercise 6.1),

$$\text{NLL} \leq \text{NLL}_{\text{hard}} \leq \text{NLL} + \log K :$$

hard-EM minimizes an upper bound on the true NLL, tight up to $\log K$ nats — the E-step’s soft posterior hardened to an indicator.

The gradient is where the design earns its keep. For fixed selection k^* (locally constant in z almost everywhere),

$$\frac{\partial \text{NLL}_{\text{hard}}}{\partial z_k} = -\frac{\partial \log \pi_{k^*}}{\partial z_k} = \pi_k - \delta_{k,k^*} = (\pi - e_{k^*})_k. \quad (6.2)$$

Compare with the prototype’s zero. The bare `argmax` routed hard and carried no objective term in π ; the surrogate scores the selected branch *including its prior probability* $\log \pi_{k^*}$, and that term is differentiable in every logit. The result is cross-entropy toward the gate’s own `argmax` — self-training: confident choices are reinforced, and the experts’ likelihoods feed back through which branch *wins*. (Switch Transformer’s trick — multiply the selected expert’s output by its gate probability — is the same maneuver in point-prediction clothing.)

Two honesty clauses, both visible in the package. First, self-training can lock in an early wrong choice (the rich-get-richer dynamic of Section 6.6); hard from a cold gate is expected to glue itself to an arbitrary expert, which is a finding about hard routing, not a bug. Second — the interface moral of Section 5.6 — the surrogate is for *training only*: `log_train_weights` carries the masked $\log \pi$ that makes (6.1) the loss, while the predictive `log_prior` is the honest one-hot (the model claims $y \sim \mathcal{N}(\mu_{k^*}, \sigma_{k^*}^2)$, full stop), and every held-out NLL is graded on that claim.

6.4 Top- k : the trap

Between soft ($k = K$) and hard ($k = 1$) sits sparse routing: keep the k largest logits, renormalize among them, zero the rest. In log domain: mask non-top- k logits to $-\infty$ and `log_softmax` the result; the $-\infty$ entries drop out of the mixture’s LSE naturally and receive exactly zero responsibility.

The trap is the $k = 1$ case, and it is worth a formal statement because it *looks* like a soft, differentiable operation while being exactly defect 1.

Theorem 6.1 (Renormalized top-1 is gradient-dead). *Let $S(z)$ be the index set of the k largest logits and $\tilde{\pi}_j = e^{z_j} / \sum_{l \in S(z)} e^{z_l}$ for $j \in S$, zero otherwise. If $k = 1$, then away from ties $\tilde{\pi} = e_{k^*}$ identically in a neighborhood of z , and hence $\partial \tilde{\pi} / \partial z = 0$ almost everywhere — no gradient reaches the gate through the mixture weights. If $k \geq 2$, the within- S Jacobian is that of a k -way softmax, which is nonzero wherever the kept logits are finite and distinct.*

Proof. For $k = 1$: on the open set where z_{k^*} strictly exceeds all other logits, S is locally constant, and the renormalized weight is $e^{z_{k^*}} / e^{z_{k^*}} = 1$ — a constant function of z . Its derivative is zero there; the tie set where S changes has measure zero and no derivative at all. For $k \geq 2$: with S locally constant, $\tilde{\pi}$ restricted to S is $\text{softmax}(z_S)$, whose Jacobian $\text{diag}(\tilde{\pi}) - \tilde{\pi} \tilde{\pi}^\top$ vanishes only at vertices of the simplex, unreachable for finite distinct logits. The mixture weights then depend non-trivially on the kept logits, and Theorem 5.1’s computation goes through within S . $\square$

The lesson generalizes the autopsy’s: defect 1 was easy to see (`argmax` announces itself); *renormalization after selection reconstructs the same death silently*, because normalizing a single survivor erases the very number that carried the gradient. The package does not document this hazard — it makes it unrepresentable: `TopKRegimePrior` validates $2 \leq k \leq K$ at construction and its error message redirects top-1 seekers to hard, whose surrogate supplies the gradient renormalization cannot. (The validation exists because a test asserting “top-1 trains the gate” *failed*; the theorem above was written after the test, which is the correct order of events in engineering.) At the other end, $k = K$ keeps every logit and reduces to soft exactly — a free consistency check the test suite also pins.

6.5 Gumbel-softmax: stochastic routing with a temperature

The mechanisms so far are deterministic. A stochastic gate — sample a regime, rather than blend — brings exploration (early training visits minority experts) at the price of gradient variance. The classical route to differentiable sampling is a two-step derivation.

Lemma 6.2 (Gumbel-max trick). *Let $g_1, \dots, g_K$ be i.i.d. standard Gumbel ($\mathbb{P}(g \leq t) = e^{-e^{-t}}$, generated as $-\log(-\log U)$ for uniform U). Then*

$$\mathbb{P}\left(\arg \max_k (z_k + g_k) = j\right) = \frac{e^{z_j}}{\sum_l e^{z_l}} = \text{softmax}(z)_j.$$

Proof. j wins iff $z_j + g_j > z_l + g_l$ for all $l \neq j$. Condition on $g_j = t$: the events $\{g_l < z_j - z_l + t\}$ are independent with probabilities $\exp(-e^{-(z_j - z_l + t)})$. Integrate over the Gumbel density $e^{-t}e^{-e^{-t}}$:

$$\mathbb{P}(j \text{ wins}) = \int_{-\infty}^{\infty} e^{-t} e^{-e^{-t}} \prod_{l \neq j} e^{-e^{-(z_j - z_l + t)}} dt = \int_{-\infty}^{\infty} e^{-t} \exp\left(-e^{-t} \underbrace{\sum_l e^{-(z_j - z_l)}}_{=c_j}\right) dt,$$

where the sum now includes $l = j$ (its term is 1). Substitute $u = e^{-t}$, $du = -e^{-t} dt$: the integral is $\int_0^{\infty} e^{-c_j u} du = 1/c_j = e^{z_j} / \sum_l e^{z_l}$. $\square$

Sampling a category is thus an argmax over perturbed logits — which is again an untrainable index. The relaxation (Jang–Gu–Poole and Maddison–Mnih–Teh, independently, 2016) replaces that argmax with a temperature softmax:

$$\pi^{(\tau)} = \text{softmax}\left(\frac{z + g}{\tau}\right), \quad (6.3)$$

a *random point on the simplex* that concentrates at a random vertex as $\tau \rightarrow 0$ (recovering exact categorical sampling by the lemma) and flattens toward uniform as $\tau \rightarrow \infty$ (Figure 6.1). Finite τ buys differentiability — gradients flow through (6.3) into z by the ordinary softmax Jacobian, with the noise acting as a reparameterization — at the price of bias (the relaxed weights are not one-hot) and of gradient variance that grows as τ falls (Exercise 6.5). The standard protocol, and the package’s, manages the trade with an *annealing schedule*: start warm (exploration, low variance), cool exponentially toward a floor,

$$\tau(s) = \tau_{\text{init}} \left(\frac{\tau_{\text{min}}}{\tau_{\text{init}}} \right)^{\min(s/S, 1)},$$

with s the training step. At evaluation the noise and temperature are dropped entirely — deterministic $\log_{\text{softmax}}(z)$ — because stochastic routing is a *training* device, and a desk quoting a prediction does not want it resampled per query.

Engineering consequence — schedules make state, state makes contracts

The temperature depends on the training step, so the step must reach the prior: the trainer threads `PriorContext(step=step_count)` into every forward pass. That one line quietly creates a contract with two distant subsystems. Determinism: two runs are comparable only if their schedules align step-for-step — seeds fix the Gumbel draws, the threaded

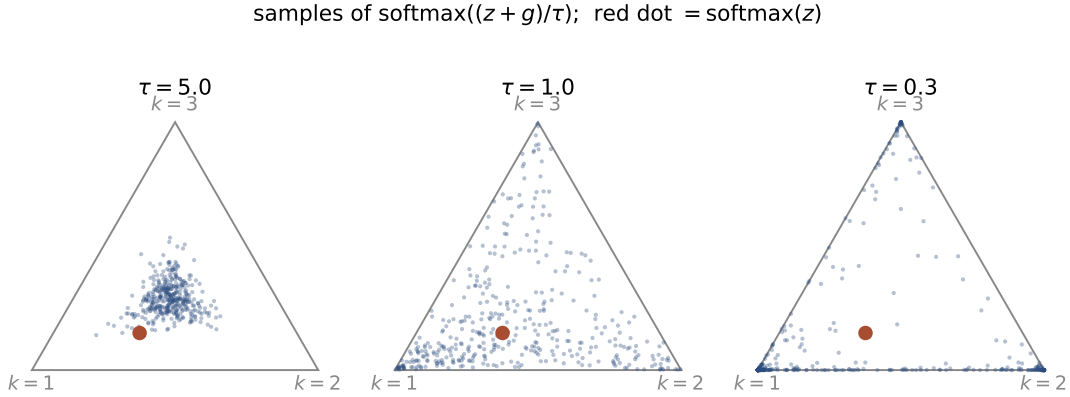


Figure 6.1: The temperature knob. 400 draws of $\text{softmax}((z + g)/\tau)$ on the 3-expert simplex, fixed logits (red dot = $\text{softmax}(z)$). Warm ($\tau = 5$): samples hug the mean — soft routing with jitter. Unit temperature: spread across the simplex. Cold ($\tau = 0.3$): mass concentrates at the vertices with frequencies approaching $\text{softmax}(z)$ — Lemma 6.2 becoming visible. Annealing walks the gate from the left panel to the right one over training.

step fixes τ . Checkpointing: a resumed run must restore `step_count` (and the RNG state feeding g) or the schedule silently restarts warm and the trajectories diverge — one of the exact invariants Chapter 10’s bit-identical-resume test pins. A scheduled quantity is never just a formula; it is state, and state must be owned, threaded, and persisted.

6.6 Load balancing, and where to measure it

Learned routing has a known instability: *rich-get-richer*. An expert that wins early gets more gradient, improves faster, and wins more; its siblings starve at $r_k \approx 0$ (equation (5.4): no responsibility, no learning). Sparse and hard variants, which zero out losers entirely, are the most exposed. The standard counterweight (Shazeer et al. 2017) is an auxiliary loss

$$\mathcal{L}_{\text{lb}} = K \sum_k f_k \bar{p}_k, \quad \bar{p}_k = \frac{1}{B} \sum_b \pi_{bk}, \quad (6.4)$$

with f_k the fraction of samples *assigned* to expert k (argmax counts, gradient-stopped) and $\bar{p}_k$ the mean gate probability (gradient-carrying). Its gradient pushes probability away from over-assigned experts; at perfect balance $f_k = 1/K$ the loss is identically $K \sum_k \bar{p}_k / K = 1$ *regardless of how sharp individual rows are* — balance is enforced marginally, sharpness is free.

That last property holds only if f is measured over the right pool, and this is the one place where the finance setting genuinely diverges from the language-model setting the loss was invented for.

What breaks without this — balancing inside a regime-pure batch

In our panels a batch is often date-sliced, and the regime is a *date-level* variable: a batch drawn from March 2020 is legitimately, informatively one-sided. Estimate f on that batch alone and (6.4) punishes the gate for being right — it demands within-date uniformity,

which is precisely the sharp regime call the whole architecture exists to make. Applied naively, the balancing loss converges the gate toward uniform and quietly deletes the thesis. (This is the central warning of arXiv:2501.11873, and our Chapter 2 persistence math says regime-pure stretches last *weeks*, so the hazard is the common case, not the corner case.) The package’s `LoadBalanceBuffer` therefore accumulates assignment counts over a rolling window of many batches — spanning many dates — so f estimates the *marginal* utilization across regimes; a single date remains free to route as one-sidedly as reality demands. Same formula, different estimator scope, opposite scientific outcome.

Both auxiliary losses in the package (the balance term and an expert output-decorrelation term aimed at the homogenization failure) default to *off*: the baseline objective is the pure likelihood, and each aux term is a pre-registered ablation lever, not silent seasoning. The buffer, however, always runs — feeding the utilization dashboard, because the difference between “the gate is specializing” and “an expert is dying” is a monitoring question long before it is a loss question.

Exercises

Exercise 6.1. Prove $\max_k a_k \leq \text{LSE}(a) \leq \max_k a_k + \log K$, and exhibit the two extreme cases. Conclude the hard-EM objective brackets the true NLL within $\log K$ nats, and compute what that slack is worth (in NLL units) for $K = 2$ versus $K = 8$ — one reason small- K mixtures tolerate hard training better.

Exercise 6.2. Derive (6.2) carefully, confirming the likelihood term contributes nothing to the gate gradient for fixed selection, and show the expected update direction reinforces the current argmax. Then describe the fixed points of pure self-training with frozen experts: which gates are stationary, and why a cold-start gate gluing to an arbitrary expert is the *generic* outcome rather than bad luck.

Exercise 6.3. (Hands-on.) Reproduce Theorem 6.1 in six lines of torch: build z with `requires_grad`, mask to top- k , renormalize with `log_softmax`, feed a fused mixture NLL, and print `z.grad` for $k = 1$ and $k = 2$ ($K = 3$). Then explain in one paragraph why the $k = 1$ case cannot be rescued by any smooth renormalization of a single kept entry — the information destroyed by “keep one, normalize” is not recoverable downstream.

Exercise 6.4. Fill in every step of Lemma 6.2’s proof: verify the Gumbel density, the conditioning step, the inclusion of the $l = j$ term, and the substitution. Then show that $z + g$ shifted by any constant leaves the argmax distribution invariant, and connect this to Exercise 5.2’s observation about softmax logits.

Exercise 6.5. For $K = 2$, write $\pi_1^{(\tau)} = \text{sigmoid}((\Delta z + \Delta g)/\tau)$ with Δg logistic. (a) Show $\mathbb{E}[\pi^{(\tau)}] \rightarrow \text{softmax}(z)_1$ as $\tau \rightarrow 0$ and $\rightarrow 1/2$ as $\tau \rightarrow \infty$. (b) Simulate the variance of $\partial \pi_1^{(\tau)} / \partial \Delta z$ at $\tau \in \{5, 1, 0.3, 0.1\}$ and describe the trade the annealing schedule is walking. (c) Why does evaluating with noise on (rather than the package’s deterministic eval mode) make two models’ held-out NLLs incomparable at different τ ?

Exercise 6.6. (a) Verify that (6.4) equals 1 identically under balanced f , for *any* gate matrix — sharpness is unpenalized. (b) Simulate rich-get-richer: two experts, one initialized slightly better; train the mixture NLL without balancing and plot utilization; repeat with (6.4) under (i) per-batch f on regime-pure batches and (ii) buffered f spanning both regimes. Confirm (i) destroys sharp routing while (ii) rescues the starving expert without flattening within-date decisions — the package’s scope choice, reproduced from scratch.

Chapter 7

Hidden Markov Dynamics

Chapter 2 proved that regime persistence is where volatility clustering lives, yet every mechanism in Chapter 6 treats each date as a fresh draw. Variation 3 closes the gap: replace the memoryless prior with a Markov chain over regimes, and the gate becomes the predict step of a forward filter. This chapter derives that filter from Bayes' rule, proves that its per-step normalizers are the likelihood, draws the hard line between filtering (admissible) and smoothing (look-ahead), and works through what it means to train by backpropagating through the recursion. The punchline is architectural: the entire upgrade changes one pluggable box, because the mathematics factors exactly where the code does.

7.1 The model: one chain, many names

Let $S_t \in \{1, \dots, K\}$ be the market regime at date t — one latent state per date, shared by the whole cross-section, in contrast to the memoryless model's per-sample assignments. The regime follows a first-order Markov chain,

$$\mathbb{P}(S_t = k \mid S_{t-1} = j) = A_{jk}, \quad \mathbb{P}(S_1 = k) = \pi_0(k),$$

and, given the regime, the date's cross-section is generated by that regime's expert exactly as before:

$$p(\mathbf{y}_t \mid S_t = k, X_t) = \prod_{i \in \mathcal{U}_t} \mathcal{N}(y_{i,t}; \mu_k(\mathbf{x}_{i,t}), \sigma_k^2) =: \ell_t(k), \quad (7.1)$$

the *per-date emission likelihood*: one number per regime per date, aggregating the whole cross-section (in log domain, a sum of the familiar per-name Gaussian log-densities over the date's names). The pair (Markov prior, arbitrary emission) is a hidden Markov model; with neural $\mu_k(\cdot)$ it is the thesis's Variation 3, and with constant μ_k it is exactly Hamilton's model from Chapter 2's Origins box.

7.2 The forward filter, derived

Everything an online system may know at date t is $\mathbf{y}_{1:t}$. Write the *filtered posterior* $r_t(k) = \mathbb{P}(S_t = k \mid \mathbf{y}_{1:t})$. It obeys a two-step recursion.

Proposition 7.1 (Forward filter). With $\pi_t(k) := \mathbb{P}(S_t = k \mid \mathbf{y}_{1:t-1})$ (the predictive prior),

$$\text{predict: } \pi_t(k) = \sum_j A_{jk} r_{t-1}(j), \quad (7.2)$$

$$\text{update: } r_t(k) = \frac{\pi_t(k) \ell_t(k)}{\sum_l \pi_t(l) \ell_t(l)}, \quad (7.3)$$

initialized by $\pi_1 = \pi_0$. The cost is $O(K^2)$ per date, $O(TK^2)$ per sequence.

Proof. Predict: condition on yesterday's state and use the Markov property ($S_t \perp \mathbf{y}_{1:t-1} \mid S_{t-1}$):

$$\mathbb{P}(S_t = k \mid \mathbf{y}_{1:t-1}) = \sum_j \mathbb{P}(S_t = k \mid S_{t-1} = j) \mathbb{P}(S_{t-1} = j \mid \mathbf{y}_{1:t-1}) = \sum_j A_{jk} r_{t-1}(j).$$

Update: Bayes' rule at date t , with $\mathbf{y}_t \perp \mathbf{y}_{1:t-1} \mid S_t$ (emissions depend only on the current state):

$$r_t(k) = \frac{p(\mathbf{y}_t \mid S_t = k) \mathbb{P}(S_t = k \mid \mathbf{y}_{1:t-1})}{p(\mathbf{y}_t \mid \mathbf{y}_{1:t-1})} = \frac{\ell_t(k) \pi_t(k)}{\sum_l \ell_t(l) \pi_t(l)}. \quad \square$$

Now place (7.3) next to Chapter 5's responsibilities: *it is the same formula*. A prior over components, times per-component likelihoods, normalized. The only thing that changed is where the prior comes from — yesterday's posterior pushed through the transition matrix, instead of a softmax of gate logits. This is the observation the package's whole gate abstraction rests on:

Engineering consequence — the factorization is the architecture

The fused loss (`mixture_nll`) computes “prior + likelihood $\rightarrow$ evidence + posterior” without caring where the prior came from. So the HMM upgrade touches exactly one component: a `RegimePrior` whose forward implements the predict step (7.2) in log domain,

$$\log \pi_t(k) = \text{LSE}_j (\log r_{t-1}(j) + \log A_{jk}),$$

consuming the previous filtered posterior through `PriorContext.prev_filtered` and marked `stateful = True` so the trainer threads state chronologically. The expert bank, the likelihood core, the loss, the diagnostics: byte-identical between Variations 2 and 3. When mathematics factors cleanly, the highest form of respect an implementation can pay is to factor in the same place — and the reward is scientific: any performance difference between the variations is attributable to the prior mechanism, because nothing else differs.

7.3 The likelihood is the filter's by-product

Training needs $\log p(\mathbf{y}_{1:T})$. It seems to require summing over K^T state paths; the filter has already done so.

Proposition 7.2 (Prediction-error decomposition).

$$\log p(\mathbf{y}_{1:T}) = \sum_{t=1}^T \log p(\mathbf{y}_t \mid \mathbf{y}_{1:t-1}) = \sum_{t=1}^T \log \sum_k \pi_t(k) \ell_t(k) : \quad (7.4)$$

the sequence log-likelihood is the sum of the per-date log-normalizers of the update step (7.3).

Proof. The first equality is the chain rule of probability. The second is the law of total probability at date t conditioned on S_t , with $\mathbb{P}(S_t = k \mid \mathbf{y}_{1:t-1}) = \pi_t(k)$ by definition. $\square$

So the per-date training loss is $-\text{LSE}_k(\log \pi_t(k) + \log \ell_t(k))$ — *the same fused mixture NLL*, now with the filter’s predictive prior in the prior slot. One objective expression serves both variations; the sum over exponentially many paths has become T mixture losses chained by the recursion. Each term has a forecasting reading worth internalizing: it is the log-score of date t ’s cross-section under what the model believed *before seeing date t* — a strictly causal, one-step-ahead grade, which is why maximizing (7.4) cannot reward hindsight.

7.4 Filtering versus smoothing: the causal line

The filter is not the only inference the model supports. Conditioning on the *whole* sequence gives the smoothed posterior $\mathbb{P}(S_t = k \mid \mathbf{y}_{1:T})$, computed by supplementing the forward pass with a backward recursion (Exercise 7.1 derives it). Smoothing is strictly better inference about S_t — it uses more data — and it is precisely what this project must refuse to use.

The reason is Chapter 1’s leakage ratchet, now in state space. The smoothed belief about date t depends on returns from dates $t + 1, \dots, T$. Any prediction, position, or regime call at t that consumes a smoothed quantity has read the future — not through a mislabeled column this time, but through a posterior. The failure is seductive precisely because smoothed regime paths look *beautiful*: crisp, confident, aligned with hindsight. (Every regime-model figure in a paper that shades recessions with eerie precision is showing you a smoother.) Filtered paths, by contrast, are visibly humbler — Figure 7.1 shows the honest product: the belief lags each regime flip by the few days genuinely needed to distinguish signal from one loud day, hesitates on short excursions, and that lag *is the price of causality*, not an implementation defect.

What breaks without this — a smoothed backtest

Wire the smoothed posterior into a backtest — regime-conditional positions sized by $\mathbb{P}(S_t \mid \mathbf{y}_{1:T})$ — and regime turns are called with hindsight’s timing: the system de-risks *at* the peak and re-risks *at* the trough, manufacturing a Sharpe ratio that no live deployment can reproduce. Nothing crashes; the results are simply fiction, and (ratchet clause) fiction in the flattering direction. The package’s defenses are structural: no smoother is implemented at all — `evaluate_sequence` runs (7.2)–(7.3) forward only — and the causality test perturbs future dates’ data and asserts bit-identical filtered posteriors and predictions at every earlier date. The honest quantity is also the useful one: the *predictive* prior π_t , available before date t ’s returns, is what a desk would actually position on.

One clarification keeps the rule precise rather than superstitious: smoothing over the *training window* during *parameter estimation* is legitimate — classical Baum–Welch does exactly that, and every date it smooths over lies in the past by construction. The prohibition is on smoothed quantities crossing into prediction, evaluation, or anything time-indexed that a deployed system would consume. The package nonetheless trains on the filtered objective too (next section), for reasons of its own.

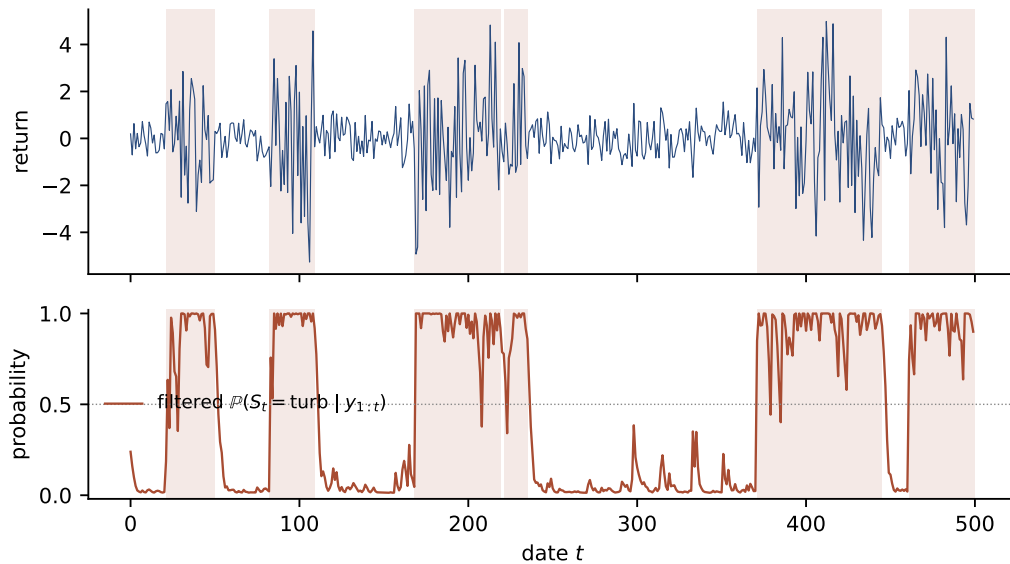


Figure 7.1: Filtering, the honest inference. Top: returns from a two-state Markov scale mixture (turbulent state shaded). Bottom: the forward filter’s $\mathbb{P}(S_t = \text{turb} \mid \mathbf{y}_{1:t})$ computed strictly causally with the true parameters. The belief tracks every sustained regime but reacts with a short delay and wavers on brief excursions — exactly what “known at time t ” looks like. A smoother would clean all of this up, using data no deployable system has.

7.5 Training through the filter

Two estimation routes exist for HMM parameters.

Route 1: Baum–Welch — EM instantiated for chains. E-step: forward–backward smoothing yields state marginals and transition pair-marginals over the training window. M-step: closed-form reestimates of A (expected transition counts) and, for *Gaussian* emissions, of (μ_k, σ_k) by smoothed-weighted moments. Elegant, monotone (Theorem 4.3 applies), and the classical choice — but the M-step’s closed forms die the moment $\mu_k(\cdot)$ is a network, which would leave EM’s outer loop wrapped around an inner SGD anyway.

Route 2: direct gradient on the filtered NLL — Decision C. Treat (7.4) as a loss and let autograd differentiate through the recursion itself: the predict step is a LSE over yesterday’s *attached* log-posterior, so the chain rule flows from date t ’s loss back through $r_{t-1}, r_{t-2}, \dots$ into every parameter — the transition logits accumulate credit for how *beliefs they shaped days ago* paid off today. (In the classical literature this is the “recursive score” computation; automatic differentiation delivers it for free.) The package takes Route 2 for every emission type: one optimizer, one code path shared with Variation 2, neural emissions welcome, and the trained objective is the same strictly-causal score the evaluation grades — an alignment Baum–Welch does not offer.

The practical cost is the usual cost of backpropagating through time: memory and gradient conditioning over long sequences. The trainer therefore chunks: within a chunk of C dates the state is threaded attached (full gradient flow); at chunk boundaries the carried posterior is `detach()`ed — truncated BPTT over the regime posterior, with the truncation length an explicit config knob rather than a hidden constant. Two engineering clauses follow from the

mathematics. The recursion’s correctness depends on *chronological, complete per-date batches* — shuffled minibatches would feed the filter scrambled time — so the stateful path validates its input ordering and the memoryless `train_step` refuses stateful priors outright. And the carried state is part of training state: Chapter 10’s checkpointing persists the chunk cursor and carried posterior precisely so a resumed run continues the recursion bit-exactly.

7.6 Parameterizing the transition matrix

A must be a stochastic matrix (rows on the simplex). Constrained optimization is the wrong tool at this scale; the package uses the standard unconstrained parameterization

$$A = \text{row_softmax}(L), \quad L \in \mathbb{R}^{K \times K} \text{ free,}$$

so any gradient step on L yields a valid A . Two choices ride on top, both traceable to Part I mathematics:

Persistence-biased initialization. L starts as $\beta_{\text{diag}} I$ (default bias 2), making A ’s diagonal dominant from step one: with $K = 2$, $A_{kk} = \sigma(\beta_{\text{diag}}) \approx 0.88$ initial stay probability. Chapter 2 estimated real regime persistence at $\lambda \approx 0.9\text{--}0.98$; starting the chain sticky encodes that prior belief and — more practically — keeps early training out of the fast-mixing region where the filter’s beliefs slosh datewise and the emission gradients are noise. It is the transition-matrix analogue of initializing an LSTM’s forget-gate bias positive, and like all initializations it is a starting point, not a constraint: the data can unlearn it.

No weight decay on L . Generic weight decay shrinks parameters toward zero; $L \rightarrow 0$ means $A \rightarrow \text{uniform}$ — decay would actively pull the chain toward forgetting its own persistence, silently fighting the initialization and the data. The trainer’s parameter groups therefore exempt prior parameters (and $\log \sigma$, for the analogous reason) from decay. A regularizer is a prior; applying one that contradicts the model’s actual prior belief is not hygiene, it is sabotage — one more instance of the book’s running rule that defaults are decisions.

7.7 Label permutation across refits

The HMM likelihood, like every mixture’s, is invariant under relabeling the states (permute π_0 , rows and columns of A , and the emission parameters together). Within one training run this is harmless — gradient descent stays in one labeling basin. Across *independent refits* — one per walk-forward window (Decision D), one per seed — it is a real hazard: window 3’s “regime 1” may be window 4’s “regime 2,” and any cross-window aggregation of per-regime quantities (mean utilization of the turbulent expert, say) silently averages apples with oranges. The repair is declared, not discovered: the package’s canonical order sorts experts by σ_k ascending after every fit, records the permutation per fold, and all cross-fit reporting uses canonical indices. Chapter 8 folds this into the broader identifiability picture.

Code walkthrough — `nec_moe/priors.py` --- the predict step

The entire Variation-3 mechanism is one forward:

```
def forward(self, gate_logits, ctx=None):
    log_a = self.log_transition()          # (K, K) row log-softmax of L
    if ctx is None or ctx.prev_filtered is None:
```

```

log_prior = F.log_softmax(self.pi0_logits, 0).expand(b, -1)
else:
    log_prior = torch.logsumexp(
        ctx.prev_filtered.unsqueeze(2) + log_a.unsqueeze(0), dim=1)
return PriorOutput(log_prior=log_prior, ...)

```

Read it against (7.2): the broadcast adds yesterday’s log-posterior (column) to log A (matrix) and LSEs over the source state — matrix–vector multiplication in log space, numerically safe for beliefs as confident as the data makes them. Sequence starts fall back to π_0 . Note what is *absent*: no gate-logit dependence (the encoder is dormant in the static-transition variant — its return would be the guarded-off time-varying-transition extension), and no backward pass anywhere in the file. The trainer’s side is `train_step_sequence`: thread state attached within a chunk, `state.detach()` at the boundary, `loss = mean per-date fused NLL` — Proposition 7.2 rendered as a for loop.

Exercises

Exercise 7.1. Derive the backward recursion: define $\beta_t(k) = p(\mathbf{y}_{t+1:T} \mid S_t = k)$, show $\beta_t(j) = \sum_k A_{jk} \ell_{t+1}(k) \beta_{t+1}(k)$ with $\beta_T \equiv 1$, and conclude the smoothed marginal is $\mathbb{P}(S_t = k \mid \mathbf{y}_{1:T}) \propto r_t(k) \beta_t(k) / \pi$ (state the exact normalization). Verify that computing it requires the full future — the formal content of “smoothing is look-ahead.”

Exercise 7.2. Construct the smallest possible counterexample to “filtered = smoothed”: $T = 2$, $K = 2$, symmetric sticky A , and emissions where date 2 is loud. Compute r_1 and $\mathbb{P}(S_1 \mid \mathbf{y}_{1:2})$ by hand and show date 2’s observation moves the belief about date 1. Then say in one sentence what a backtest that positions at date 1 using the latter quantity has done.

Exercise 7.3. Differentiate one step of the filtered objective with respect to the transition logits: for the per-date loss $-\log \sum_k \pi_t(k) \ell_t(k)$ with π_t from (7.2) and $A = \text{row_softmax}(L)$, show the gradient with respect to L_{jk} is $-r_{t-1}(j)(r_t(k) - A_{jk} [\text{norm. terms}])$ -shaped — derive the exact expression — and interpret: transitions are reinforced in proportion to yesterday’s belief in the source state times today’s posterior surprise in the destination. Where does Theorem 5.1’s $\pi - r$ pattern reappear?

Exercise 7.4. Show that with `ClassicalGaussianEmission` (constant μ_k, σ_k), the filtered objective (7.4) is exactly the likelihood Hamilton (1989) maximized, and that the package’s parameter-recovery test — generate from a known sticky two-state scale mixture, fit `classical × hmm`, compare $\hat{A}$ and $\hat{\sigma}$ — is a Hamilton replication on synthetic data. Why is rank-IC structurally meaningless for this emission (Chapter 1’s grading vocabulary), and what is graded instead?

Exercise 7.5. (Hands-on.) Implement the log-domain forward filter in ≤ 20 lines of numpy. Generate a sticky two-state panel with the package’s synthetic generator, run your filter with the *true* parameters, and compare against `Trainer.evaluate_sequence`’s `log_filtered` on the same data (after loading the true parameters into a `classical × hmm` model): they should agree to numerical precision. Then break causality on purpose — flip the sign of the last date’s returns — and verify your filtered path before that date is unchanged, bit for bit. You have just reimplemented the package’s causality test.

Exercise 7.6. Truncated BPTT with chunk length C zeroes gradient flow across boundaries.

(a) Argue that the *filter values* are unaffected — truncation changes gradients, never the forward computation. (b) Identify which parameter’s learning is most damaged by small C (the answer moves through multi-date credit assignment), and design a synthetic experiment that would expose it: what would you plant, and what would you measure as a function of C ? (c) Explain why mean-per-date (not sum) chunk losses keep gradient magnitudes comparable across unequal final chunks.

Chapter 8

Local Optima and Symmetry

Every guarantee so far has been local: EM ascends, gradients are alive, the filter is exact. None of it says the optimizer arrives anywhere worth being. This chapter studies the mixture likelihood’s geometry — the permutation symmetry that makes “which expert is which” undefined, and the symmetric stationary point at which a mixture quietly collapses into the pooled model it was built to beat. The centerpiece is a negative result with teeth: on the very task the thesis cares about, random restarts provably do not help, and the escape requires an asymmetry you must inject on purpose. The chapter closes with the package’s device for doing so legally — and with what remains irreducibly multimodal after all of it, which Part V’s multi-seed protocol treats as variance to be reported, not hidden.

8.1 The landscape has $K!$ of everything

Proposition 8.1 (Permutation symmetry). *Let ρ be any permutation of $\{1, \dots, K\}$ and let $\rho\theta$ denote a mixture’s parameters with components relabeled by ρ (weights or gate outputs, means, scales — and for the HMM, π_0 and both indices of A — permuted together). Then the likelihood is invariant: $\ell(\rho\theta) = \ell(\theta)$ for all θ . Consequently every stationary point, local optimum, and gradient trajectory has $K!$ images, and “expert k ” is not an identified statistical quantity.*

Proof. The mixture density (4.1) (and the HMM likelihood via (7.4)) is a sum over components; relabeling permutes the summands. Gradient flow commutes with the relabeling because the objective does. $\square$

Two practical corollaries. First, the loss surface is never single-basined for $K \geq 2$: whatever optimum exists, its mirror images exist at the same height, with saddles or ridges between them. Second — the operational one — any quantity indexed by raw expert number is meaningless across independent fits. Chapter 7 met this as label permutation across walk-forward windows; the repair is worth restating as a rule: *identification is declared, not discovered*. The package sorts experts by σ_k ascending after every fit (“expert 0” means the calm expert, by definition), records the permutation per fold, and aggregates only canonical indices. The declaration costs one `argsort`; its absence costs silent apples-to-oranges averages in every cross-fit table.

8.2 The symmetric stationary point

Permutation twins are benign — all $K!$ optima are equally good. The dangerous geometry lives *between* them. Work in the cleanest setting that exhibits it, which is also the thesis’s own planted world (Proposition 2.3): the sign-flip task, $y = S\beta x + \varepsilon$ with $S = \pm 1$ equiprobable, fit by a two-expert mixture with linear experts $\mu_i(x) = b_i x$, a gate, and fixed equal scales.

Proposition 8.2 (The pooled point is stationary). *Consider the population NLL of the two-expert model on the sign-flip task. At the pooled point — both experts at the pooled regression solution ($b_1 = b_2 = 0$, since the pooled slope is zero) and the gate at any state, e.g. uniform — every gradient in the system vanishes: gate, experts, and (if free) scales.*

Proof. With $b_1 = b_2 = 0$ the two experts define *identical* densities for every sample, so the responsibilities are uninformative: $r_1 = \pi_1$, $r_2 = \pi_2$ pointwise (Bayes with equal likelihoods returns the prior). The gate gradient $\pi - r$ (Theorem 5.1) is identically zero. The expert- i gradient (5.4) becomes $\mathbb{E}[\pi_i(0 - y)x]/\sigma^2 = -\pi_i\mathbb{E}[yx]/\sigma^2 = 0$, since the pooled cross-moment $\mathbb{E}[yx] = \beta\mathbb{E}[S]\mathbb{E}[x^2] = 0$ is exactly what Proposition 2.3 established. The scale gradient (5.5) is a responsibility-weighted moment condition satisfied by the pooled residual variance. Nothing moves. $\square$

The pooled point is not a pathology of bad luck; it is the mixture disguised as the single model it should dominate — same predictions, same NLL, $r \equiv \pi$, gate silent. Two features make it a trap rather than a curiosity.

Proposition 8.3 (The symmetric subspace is invariant). *Under gradient flow (or EM, or any update that is a function of the current parameters through the objective), the subspace $\mathcal{S} = \{b_1 = b_2\}$ is invariant: a trajectory that starts with identical experts keeps them identical forever. Within $\mathcal{S}$, the dynamics reduce to fitting one pooled regressor, whose optimum on the sign-flip task is the pooled point.*

Proof. If $b_1 = b_2$ then the experts’ densities coincide sample-wise, so $r_1 = \pi_1$, $r_2 = \pi_2$ as above, and the two expert gradients (5.4) are *equal*: $\partial\text{NLL}/\partial b_1 = \pi_1 g$, $\partial\text{NLL}/\partial b_2 = \pi_2 g$ with the same $g = -\mathbb{E}[(y - bx)x]/\sigma^2 \dots$ equal up to the scalar weights, hence — for the symmetric gate states reachable from a symmetric start, $\pi_1 = \pi_2 = \frac{1}{2}$ — identical updates. Equal parameters plus equal updates stay equal; on $\mathcal{S}$ the mixture NLL restricted to the common slope b is a single-Gaussian regression loss, minimized at the pooled slope. $\square$

What breaks without this — the collapse you cannot see in the loss

This failure produces no red flags in the quantities beginners watch. The loss *decreases* (to the pooled model’s respectable-looking NLL); training is “stable”; predictions are sensible. What is quietly false is everything the thesis is about: the experts are clones, the gate has learned nothing, the mixture’s IC on the sign-flip world is ≈ 0 where the specialized solution scores ≈ 0.9 . The tells are in the mixture diagnostics, which is why the package’s dashboard tracks them from step zero: expert output correlation $\rightarrow 1$, both $\hat{\sigma}_k$ equal (each inflated to the pooled residual scale — Chapter 2’s “variance of the slope” reappearing as phantom noise), responsibilities carrying no information about the true regime (AUC ≈ 0.5). During the build this happened exactly as the theory predicts, across *every* seed tried — which is the next proposition’s point.

8.3 Why seed diversity fails

The standard reflex — restart from a few random seeds, keep the best — rests on an assumption: that random initializations land in different basins, some of them good. Here the assumption breaks structurally. A fresh initialization puts the two expert networks at independent random weights, which in *function* space places both near the same small random function: the experts start ε -close to the invariant subspace, with ε the scale of initialization noise. The dynamics near $\mathcal{S}$ then factor into a fast component *along* the subspace (both experts racing to the pooled fit, driven by the full pooled-regression gradient) and a slow component *transverse* to it, whose growth must be bootstrapped by the responsibility contrast — which is itself proportional to the experts’ separation. Small separation $\Rightarrow$ near-uniform $r \Rightarrow$ near-equal expert gradients $\Rightarrow$ separation grows only at a rate proportional to itself: exponential escape from an exponentially small seed, which prices each decade of initial symmetry at a constant *additive* number of plateau steps. In the computed toy of Figure 8.1 the tax is already $4\times$ the warm start’s entire budget at a transverse asymmetry of 10^{-5} ; in the full model — deep experts whose function-space asymmetry is smaller still, stochastic gradients whose noise the contrast must out-shout, and σ calibrated to the pooled scale precisely when it matters — the tax compounds past any budget actually run: in the package’s experiments on the symmetric task, no plain seed ever specialized. Figure 8.1 shows both fates at an equal step budget: the near-symmetric start funnels into the saddle and sits; the asymmetric start rides straight into a specialized optimum.

The statement deserves its honesty clause: this is an argument with a theorem at its core (the invariance) and dynamics reasoning around it, not a full proof of basin measure — and it does not need to be more. In the package’s own experiments on the symmetric task, every plain seed collapsed and every warm-started run specialized; the design document records the observation, the test suite pins the warm-started outcome, and the mechanism above explains rather than merely reports it.

8.4 Breaking symmetry on purpose

If the trap is symmetry, the cure is a deliberate, *legal* asymmetry. “Legal” carries two constraints from earlier chapters: it must use only training-window information (Chapter 1’s point-in-time discipline), and it must not supervise the gate (Decision B — else the thesis question “do learned regimes align with real ones?” is answered by construction). The package’s device threads both needles:

Sort the training window by an observable volatility proxy; partition into K quantile slices; pre-fit expert k on slice k alone; leave the gate untouched.

Each ingredient earns its place. The *proxy* (trailing window volatility on synthetic panels; realized vol or VIX terciles on real ones) is chosen because Chapter 2’s entire argument says volatility is the observable shadow of the latent regime — the sort key is correlated with the truth without being the truth. The *slices* hand each expert a regime-enriched diet, so the pre-fit experts end up on *opposite sides* of the invariant subspace with macroscopic separation — the gold start of Figure 8.1. The *gate’s exclusion* preserves the experiment: routing must still be discovered by $\pi - r$ from the likelihood contrast the now-distinct experts create. And the

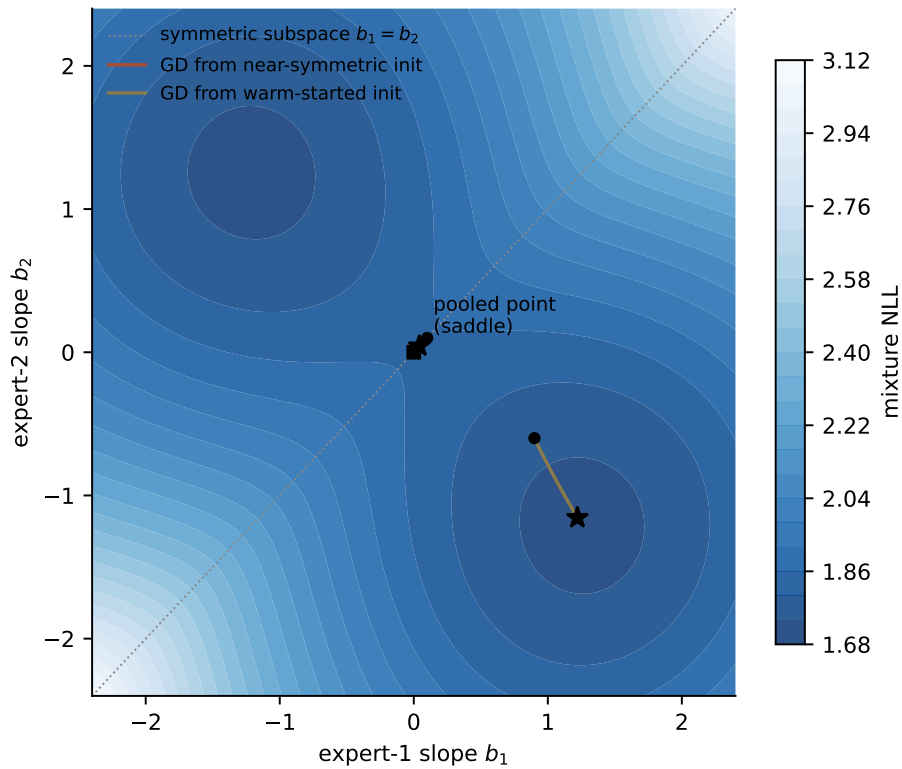


Figure 8.1: The symmetric trap, computed. Mixture NLL over the two expert slopes (b_1, b_2) for sign-flip data with a uniform gate and the model scale set to $\text{std}(y)$ — the package’s `sigma_init` regime, where early-training responsibilities are gentle; darker is lower. Twin specialized optima sit near $(\pm 1.25, \mp 1.2)$ (pulled inside $\pm\beta$ by the soft responsibilities at this σ); the pooled point $(0, 0)$ is a saddle on the invariant diagonal (dotted). Both trajectories get the same 60-step budget. The warm-started initialization (gold) converges in ~ 50 steps; the near-symmetric initialization (red, transverse asymmetry 10^{-5}) is still pinned at the saddle, and needs ~ 200 steps to escape — with each further decade of initial symmetry costing ~ 25 more steps, the measured form of Exercise 8.3’s exponential-escape law. Seeds move the red start by invisible amounts; the warm start moves it across the valley.

mechanics respect the emission abstraction — neural experts pre-train briefly as plain MSE regressors on their slice; classical Gaussian emissions initialize by their slice’s moments in closed form — so the device is one polymorphic method, not a special case.

After warm-starting, the previously indistinguishable outcomes separate cleanly in every diagnostic: on the package’s symmetric synthetic task, responsibility-regime AUC moves from ≈ 0.5 (every plain seed) to ≈ 0.99 , the $\hat{\sigma}_k$ split, and the gate — never shown a label — learns the routing. That last clause is the payoff worth savoring: symmetry breaking gave the gate something to distinguish, and Bayes’ gradient did the rest.

Engineering consequence — initialization is epistemology here

Note what kind of object the warm start is: not a model change (the architecture is untouched), not a loss change (the objective is the same NLL), but a *choice of starting point* — and yet it is the difference between the thesis model existing and not existing. The

general lesson for non-convex research systems: where the landscape has structural symmetries, initialization is not a numerical detail but part of the scientific design, deserving the same documentation, testing, and pre-registration as the loss. Concretely in the package: `warmstart_experts` is a first-class, tested trainer method; the walk-forward harness re-applies it per window from each window’s own training data (no leakage across folds); and the checkpoint/resume layer stores post-warm-start state so a resumed run never re-runs it. Its *absence* is equally first-class: the plain-seed collapse is kept as a documented negative result, because “we tried the obvious thing and here is precisely why it fails” is thesis material, not an embarrassment.

8.5 What remains multimodal

The warm start removes one structural trap. It does not make the problem convex, and the book’s honesty rule requires saying exactly what is left. Different seeds still reach different interior optima — different encoder features, different expert nonlinearities, different terminal NLLs — on real data where no planted truth marks one basin as “correct.” The response is not further initialization surgery but a change of *reporting* category: seed-to-seed dispersion is treated as genuine model variance, measured by running every configuration across a seed grid and reported as mean $\pm$ std in every table (nothing is a result until it has a seed std — Part V’s first commandment). Local optima thus exit the optimization chapter as a bug and re-enter the statistics chapters as a variance component — which is, in the end, the correct ontology for them.

Code walkthrough — `nec_moe/train.py + experts.py` --- the warm start

Trainer side — build the sort key, slice, delegate:

```
order = torch.argsort(sort_key)           # vol proxy, train window only
slices = torch.chunk(order, n_experts)    # K quantile slices
self.model.experts.warmstart_slices(x_exp, batch.y, slices, ...)
```

Emission side, neural case — expert k becomes a brief MSE regressor on its own slice:

```
for _ in range(steps):
    for k, idx in enumerate(slices):
        mu, _ = self(x[idx])
        loss = torch.mean((mu[:, k] - y[idx]) ** 2) # expert k, slice k
        ...
```

The classical emission overrides the same method with two lines of closed-form moments per slice. What is *not* in either excerpt is the point: no gate parameter appears anywhere — Decision B is enforced by the code’s shape, and the symmetry is broken where the symmetry lives, in the emissions.

Exercises

Exercise 8.1. Verify Proposition 8.2 concretely in torch: build the two-expert linear mixture on simulated sign-flip data, set $b_1 = b_2 = 0$ with a uniform gate, and confirm every parameter’s gradient is numerically zero after `backward()` on the batch NLL (up to sampling noise in the finite-sample $\mathbb{E}[yx]$ — quantify that noise and pick n to make the point cleanly).

Exercise 8.2. Extend Proposition 8.3 beyond the uniform gate: show that if $b_1 = b_2$ and the gate is *any* function of $\mathbf{h}$, the two experts still receive proportional gradients, and characterize the full invariant set of the joint (gate, experts) dynamics. What does the gate converge to within it, and why does that make the collapsed state even quieter diagnostically?

Exercise 8.3. Simulate escape times: initialize $(b_1, b_2) = (\varepsilon_0 + \delta, \varepsilon_0 - \delta)$ for a grid of transverse asymmetries $\delta \in \{10^{-3}, 10^{-2}, 10^{-1}, 0.5\}$, run gradient descent on the sign-flip mixture NLL, and record steps-to-specialization (say, $|b_1 - b_2| > \beta$). Plot log escape time against $\log \delta$ and reconcile the slope with Section 8.3’s “growth rate proportional to separation” argument.

Exercise 8.4. Derive the classical emission’s warm start in closed form: expert k ’s (μ_k, σ_k) from slice k ’s sample moments. Then show that for the pure scale-mixture world (means zero, $\sigma_1 \neq \sigma_2$), a vol-sorted slicing yields $\mathbb{E}[\hat{\sigma}_1] < \mathbb{E}[\hat{\sigma}_2]$ strictly — the warm start lands the classical model on the correct side of the symmetric subspace with probability approaching one as the window grows. What property of the sort key is doing all the work?

Exercise 8.5. Count the symmetry group precisely for the full thesis model: gate head, K experts, scales, and (Variation 3) π_0 and A . Beyond the $K!$ relabelings, does the architecture harbor additional exact symmetries (hidden-unit permutations inside each expert MLP, sign flips)? Which of them threaten cross-fit reporting the way label permutation does, and why does the σ -sort canonicalization resolve exactly the dangerous ones?

Exercise 8.6. Design (then implement against the package) a *collapse detector*: a boolean diagnostic that fires when a trained mixture is functionally pooled. Candidate ingredients: expert output correlation, σ spread, responsibility entropy, gate-gradient norm. Which single statistic is hardest to fool, what threshold would you pre-register, and on which of the package’s synthetic worlds (sign-flip, scale-only, regime-free) must your detector *not* fire despite low expert disagreement?

Appendix A

Solutions to Exercises

Chapter 1

Solution to Exercise 1.1. *Moments.* $\Pi(i)$ is uniform on $\{1, \dots, N\}$, so $\mathbb{E}[b_{\Pi(i)}^2] = \frac{1}{N} \sum_j b_j^2 = \frac{1}{N}$ by normalization. For $i \neq i'$, the pair $(\Pi(i), \Pi(i'))$ is uniform on ordered pairs $j \neq j'$, so

$$\mathbb{E}[b_{\Pi(i)} b_{\Pi(i')}] = \frac{1}{N(N-1)} \sum_{j \neq j'} b_j b_{j'} = \frac{(\sum_j b_j)^2 - \sum_j b_j^2}{N(N-1)} = \frac{0 - 1}{N(N-1)}.$$

Asymptotic normality. $\text{IC} = \sum_i a_i b_{\Pi(i)}$ is a simple linear rank statistic. Hoeffding's combinatorial CLT gives normality provided no single term dominates (the Noether condition $\max_i a_i^2 / \sum_i a_i^2 \rightarrow 0$). For rank scores, $a_i \propto i - \frac{N+1}{2}$ with $\sum_i (i - \frac{N+1}{2})^2 = \frac{N(N^2-1)}{12}$, so $\max_i a_i^2 = \frac{(N-1)^2/4}{N(N^2-1)/12} = \frac{3(N-1)}{N(N+1)} \rightarrow 0$. Hence $\text{IC} / \sqrt{\text{Var}(\text{IC})} = \sqrt{N-1} \text{IC} \Rightarrow \mathcal{N}(0, 1)$.

Solution to Exercise 1.2. Standardize the score, $Z = s/\sigma_s$. Joint normality gives $\mathbb{E}[y \mid Z] = \rho \sigma_y Z$, so by the tower property the long leg's expected return is $\rho \sigma_y \mathbb{E}[Z \mid Z > z_{0.8}]$ and the short leg's is $\rho \sigma_y \mathbb{E}[Z \mid Z < z_{0.2}]$. With $z_{0.8} = 0.8416$, $\phi(z_{0.8}) = 0.2800$: $\mathbb{E}[Z \mid Z > z_{0.8}] = \phi(z_{0.8})/0.2 = 1.400$, and by symmetry the lower tail gives -1.400 . Hence $\mathbb{E}[\text{L-S}] = 2.80 \rho \sigma_y$. Numerically: $2.80 \times 0.03 \times 0.02 = 1.68 \times 10^{-3}$, i.e. ≈ 17 basis points per holding period, gross. Against Chapter 1's cost accounting (a fresh two-sided book turns over fully: $2 \times \text{turnover} \times 10 \text{ bp} = 4 \text{ bp}$ if fully rebuilt daily), the margin is real but thin — which is why turnover is a headline number, not a footnote.

Solution to Exercise 1.3. Core simulation (vectorized; ranks via double argsort):

```
rng = np.random.default_rng(0)
best_t = []
for rep in range(1000):
    ics = np.empty((20, 250))
    for m in range(20): # 20 strategies
        p = rng.standard_normal((250, 100))
        y = rng.standard_normal((250, 100))
        rp = p.argsort(1).argsort(1); ry = y.argsort(1).argsort(1)
        rp = rp - rp.mean(1, keepdims=True); ry = ry - ry.mean(1, keepdims=True)
```

```

ics[m] = (rp * ry).sum(1) / np.sqrt((rp**2).sum(1) * (ry**2).sum(1))
t = ics.mean(1) / (ics.std(1, ddof=1) / np.sqrt(250))
best_t.append(t.max())

```

Expected findings: each strategy's t is approximately standard normal under the null, so the best of $M = 20$ has $\mathbb{P}(\max t > 2) = 1 - \Phi(2)^{20} = 1 - 0.97725^{20} \approx 0.37$ — simulation lands near 37% — and the mean best- t sits near the $\sqrt{2 \ln 20} \approx 2.45$ heuristic. The best strategy's $\overline{\text{IC}}$ is about $1.87 \times 0.1 / \sqrt{250} \approx 0.012$ (expected maximum of 20 standard normals times the SE): a plausible-looking “signal,” manufactured from nothing, better than one date-per-week of real research projects. This is Remark 1.6 in numbers, and the reason Chapter 17's registry counts trials.

Solution to Exercise 1.4. (a) $T \geq (3 \times 0.12 / 0.02)^2 = 18^2 = 324$ dates. (b) If the IC series has autocorrelations $\rho_s = 1 - s/h$ for $s < h$ and 0 beyond (the linear-decay overlap profile), then for $T \gg h$,

$$\text{Var}(\overline{\text{IC}}) \approx \frac{\sigma_{\text{IC}}^2}{T} \left(1 + 2 \sum_{s=1}^{h-1} \left(1 - \frac{s}{h} \right) \right) = \frac{\sigma_{\text{IC}}^2}{T} (1 + (h-1)) = \frac{\sigma_{\text{IC}}^2}{T} h,$$

using $\sum_{s=1}^{h-1} (1 - s/h) = (h-1)/2$. The variance inflation factor is exactly h : overlapping 5-day labels make T dates worth $T/5$ independent ones. Corrected requirement: $324 \times 5 = 1,620$ dates — about 6.5 years of daily out-of-sample. (c) Any $n_{\text{folds}} \times \text{test_dates_per_fold} \geq 1,620$ works arithmetically, e.g. 9×180 ; on a 2,500-date panel this leaves ≈ 880 dates for the first fold's training window — the practical tension (evaluation appetite versus training data) that walk-forward design always negotiates, and a reason thesis-scale claims favor pooled multi-year evaluations over quarterly ones.

Solution to Exercise 1.5. (a) $\partial \text{NLL} / \partial (\sigma^2) = \frac{1}{2\sigma^2} - \frac{1}{2\sigma^4} \cdot \frac{1}{n} \sum_j e_j^2 = 0$ gives $\hat{\sigma}^2 = \frac{1}{n} \sum_j e_j^2$; substituting back yields $\frac{1}{2} \log \hat{\sigma}^2 + \frac{1}{2} \log 2\pi + \frac{1}{2}$. (b) $\Delta \text{NLL} = \frac{1}{2} \log(\hat{\sigma}_A^2 / \hat{\sigma}_B^2)$, so $\hat{\sigma}_A^2 / \hat{\sigma}_B^2 = e^{2 \times 0.021} = 1.043$: model A's residual variance is about 4.3% higher. (c) Two points with heteroskedastic truth. Model A predicts errors (0.1, 10) but knows its uncertainty, $\sigma = (0.1, 10)$: per-point NLL (dropping constants) = $[\log 0.1 + \frac{1}{2}] + [\log 10 + \frac{1}{2}] = 1.0$, MSE = 50. Model B predicts better, errors (0.05, 8), but claims $\sigma = 1$ everywhere: NLL = $\frac{1}{2}(0.0025) + \frac{1}{2}(64) \approx 32$, MSE = 32. B wins MSE $32 < 50$; A wins NLL $1.0 \ll 32$. The likelihood is rewarding *calibrated uncertainty* — being right about how wrong you will be — which is precisely the capacity Chapter 2 argues regime models must have, and MSE cannot see.

Solution to Exercise 1.6. (a) $\mathbf{x}_{\text{seq}}$ is $(n, 8, 2)$ and $\mathbf{x}_{\text{snap}}$ $(n, 3)$ for the test spec, $\mathbf{y}$ a separate $(n,)$ tensor; no feature column can equal the target because the target is not stored among features at all. (b) `split_by_date(0.8)` yields `max(train.date) < min(test.date)`; assert it directly. (c) The public surface offers `full_batch`, `minibatches` (within a panel already split), `date_batches`, `subset_dates`, `split_by_date` — every subsetting operation is keyed by date; row-level selection (`_take`, `_batch`) is private by underscore convention. The honest path is the paved one. `test_no_lookahead` pins the invariant of Definition 1.2: it rebuilds features after perturbing only *future* prices and asserts every feature at date t is unchanged while the forward-return target changes — look-ahead, if ever introduced, fails a test rather than improving a backtest.

Chapter 2

Solution to Exercise 2.1. (a) $\kappa - 3 = 3(\mathbb{E}[V^2] - (\mathbb{E}V)^2)/(\mathbb{E}V)^2 = 3 \text{Var}(V)/(\mathbb{E}V)^2$. (b) Let q be the weight of the volatile state and $r = \sigma_2^2/\sigma_1^2 = 25$. Then $\kappa(q) = 3 \frac{1+(r^2-1)q}{(1+(r-1)q)^2}$; setting $\kappa'(q) = 0$ gives $(r^2 - 1)(1 + (r - 1)q) = 2(r - 1)(1 + (r^2 - 1)q)$, i.e. $q^* = \frac{1}{r+1} = \frac{1}{26} \approx 3.8\%$, with $\kappa(q^*) = \frac{3(r+1)^2}{4r} \approx 20.3$. Rare volatile regimes maximize kurtosis because $\kappa - 3$ is the *relative* variance of the randomized variance V : a scarce, extreme value of V maximizes $\text{Var}(V)$ against a small mean, exactly as a rare large claim dominates an insurer's loss distribution. (c) The three numbers say: equal-weight regime mixing underexplains empirical tails (5.6 vs. 10–30), but *realistically proportioned* regimes — turbulence as the rare state — put the same machinery in range (≈ 20). Regime switching is thus a sufficient explanation for most of empirical kurtosis *only if* the volatile state is genuinely rare and distinct, which is also what the persistence estimates of Proposition 2.2 deliver (long calm spells, occasional storms); residual excess beyond that suggests within-regime non-normality or more than two states.

Solution to Exercise 2.2. (a) With $a = b$ the stationary law is $(\frac{1}{2}, \frac{1}{2})$; write $v \in \{\sigma_1^2, \sigma_2^2\}$. From the proof: $\text{Cov}(r_t^2, r_{t+s}^2) = \text{Cov}(v_t, v_{t+s}) = \text{Var}(v)\lambda^s$ and $\text{Var}(r_t^2) = 3\mathbb{E}[v^2] - (\mathbb{E}v)^2$, so $c = \text{Var}(v)/(3\mathbb{E}[v^2] - (\mathbb{E}v)^2)$. For $\sigma = (0.6, 2.2)$: $v \in \{0.36, 4.84\}$, $\mathbb{E}v = 2.6$, $\mathbb{E}v^2 = 11.778$, $\text{Var}(v) = 5.018$, so $c = 5.018/(35.333 - 6.76) = 5.018/28.573 = 0.176$; with $\lambda = 2(0.98) - 1 = 0.96$, the lag-10 autocorrelation is $0.176 \times 0.96^{10} = 0.117$. (b) Duration in state 1 is the number of steps until the first departure: $\mathbb{P}(D = n) = a^{n-1}(1 - a)$, geometric, $\mathbb{E}D = 1/(1 - a)$. (c) GARCH(1,1) persistence $\alpha + \beta$ absorbs λ : the fitted volatility process must reproduce an ACF decaying like 0.96^s , and $\alpha + \beta$ is the ACF decay rate of a GARCH's squared returns. This is the classic Lamoureux–Lastrapes point: ignored regime switching masquerades as near-integrated GARCH ($\alpha + \beta \approx 1$), so extreme persistence estimates can be an artifact of unmodeled states.

Solution to Exercise 2.3. (a) $\beta_{\text{pool}} = \mathbb{E}[\beta_S] = p\beta_1 + (1 - p)\beta_2$ (independence of x and S); since $\text{Var}(y) = \mathbb{E}[\beta_S^2] + \sigma_\varepsilon^2$ (unit-variance x), $R_{\text{pool}}^2 = (\mathbb{E}\beta_S)^2/(\mathbb{E}[\beta_S^2] + \sigma_\varepsilon^2)$. (b) Pooled residual: $y - \beta_{\text{pool}}x = (\beta_S - \mathbb{E}\beta_S)x + \varepsilon$, whose variance is $\text{Var}(\beta_S)\mathbb{E}[x^2] + \sigma_\varepsilon^2$ — exceeding the oracle's σ_ε^2 by $\text{Var}(\beta_S)\mathbb{E}[x^2]$. (c) Both results are the law of total variance with a latent index: unexplained dispersion of a latent *parameter* (variance σ_S^2 in Proposition 2.1, slope β_S here) reappears as inflation of an unconditional moment (kurtosis there, residual variance here). “Variance of the variance” produces fat tails; “variance of the slope” produces phantom noise — one phenomenon, two disguises, and the mixture model is the device that gives the latent index somewhere to live instead.

Solution to Exercise 2.4. Center and scale: $\mu_{1,2} = \pm\delta$, $\sigma = 1$. The density is proportional to $e^{-(x-\delta)^2/2} + e^{-(x+\delta)^2/2} = 2e^{-(x^2+\delta^2)/2} \cosh(\delta x)$. Setting the log-derivative to zero: $x = \delta \tanh(\delta x)$. $x = 0$ is always critical. The map $x \mapsto \delta \tanh(\delta x)$ has slope δ^2 at the origin and is strictly concave on $x > 0$; nonzero fixed points therefore exist iff $\delta^2 > 1$. Checking curvature, $x = 0$ is the unique maximum when $\delta \leq 1$ and becomes a local *minimum* flanked by two symmetric maxima when $\delta > 1$. In original units: bimodal iff $|\mu_1 - \mu_2| > 2\sigma$. Conclusion: components must be separated by more than two standard deviations before the mixture *looks* like a mixture; regime structure at realistic separations is invisible to the eye, so recovery must be graded against planted latent states (Chapter 14), not histograms.

Solution to Exercise 2.5. Sketch of expected results (exact values vary with seed): (a) sample kurtosis of the pooled series lands near the $3\mathbb{E}[V^2]/(\mathbb{E}V)^2 = 5.56$ prediction for equal-weight $\sigma = (0.5, 2.5)$ (the Markov chain with $a = b$ spends half its time in each state); the log-ACF of r_t^2 is linear in lag with slope $\log(0.96) \approx -0.041$, i.e. $\approx 4\%$ decay per day. (b) Pooled OLS slopes on the snapshot features are statistically indistinguishable from zero while within-regime regressions recover $\pm\beta$ with $R^2 = \beta^2/(\beta^2 + \sigma_\varepsilon^2)$; with `beta_scale = 2`, `noise_std = 0.4` the oracle R^2 exceeds 0.9 — the full Proposition 2.3 contrast. (c) With `regime_process="iid"` the squared-return ACF collapses to zero at all positive lags while the kurtosis is unchanged — mixing makes tails, persistence makes memory, exactly the division of Sections 2.2.

Chapter 3

Solution to Exercise 3.1.

```
z = torch.tensor([0.2, 1.0, -0.5], requires_grad=True)
a, b = torch.tensor(3.0), torch.tensor(-3.0)
pred = torch.where(torch.softmax(z, 0).argmax() == 1, a, b)
pred.pow(2).backward()
print(z.grad)  # -> None (no graph reaches z), or zeros if z re-enters
```

`argmax` returns an integer with no gradient function, and `where` on that condition selects between tensors that do not depend on z ; the loss is a piecewise-constant function of z , whose derivative is zero wherever it exists. Replacing the selection with $\hat{y} = \sum_k \pi_k \mu_k ((\text{torch.softmax}(z, 0) * \text{torch.stack}([b, a, b])).\text{sum}())$ routes real gradient into every logit: $\partial \hat{y} / \partial z_k = \pi_k (\mu_k - \hat{y})$, alive whenever the experts disagree. The package's `test_gate_receives_gradient` asserts the second fact; its `argmax` companion asserts the first — pinning both the defect and the repair.

Solution to Exercise 3.2. (a) Each LSTM gate $g \in \{i, f, o, \tilde{c}\}$ computes $g = \phi(W_g x_t + U_g h_{t-1} + b_g)$ with $W_g \in \mathbb{R}^{n \times d}$, $U_g \in \mathbb{R}^{n \times n}$, $b_g \in \mathbb{R}^n$: four gates give $4(n(n+d)+n)$. The GRU has three such blocks (r, z , candidate): $3(n(n+d)+n)$. (b) Prototype, $d = 5$: layer 1, $4(256 \cdot 261 + 256) \approx 268\text{K}$; layers 2–8 have input dimension 256: $4(256 \cdot 512 + 256) \approx 525\text{K}$ each, $\times 7 \approx 3.68\text{M}$; total $\approx 3.95\text{M}$. Corrected, GRU $n = 32$, $d = 2$: $3(32 \cdot 34 + 32) = 3,360$. Three orders of magnitude apart. (c) (i) All rows: 1.25M samples $\rightarrow$ budget $\approx 125\text{K}$ parameters. (ii) $N_{\text{eff}} = 20$: 50K effective samples $\rightarrow$ budget $\approx 5\text{K}$. The prototype exceeds even the naïve budget 30 $\times$ and the discounted one $\approx 800\times$; the corrected encoder fits *inside* the pessimistic budget. The point of the exercise is that this verdict required no training run — it is design-time arithmetic.

Solution to Exercise 3.3. Build eight samples for date 1 ($\mathcal{N}(0, 1)$ features) and eight for date 2 with a common +5 shift. `BatchNorm1d` in train mode, forward on date 1 alone versus on the concatenated batch: date-1 outputs differ (the batch mean absorbs date 2's shift — with the shift included, the batch mean rises by 2.5, so date-1 samples are pushed down), a back-in-time flow of date-2 information: consequence (iii), leakage, via consequence (ii), batch-composition sensitivity. Switching the layer to `eval()` changes date-1 outputs yet again (running statistics replace batch statistics): consequence (i), train/eval asymmetry. `LayerNorm` normalizes each sample by *its own* feature moments, so date-1 outputs are bit-identical with or without date 2 present — `torch.equal` returns `True` — and there is nothing to leak.

Solution to Exercise 3.4. With true signal $R^2 = 0.01$: model (i) scores in- and out-of-sample $R^2 \approx 0.01$. Model (ii) — target among its inputs — scores $R^2 \approx 1.0$ *both* in-sample and out-of-sample, because the pipeline that packs the target into the last column does so for test data too: the leak follows the data, and no holdout detects it. That is the chilling part: ordinary validation is defenseless against a layout bug. Appending an innocent feature after the reserved column shifts the slice: model (i)’s “correct” `[:, :-1]` now *includes* the target (instant $R^2 \approx 1$) while dropping a real feature — one appended column flipped a correct model into a leaking one without touching its code. The two-line defense:

```
if x.shape[-1] != len(schema.feature_names):
    raise ValueError(f"width {x.shape[-1]} != schema {len(schema.feature_names)}")
```

plus the structural fix of never storing the target among features — at which point all three configurations fail loudly at construction, which is the only acceptable behavior.

Solution to Exercise 3.5. (a) Leakage channel (near-duplicate rows with overlapping labels cross the split): defense — a date-keyed splitting API that offers no random row split. (b) Leakage channel (test-period moments enter the transform): defense — fit-on-train-only as a pipeline contract, pinned by a shifted-history invariant test. (c) Leakage channel (future positions visible in training): defense — a causality test asserting predictions at t are invariant to perturbations of inputs after t (the package runs exactly this for the HMM filter). (d) Contract failure (three copies of one truth): defense — a single validated config tree where derived quantities are computed, not restated. (e) Leakage channel (reward peeks at prices the state already contains): defense — point-in-time discipline applied to the *reward* path, same invariant test family as (b).

Solution to Exercise 3.6. (Model answers; yours may be better.) *The brief against:* the skeleton multiplies the parameter and failure-mode budget of a linear baseline by an order of magnitude — an encoder, a gate, K experts, a multimodal likelihood with symmetric local optima and permutation ambiguity, and (Chapter 1) a target whose predictable component is $R^2 \sim 10^{-3}$; a convex, deterministic, one-line ridge regression captures most linear structure at near-zero variance cost, trains in milliseconds, cannot be mis-optimized, and by Proposition 2.3 the mixture’s *categorical* advantage exists only if sign-flipping state-dependence is real and strong — an empirical bet, not a theorem, and the burden of proof sits on the expensive model. *The rebuttal:* the bet is precisely what the system is built to settle — cheaply and fairly: capacity-matched baselines remain permanently in the comparison, planted-truth worlds verify both that the mixture wins where regimes exist and that the baselines win where they don’t, and the evaluation protocol (purged walk-forward, multiplicity-corrected) prices the extra failure modes rather than hiding them. Keeping the skeleton is defensible *because* the referee is already hired; keeping it on enthusiasm alone would not be. Part V holds the whistle.

Chapter 4

Solution to Exercise 4.1. (a) Fix k and differentiate $\sum_j r_{jk} \log \mathcal{N}(y_j; \mu_k, \sigma_k^2)$: the μ_k equation $\sum_j r_{jk}(y_j - \mu_k)/\sigma_k^2 = 0$ gives the weighted mean; the σ_k^2 equation $\sum_j r_{jk}(-\frac{1}{2\sigma_k^2} + \frac{(y_j - \mu_k)^2}{2\sigma_k^4}) = 0$

gives the weighted mean square. (b) With $R_k = \sum_j r_{jk}$, maximize $\sum_k R_k \log \pi_k + \lambda(1 - \sum_k \pi_k)$: $\partial_{\pi_k} = R_k/\pi_k - \lambda = 0$, so $\pi_k = R_k/\lambda$; the constraint forces $\lambda = \sum_k R_k = n$, hence $\pi_k = R_k/n$. (c) If $R_k = 0$ the weight update sends $\pi_k \rightarrow 0$ and the component's moment equations become $0/0$ — the M-step is undefined in the direction of a dead expert. The dashboard's running-utilization minimum is the early-warning system for exactly this state: an expert whose assignment mass decays toward zero is exiting the model, and whether that is specialization (fine) or starvation (a failure) is a judgment the monitoring exists to force *before* the arithmetic degenerates.

Solution to Exercise 4.2. Write $\ell(\theta) = \mathcal{L}(q, \theta) + \text{KL}(q \| r(\theta))$ and differentiate in θ at θ_0 with $q = r(\theta_0)$ held fixed. The map $\theta \mapsto \text{KL}(q \| r(\theta))$ is smooth, non-negative, and attains its minimum value 0 at $\theta = \theta_0$; a smooth function is stationary at an interior minimum, so its θ -gradient vanishes there. Hence $\nabla_{\theta} \ell(\theta_0) = \nabla_{\theta} \mathcal{L}(q, \theta)|_{\theta_0}$, and by (4.6) the latter is the gradient of the expected complete-data log-likelihood — Fisher's identity again. Since one gradient step on $\mathcal{L}(q, \cdot)$ from θ_0 increases it (small step), and $\mathcal{L}$ minorizes ℓ , the step increases ℓ : a GEM step.

Solution to Exercise 4.3. With equal, unit scales the responsibility ratio reduces to $r_{j1} = 1/(1 + e^{(\mu_2 - \mu_1)(y_j - \frac{\mu_1 + \mu_2}{2})})$; for the symmetric states here, $r_{j1} = 1/(1 + e^{2\mu_2 y_j})$. *Iteration 1* ($\mu = (-0.5, 0.5)$): $r_{\cdot 1} = (0.881, 0.731, 0.269, 0.119)$ for $y = (-2, -1, 1, 2)$; weighted mean $\mu'_1 = (0.881(-2) + 0.731(-1) + 0.269 + 0.119(2))/2.0 = -0.993$, and by symmetry $\mu'_2 = +0.993$. *Iteration 2*: $r_{\cdot 1} = (0.981, 0.880, 0.120, 0.019)$, giving $\mu'' = (-1.343, +1.343)$. The iteration is climbing toward the symmetric fixed point near ± 1.5 (each mean the soft-weighted center of its half of the data), sharpening as it separates. From $\mu = (0, 0)$: the components are identical, so $r_{j1} = \frac{1}{2}$ for every j and remain so forever; both means update to the pooled mean 0 and the algorithm is converged at the symmetric stationary point — Chapter 8's trap, reached in zero iterations, with the likelihood reporting smooth “convergence” throughout.

Solution to Exercise 4.4. The $j = 1$ term is $\log(\frac{\pi_1}{\sqrt{2\pi}\epsilon} + c_1) = \log(1/\epsilon) + O(1)$; all other terms are ϵ -independent up to the vanishing contribution of component 1, so $\ell(\epsilon) = \log(1/\epsilon) + O(1)$. To beat an interior optimum by Δ nats requires $\epsilon \lesssim C e^{-\Delta}$: every extra nat of “improvement” costs a factor e of collapse — the spike is a cliff approached logarithmically slowly, which is why early training (bad means, large residuals everywhere except by luck) is the window of vulnerability. In the conditional model, $\mu_k(\cdot)$ is one function shared by all samples: parking it on y_1 at x_1 constrains its values at nearby inputs, and — more binding — σ_k is global to the expert, so shrinking it punishes *every* sample the expert still claims. The spike now requires the expert to simultaneously memorize one point and shed responsibility everywhere else: reachable in principle, but no longer downhill from generic initializations. Harder, not impossible — hence the belt (`sigma_init`) and suspenders (`sigma_freeze_steps`).

Solution to Exercise 4.5. Expected findings. (a) The per-iteration ℓ differences are non-negative to machine precision (assert $\geq -10^{-10}$); plot them and watch the geometric tail of EM near convergence. (b) Both routes land at $\hat{\sigma} \approx (0.5, 2.5)$ up to permutation and sampling error; the package run needs the uniform prior (so the comparison is weights-fixed, like your numpy EM with π frozen) and benefits from the moment warm start for symmetry's sake. (c) EM converges in tens of iterations and milliseconds — closed forms, full-batch, no learning rate; SGD

takes hundreds of steps and hyperparameters. What SGD buys is exactly what this problem doesn't need and the thesis model cannot live without: emissions with no closed-form M-step, minibatch scaling, and one code path from the toy to the full model. The comparison is the cheapest possible demonstration that the package's trainer is "EM by other means," and that the other means are paid for in tuning.

Solution to Exercise 4.6. The E-step: $\text{KL}(q\|r) \geq 0$ with equality iff $q = r$, so the minimizer over unrestricted q is the posterior. For n independent observations the joint posterior over $z_{1:n}$ factorizes, $p(z_{1:n} | y_{1:n}, \theta) = \prod_j p(z_j | y_j, \theta)$, so restricting q to product form excludes nothing: coordinate ascent within the mean-field family attains the exact per-observation posteriors and the bound still touches. For the HMM, the latent states are *a posteriori* dependent through the chain (observing y_{t+1} moves beliefs about S_t), so the product family no longer contains the true posterior, mean-field E-steps leave a strictly positive KL gap, and exact inference requires the chain-respecting recursions of Chapter 7 — which is precisely why the forward filter, not a factorized approximation, sits inside the Variation-3 trainer.

Chapter 5

Solution to Exercise 5.1.

```
z = torch.randn(1, 3, requires_grad=True)
mu = torch.randn(1, 3); y = torch.randn(1)
log_pi = torch.log_softmax(z, -1)
log_lik = -0.5*math.log(2*math.pi) - 0.5*(y.unsqueeze(-1) - mu)**2
nll = -torch.logsumexp(log_pi + log_lik, -1).mean()
nll.backward()
r = torch.softmax((log_pi + log_lik).detach(), -1)
assert torch.allclose(z.grad, torch.softmax(z, -1).detach() - r)
```

The μ and $\log \sigma$ checks are the same pattern against (5.4)–(5.5). Replacing the mixture with `torch.where(z.argmax(-1, keepdim=True) == torch.arange(3), ...)` routing and back-propagating any loss of the selected output leaves `z.grad` None/zero: the graph from loss to `z` passes only through an integer. Both halves of the package's defect-1 regression test, reproduced.

Solution to Exercise 5.2. $\sum_k (\pi_k - r_k) = 1 - 1 = 0$. Adding c to every logit leaves softmax unchanged, so the loss is constant along the all-ones direction and its gradient must be orthogonal to it — the sum-zero property is the differential form of shift invariance. Consequently the gate learns only logit *differences*; the common level is unconstrained by the objective and would drift as a random walk under noisy gradients. In the package the drift is bounded benignly: the gate's LayerNorm standardizes the head's input scale, and weight decay on the gate's linear weight (the one decayed parameter group, Chapter 9's optimizer hygiene) shrinks the map — together anchoring the logit scale without touching the differences that carry routing.

Solution to Exercise 5.3. Condition on the component index Z : $\text{Var}(y) = \mathbb{E}[\text{Var}(y | Z)] + \text{Var}(\mathbb{E}[y | Z]) = \sum_k \pi_k \sigma_k^2 + (\sum_k \pi_k \mu_k^2 - \hat{y}^2)$. Balanced case: $\hat{y} = 0$, within = 1, between = 1: total 2, half of it regime disagreement. Confident case: $\hat{y} = 0.9$, within = 1, between = $1 - 0.81 = 0.19$. The balanced case is the one the risk manager must hear about — the model itself is saying

“this name’s expected return is +1 or −1 and I cannot tell which,” a coin flip with magnitude — and $\hat{y} = 0$ alone would file it, indistinguishably, next to a name the model finds genuinely neutral. Only the decomposition separates “no signal” from “two strong opposed signals,” which is the practical content of carrying a density.

Solution to Exercise 5.4. At fixed $\mathbf{h}$ the randomness left is in y (through r); taking expectations in (5.3): $\mathbb{E}[\partial \text{NLL} / \partial \mathbf{z} \mid \mathbf{h}] = \pi(\mathbf{h}) - \mathbb{E}[r \mid \mathbf{h}]$. Stationarity of the expected gradient at every reachable $\mathbf{h}$ is exactly $\pi(\mathbf{h}) = \mathbb{E}[r \mid \mathbf{h}]$: the prior equals the conditionally-averaged posterior — a calibration identity, learned because the gradient is its residual. Chapter 18’s reliability diagram is this identity evaluated on held-out data: bin dates by gate confidence, compare against average realized responsibility. Training enforces the identity in-sample at convergence, so an out-of-sample violation indicates the $\mathbf{h} \mapsto$ regime relationship shifted between fit and test — drift, the environment’s signature (Section 1.3) — and the honest response is recalibration or refit, not a debugger.

Solution to Exercise 5.5. With $\mu_k = \mu_k(\mathbf{x}; \mathbf{h})$, the chain rule through both consumers of $\mathbf{h}$ gives

$$\frac{\partial \text{NLL}}{\partial \mathbf{h}} = \underbrace{\left(\frac{\partial \mathbf{z}}{\partial \mathbf{h}} \right)^\top (\pi - r)}_{\text{gate path}} + \underbrace{\sum_k r_k \frac{\mu_k - y}{\sigma_k^2} \frac{\partial \mu_k}{\partial \mathbf{h}}}_{\text{expert path}}.$$

In the prototype, defect 1 zeroed $\pi - r$ ’s route (no gradient through the routing) and defect 6 removed the expert path at the source (experts read a raw slice, not $\mathbf{h}$): total encoder gradient from the prediction task, *exactly zero* — four million parameters trained by nothing but an auxiliary classification loss, if that. The corrected default keeps the expert path severed *on purpose*: snapshot-only experts mean the encoder’s sole gradient is the gate path, so its learned representation answers one question — “what in the recent sequence helps predict which regime we are in?” — rather than a blend of regime detection and per-name curve fitting. Interpretability of the encoder is a property of the gradient topology, decided by `input_mode` in config: Decision A, stated as calculus.

Solution to Exercise 5.6. The committee gradient is identical for both experts, $\partial L / \partial b_i = -\mathbb{E}[(y - \frac{b_1+b_2}{2}x)x] / 1 = \frac{b_1+b_2}{2} \mathbb{E}[x^2]$ (using $\mathbb{E}[yx] = 0$ on the sign-flip world). Gradient descent therefore drives the *sum* $b_1 + b_2 \rightarrow 0$ while leaving the *difference* $b_1 - b_2$ untouched: from (0.8, −0.6) the blend converges to slopes $\approx (0.7, -0.7)$ — residual asymmetry frozen where it started, no force growing it toward $\pm\beta = \pm 1.5$, and the ensemble prediction $\frac{1}{2}(b_1 + b_2)x \rightarrow 0$: functionally the pooled model. The mixture NLL from the same start produces informative responsibilities (the $b_1 > 0$ expert better explains $S = +1$ samples), whose weighting pushes the slopes *apart* until they reach $\pm\beta$. The precise moral is sharper than “cooperation homogenizes”: cooperation leaves specialization *neutrally stable* — unrewarded, unrestored, eroded by any noise — while the likelihood’s competition makes specialization an attractor. Architecture identical; objective decides.

Chapter 6

Solution to Exercise 6.1. With $m = \max_k a_k$: $e^m \leq \sum_k e^{a_k} \leq Ke^m$; take logs. Extremes: one term dominating (gap $\rightarrow \infty$) gives $\text{LSE} \rightarrow m$; all terms equal gives $\text{LSE} = m + \log K$. The hard surrogate is thus at most $\log 2 \approx 0.69$ nats loose for $K = 2$ but $\log 8 \approx 2.08$ for $K = 8$ — and the slack is largest exactly when components disagree least, i.e. early in training, which compounds hard routing’s cold-start fragility as K grows.

Solution to Exercise 6.2. For fixed k^* , only $\log \pi_{k^*} = z_{k^*} - \text{LSE}(z)$ depends on z (the selected log-likelihood does not), giving $\partial(-\log \pi_{k^*})/\partial z_k = \pi_k - \delta_{kk^*}$; the expected update raises z_{k^*} and lowers the rest — reinforcement of the incumbent. With frozen experts the dynamics are pure self-training: any state whose argmax is k^* flows toward higher confidence in k^* ; stationarity requires $\pi = e_{k^*}$, approached asymptotically, never crossed. Since a cold gate is near-uniform, its initial argmax is decided by initialization noise, and that arbitrary choice is then amplified — the generic outcome, not misfortune. The escape routes are exactly the ones the likelihood term provides in full training: a selected expert that explains its samples *worse* than a rival shifts which branch wins on those samples, which is why hard-EM works at all — and why it works best warm.

Solution to Exercise 6.3.

```
z = torch.tensor([[1.0, 0.4, -0.2]], requires_grad=True)
keep = torch.zeros_like(z, dtype=torch.bool).scatter_(
    1, z.topk(k, -1).indices, True)
log_pi = torch.log_softmax(z.masked_fill(~keep, float("-inf")), -1)
nll = -torch.logsumexp(log_pi + log_lik, -1).mean(); nll.backward()
```

$k = 1$: z .grad is exactly zero (the kept entry’s renormalized log-weight is $\log 1 = 0$, a constant);
 $k = 2$: nonzero gradient in the two kept coordinates. No smooth repair exists downstream because the composite map $z \mapsto \tilde{\pi}$ is *constant* on each region of fixed argmax: whatever differentiable machinery consumes $\tilde{\pi}$ is consuming a locally constant function of z , and the chain rule multiplies by its zero Jacobian. The information “how much better was the winner than the runner-up” — the only z -content that could carry gradient — was destroyed by normalizing a singleton. Retention of at least two competitors is not a tuning preference; it is the existence condition for the training signal.

Solution to Exercise 6.4. Density: $\frac{d}{dt} e^{-e^{-t}} = e^{-t} e^{-e^{-t}}$. Conditioning: independence of the g_l makes the win probability given $g_j = t$ a product of CDFs at the shifted points $z_j - z_l + t$. Including $l = j$ multiplies by $e^{-e^{-t}}$, which is exactly the density’s own exponential factor — that bookkeeping is what collapses the product into $\exp(-c_j e^{-t})$ with $c_j = \sum_l e^{-(z_j - z_l)}$. Substituting $u = e^{-t}$ turns the integral into $\int_0^\infty e^{-c_j u} du = 1/c_j = e^{z_j} / \sum_l e^{z_l}$. Shifting all logits by c shifts every perturbed logit equally, leaving every pairwise comparison — hence the argmax law — unchanged, the sampling counterpart of softmax shift invariance; and Exercise 5.2 is its differential shadow: a loss invariant along the all-ones direction has sum-zero gradients. One symmetry, three appearances.

Solution to Exercise 6.5. (a) The difference of two independent Gumbels is standard logistic, so $\pi_1^{(\tau)} = \sigma((\Delta z + \Delta g)/\tau)$ with Δg logistic. As $\tau \rightarrow 0$ the sigmoid hardens to $\mathbf{1}\{\Delta z + \Delta g > 0\}$, whose expectation is $\mathbb{P}(\Delta g > -\Delta z) = \sigma(\Delta z)$ — exact categorical sampling, the lemma again. As $\tau \rightarrow \infty$ the argument collapses to 0 and $\mathbb{E} \rightarrow \frac{1}{2}$. (b) Simulation shows the gradient $\sigma'(\cdot)/\tau$ is negligible for most draws (saturation) but of size $\sim 1/(4\tau)$ for draws near the boundary: the variance of the per-sample gradient grows roughly like $1/\tau$ — few, huge contributions. The anneal walks from low-variance/high-bias (warm, blended) toward low-bias/high-variance (cold, near-discrete), spending the variance budget only once routing is roughly right. (c) With noise on, the held-out NLL is an average over injected randomness whose magnitude depends on τ : two models at different temperatures differ in *evaluation-time noise*, not only in fit, and the comparison confounds the two. Deterministic eval ($\log_{\text{softmax}}(z)$, the package’s eval branch) grades what was learned.

Solution to Exercise 6.6. (a) $f_k = 1/K$ gives $\mathcal{L}_{\text{lb}} = K \sum_k \bar{p}_k / K = \sum_k \bar{p}_k = 1$ for *any* gate matrix — rows may be arbitrarily one-hot; only the marginal enters. (b) Expected shape of results. Unbalanced, no aux: the head-started expert’s utilization climbs to ≈ 1 , its rival’s responsibilities and gradients decay together — rich-get-richer to starvation. Per-batch f on regime-pure batches: utilization balances *within every batch*, which on one-sided batches means the gate is punished into $\approx (0.5, 0.5)$ on dates where the truth is one-sided — sharp routing destroyed, the gate drifts toward uniform. Buffered f across both regimes: the marginal balances (the starving expert is rescued early) while per-date routing stays sharp — calm dates route calm, turbulent dates route turbulent, and the buffer’s window is what made “balanced overall, decisive locally” expressible. The package’s default then turns the rescued mechanism *off* anyway — an ablation lever, not baseline seasoning — and the buffer keeps running for the dashboard.

Chapter 7

Solution to Exercise 7.1. $\beta_t(j) = p(\mathbf{y}_{t+1:T} \mid S_t = j) = \sum_k \mathbb{P}(S_{t+1} = k \mid S_t = j) p(\mathbf{y}_{t+1} \mid S_{t+1} = k) p(\mathbf{y}_{t+2:T} \mid S_{t+1} = k) = \sum_k A_{jk} \ell_{t+1}(k) \beta_{t+1}(k)$, with $\beta_T \equiv 1$ (empty product). Then $\mathbb{P}(S_t = k \mid \mathbf{y}_{1:T}) \propto \mathbb{P}(S_t = k \mid \mathbf{y}_{1:t}) p(\mathbf{y}_{t+1:T} \mid S_t = k) = r_t(k) \beta_t(k)$, normalized by $\sum_j r_t(j) \beta_t(j)$ (which equals $p(\mathbf{y}_{t+1:T} \mid \mathbf{y}_{1:t})$). The recursion for β_t starts at T and runs *backward*: computing any smoothed marginal requires every future date’s emission likelihood — look-ahead is not a side effect of smoothing, it is its definition.

Solution to Exercise 7.2. Take stay probability 0.9, $\sigma = (1, 3)$, $\pi_0 = (\frac{1}{2}, \frac{1}{2})$, $y_1 = 0$, $y_2 = 6$. Filtered at $t = 1$: $\ell_1 = (0.399, 0.133)$, so $r_1 = (0.75, 0.25)$ — the quiet day favors the calm state. Backward: $\ell_2(1) = \mathcal{N}(6; 0, 1) \approx 6.1 \times 10^{-9}$, $\ell_2(2) = \mathcal{N}(6; 0, 9) \approx 0.018$; $\beta_1(1) = 0.9 \ell_2(1) + 0.1 \ell_2(2) \approx 1.8 \times 10^{-3}$, $\beta_1(2) = 0.1 \ell_2(1) + 0.9 \ell_2(2) \approx 1.62 \times 10^{-2}$. Smoothed $\propto r_1 \cdot \beta_1 = (1.35, 4.05) \times 10^{-3}$: $\mathbb{P}(S_1 = 2 \mid \mathbf{y}_{1:2}) = 0.75$. One loud tomorrow flipped the belief about a quiet today from 75:25 calm to 75:25 turbulent. A backtest positioning at date 1 on the smoothed belief has classified date 1 using date 2’s crash — it traded on tomorrow’s news, in one arithmetic line.

Solution to Exercise 7.3. Chain through $\pi_t(k) = \sum_j A_{jk} r_{t-1}(j)$ and $A = \text{row_softmax}(L)$: $\partial(-\log \sum_k \pi_t \ell_t) / \partial \pi_t(k) = -r_t(k) / \pi_t(k)$, $\partial \pi_t(k) / \partial A_{jk} = r_{t-1}(j)$, and $\partial A_{jk} / \partial L_{jm} = A_{jk} (\delta_{km} - A_{jm})$.

Assembling,

$$\frac{\partial(-\log p_t)}{\partial L_{jm}} = -r_{t-1}(j) A_{jm} \left(\frac{r_t(m)}{\pi_t(m)} - \sum_k A_{jk} \frac{r_t(k)}{\pi_t(k)} \right).$$

Reading: $r_t(k)/\pi_t(k)$ is destination k 's *surprise ratio* (posterior over prior — how much better than expected k explained today). Row j 's update moves mass toward destinations whose surprise beats the row's A_j -weighted average surprise, all scaled by yesterday's belief $r_{t-1}(j)$ in the source. The $\pi - r$ pattern reappears within the row: defining the row's "posterior" $\tilde{r}_j(m) \propto A_{jm} r_t(m)/\pi_t(m)$ (which is $\mathbb{P}(S_t = m \mid S_{t-1} = j, \mathbf{y}_{1:t})$), the bracket is proportional to $\tilde{r}_j - A_j$ — transitions are trained toward their own conditional posterior, exactly as the gate was.

Solution to Exercise 7.4. With constant (μ_k, σ_k) , (7.1) is a product of fixed Gaussians and (7.4) is precisely the prediction-error-decomposed likelihood Hamilton maximized (he ran the same filter; his numerical maximization is our gradient ascent on L, μ, σ). The package's recovery test generates a sticky two-state scale mixture, fits `classical × hmm`, and checks $\hat{A}$'s diagonal and $\hat{\sigma}$'s ordering — a synthetic Hamilton replication run by the same Trainer as the neural model. Rank-IC is meaningless because per date every name receives the same prediction $\sum_k \pi_t(k) \mu_k$ — a constant cross-section has no ranking; the harness raises `ValueError` with exactly this explanation, and the model is graded on held-out NLL, filtered-state AUC against the planted regime, and parameter recovery.

Solution to Exercise 7.5. The 20-line filter is the chapter's (7.2)–(7.3) with `logaddexp`; agreement with `evaluate_sequence` lands at 10^{-6} -level (float32 vs float64) once the same parameters are loaded. The causality perturbation — flip the final date's returns — changes the last filtered entry only; every earlier row is bit-identical (`np.array_equal`), because the recursion consumed only past values. That bitwise assertion, not a statistical one, is the correct form for a causality test: leakage is a structural property of the computation graph, and the package's test asserts it structurally.

Solution to Exercise 7.6. (a) Detaching modifies only the backward graph; forward values are computed identically, so filtered beliefs, predictions, and NLLs are unchanged by C . (b) The transition logits: their gradient's value comes from multi-date chains — belief shaped at t paying off at $t + s$ — and truncation at C discards all credit at ranges beyond the chunk. Plant a chain with high persistence and *weak per-date evidence* (small vol separation), so correct filtering must integrate evidence across many dates; measure $\hat{A}$'s stay-probability bias as C shrinks. Expected signature: persistence underestimated for small C , recovering as C passes the evidence-integration length. (c) Mean-per-date normalization makes each chunk's loss (and gradient scale) independent of its length, so the ragged final chunk neither dominates nor vanishes — a two-character choice (mean vs sum) that determines whether late-sample dates are systematically over- or under-weighted.

Chapter 8

Solution to Exercise 8.1. At $(b_1, b_2) = (0, 0)$ with a uniform gate, autograd returns gate gradients that are *exactly* zero (the equality $r = \pi$ is algebraic, not statistical) and expert gradients equal to $-\frac{1}{2}\widehat{\mathbb{E}}[yx]/\sigma^2$ — zero only up to sampling noise. Quantify: $\text{Var}(yx) = \beta^2\mathbb{E}[x^4] + \sigma_\varepsilon^2 = 3\beta^2 + \sigma_\varepsilon^2 \approx 6.9$ for $\beta = 1.5, \sigma_\varepsilon = 0.4$, so the empirical moment’s sd is $\sqrt{6.9/n}$ — at $n = 4,000$ about 0.04, comfortably distinguishable from a genuine gradient of order β . The instructive contrast: one gradient vanishes identically (structure), the other statistically (sampling) — and only the first kind defines the trap.

Solution to Exercise 8.2. With $b_1 = b_2 = b$ the densities coincide, so $r(\mathbf{h}) = \pi(\mathbf{h})$ pointwise and the gate is frozen wherever it is. The expert gradients become $\partial/\partial b_i = \mathbb{E}[\pi_i(\mathbf{h})(bx - y)x]/\sigma^2$: *exactly* proportional only when π is constant across samples (uniform gate); for a frozen $\mathbf{h}$ -dependent gate, $\frac{d}{dt}(b_1 - b_2) \propto \mathbb{E}[(\pi_1(\mathbf{h}) - \pi_2(\mathbf{h}))(bx - y)x]$, which is nonzero only insofar as the *untrained* gate’s fluctuations correlate with the regime-flipped residual moment. At random initialization that correlation is $O(\text{init scale})$ — so the exact invariant set is “equal experts + gate uncorrelated with regime,” and its neighborhood exhibits the same slow-bootstrap escape as Section 8.3: the refinement softens the theorem and preserves the phenomenon. Within the invariant set the gate converges to nothing at all (zero gradient), which is what makes the collapse quiet: a frozen gate looks identical to a converged one in the loss curve.

Solution to Exercise 8.3. The transverse dynamics near the diagonal are $\dot{\delta} \approx c \delta$ (c set by the responsiveness of r to expert separation), so escape to a fixed threshold takes $t \approx \frac{1}{c} \log(\beta/\delta)$: the log-log plot of escape steps against δ is linear with slope $-1/c$ in $\log \delta$ — each decade of smaller initial asymmetry costs a *constant additive* number of steps. The simulation shows exactly this line (with saturation at large δ , where the linearization fails), making the abstract claim “seeds are ε -symmetric, hence plateau-bound” quantitative: a seed-scale $\delta \sim 10^{-2}$ costs several times the entire training budget of the warm-started run.

Solution to Exercise 8.4. Slice k ’s closed-form warm start is $\hat{\mu}_k = \bar{y}_{(k)}, \hat{\sigma}_k^2 = \overline{(y - \bar{y})^2}_{(k)}$. In the scale world the sort key (window vol) is positively correlated with the latent state, so the top slice over-represents the turbulent regime: $\mathbb{P}(\text{high} \mid \text{top slice}) > \mathbb{P}(\text{high} \mid \text{bottom slice})$, and each slice’s expected sample variance is the corresponding mixture of $\sigma_1^2 < \sigma_2^2$ — ordered strictly, with the gap governed by the proxy-state correlation and concentrating by the law of large numbers as the window grows. The property doing *all* the work is that single positive correlation: the proxy need not identify the regime, only tilt the slices — an importantly weak requirement, since any observable that genuinely identified the regime would raise the question of why the model is needed.

Solution to Exercise 8.5. Exact symmetries: the $K!$ expert relabelings (acting jointly on gate output coordinates, emissions, scales, and for Variation 3 on π_0 and both axes of A); within each expert MLP, hidden-unit permutations ($h!$ per layer) and, for ReLU units, positive input-output rescalings — a continuous symmetry. The intra-expert symmetries are harmless for reporting: no reported statistic attaches to a hidden unit, and two weight-space-different but function-identical experts produce identical tables. The dangerous group is exactly the one

whose orbits cross *reported labels* — expert relabeling, because per-expert σ_k , utilization, and alignment rows are published. The σ -sort canonicalization picks one representative per orbit of that group (well-defined off the measure-zero tie set), which is precisely sufficient: canonicalize what you report, ignore what you don't.

Solution to Exercise 8.6. Static statistics all admit false positives or negatives: output correlation ≈ 1 and equal $\hat{\sigma}$ are the *correct* end state on regime-free data (the mixture should collapse there); responsibility-proxy AUC needs a proxy; gate-gradient norm vanishes at any convergence. The detector hardest to fool is therefore *paired and counterfactual*: declare collapse when (i) the mixture's held-out NLL is statistically indistinguishable from the pooled single-model baseline's, *and* (ii) a warm-started refit on the same window improves held-out NLL by a pre-registered margin. Condition (ii) is the crux — it distinguishes “the world is regime-free” (warm start helps nothing; detector silent, correctly, on the regime-free synthetic world) from “the optimizer sat down at the saddle” (warm start unlocks the improvement; detector fires, as it must on the sign-flip world's plain-seed runs). Collapse, properly defined, is not a property of the trained model alone but of the model *relative to what training could have found* — which is why the honest detector must run the counterfactual.